

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
Факультет інформаційних технологій
Кафедра інтелектуальних технологій**

**ВИПУСКНА КВАЛІФІКАЦІЙНА РОБОТА
БАКАЛАВРА
НА ТЕМУ**

**«Інтелектуальний додаток ідентифікації захворювань
сільськогосподарських рослин на основі
глибинного навчання»**

Галузь знань **12 «Інформаційні технології»**

Спеціальність **122 «Комп'ютерні науки та інформаційні технології»**

Освітня програма **«Комп'ютерні науки»**

Освітній рівень: бакалавр

Виконала: студентка 4 курсу, групи КН- 41,
Антоневич М. М.
(прізвище та ініціали)

Керівник: Доманецька І. М.
(прізвище та ініціали)
к.т.н., доцент
(науковий ступінь, звання)

Випускна кваліфікаційна робота бакалавра допущена до захисту
рішенням кафедри *інтелектуальних технологій*
Протокол № від р.
В.о. зав. кафедри _____ доц. Красовська Г.В.

Київ - 2020

АНОТАЦІЯ

Дана кваліфікаційна робота присвячена розробці засобів автоматизованої інтелектуальної ідентифікації захворювань сільськогосподарських рослин. У роботі проаналізовано існуючі методи розпізнавання образів, зроблено огляд основних попередніх робіт, що досліджували хвороби рослин та їх класифікацію за допомогою методів штучного інтелекту. Виконане проектування та програмна реалізація інтелектуального додатку на базі згорткової нейронної мережі AlexNet. Виконана оптимізація характеристик мережі шляхом тестування різноманітних налаштувань параметрів моделі та ресурсних можливостей комп'ютерів. Створений інтелектуальний додаток за допомогою веб-складових HTML/CSS/JavaScript, серверної сторони Python Flask та бази даних MySQL.

Ключові слова: згорткова нейронна мережа, розпізнавання образів, хвороби сільськогосподарських рослин, штучний інтелект, веб-додаток

ABSTRACT

This bachelor's thesis is devoted to the development of automated intellectual identification of agricultural plants' diseases. The paper analyzes the existing methods of pattern recognition, provides an overview of the main preliminary works that investigated plant diseases and their classification using artificial intelligence methods. Design and software implementation of an intelligent application based on the convolutional neural network AlexNet was completed. The network characteristics were optimized by testing various settings of the model parameters and resource capabilities of computers. Created by an intelligent application using the web components of HTML / CSS / JavaScript, the server side of Python Flask and the MySQL database.

Keywords: convolutional neural network, pattern recognition, crop diseases, artificial intelligence, web application

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ ТА СКОРОЧЕНЬ.....	7
ВСТУП	8
1 АНАЛІТИЧНИЙ ОГЛЯД ПРОБЛЕМАТИКИ ІДЕНТИФІКАЦІЇ ХВОРОБ РОСЛИН.....	9
1.1 Економічні аспекти захворюваності сільськогосподарських культур	9
1.2 Типові хвороби сільськогосподарських культур	11
1.3 Відомі методи дослідження хвороб рослин	14
1.4 Машинне навчання на службі у агробізнесу	16
1.5 Задача ідентифікації захворювання сільськогосподарських культур як задача штучного інтелекту	17
1.6 Методи розпізнавання образів	18
1.7 Приклади використання нейронних мереж для вирішення задачі розпізнавання захворювань рослин.....	20
1.8 Існуючі набори даних для вирішення задачі.....	26
1.9 Постановка задачі.....	28
1.10 Системні вимоги.....	28
1.11 Функціональні вимоги	29
1.12 Нефункціональні вимоги	29
1.13 Вхідна та вихідна інформація	30
2 ПРОЕКТУВАННЯ ІНТЕЛЕКТУАЛЬНОГО ДОДАТКА ІДЕНТИФІКАЦІЇ ЗАХВОРЮВАНЬ СІЛЬСЬКОГОСПОДАРСЬКИХ РОСЛИН	31
2.1 Розробка архітектури ІДІЗСР.....	31
2.2 Проектування нейромережних засобів розпізнавання захворювання рослин	34
2.3 Проектування БД.....	49
2.4 Розробка інтерфейсу інтелектуального додатку.....	55
3 РЕАЛІЗАЦІЯ ІНТЕЛЕКТУАЛЬНОГО ДОДАТКУ, ТЕСТОВИЙ ПРИКЛАД	62
3.1 Вибір середовища реалізації проекту	62
3.2 Python-реалізація згорткової мережі	64
3.3 Опис структури датасету	65

3.4 Вибір оптимальної моделі мережі	66
3.5 Аналіз коректності роботи застосунку	72
ВИСНОВКИ.....	87
ВИКОРИСТАНІ ДЖЕРЕЛА	88
ДОДАТОК А Лістинг програми навчання згорткової мережі	94
ДОДАТОК Б Лістинг програми створення Flask-додатку.....	100

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ ТА СКОРОЧЕНЬ

- БД – база даних
- ЗНМ – згорткова нейронна мережа
- ІДІЗСР – інтелектуальний додаток ідентифікації захворювань сільськогосподарських рослин
- с/г – сільськогосподарський
- ШНМ – штучна нейронна мережа

ВСТУП

Дана кваліфікаційна робота присвячена розробці засобів автоматизованої інтелектуальної ідентифікації захворювань сільськогосподарських рослин. Ця тема достатньо актуальна, так як хвороби рослин є головною загрозою для продовольчої безпеки в усьому світі. Більше того, ці проблеми призводять до катастрофічних наслідків для дрібних фермерів, життєдіяльність яких залежить від здорових культур.

Об'єкт дослідження: засоби автоматизованого розпізнавання хвороб сільськогосподарських рослин з використанням технологій комп'ютерного зору. Предмет дослідження: методи розпізнавання захворювань сільськогосподарських рослин на основі їх зображень з використанням згорткових нейронних мереж. Мета: розробка інтелектуального додатку ідентифікації захворювань сільськогосподарських рослин.

У першому розділі розглянуто основні проблеми сільського господарства, а саме, хвороби рослин. Досліджено, які є захворювання рослин. Проаналізовано методи розпізнавання образів існують. Зроблено огляд основних попередніх робіт, що досліджували хвороби рослин та їх класифікацію за допомогою методів штучного інтелекту. У другому розділі спроектовано майбутню систему інтелектуального додатку із використанням DFD діаграм. Розглянуто згорткові мережі та їх основні типи. Для проектування БД було здійснено вибір виконувати у СУБД MySQL. Побудовано фізичну модель БД, виділено основні зв'язки у схемі, описано атрибути таблиць БД. У третьому розділі навчено згорткову нейронну мережу AlexNet шляхом тестування різноманітних налаштувань параметрів моделі та комп'ютерів. Побудовано функції втрат та точності для всіх моделей, як для навчальної, так і для тестової вибірок. Після цього був створений інтелектуальний додаток за допомогою веб-складових HTML/CSS/JavaScript, серверної сторони Python Flask та бази даних MySQL, із використанням MySQL Workbench та MySQL Server.

1 АНАЛІТИЧНИЙ ОГЛЯД ПРОБЛЕМАТИКИ ІДЕНТИФІКАЦІЇ ХВОРОБ РОСЛИН

1.1 Економічні аспекти захворюваності сільськогосподарських культур

Сільське господарство є однією із провідних галузей, що стосується економіки України. Воно спроможне надати вагомий внесок у вирішення проблеми голоду у всьому світі. Відомо, що його здатність виробництва значно більша, ніж потреби населення країни. Згідно з даними із джерела [2], маємо такий розподіл земель країни під сільське господарство, як показано на рисунку 1.1. Також, загальновідомо, що всі сільськогосподарські угіддя займають близько 42 млн. гектарів, а це є близько 70 % від загального фонду України.

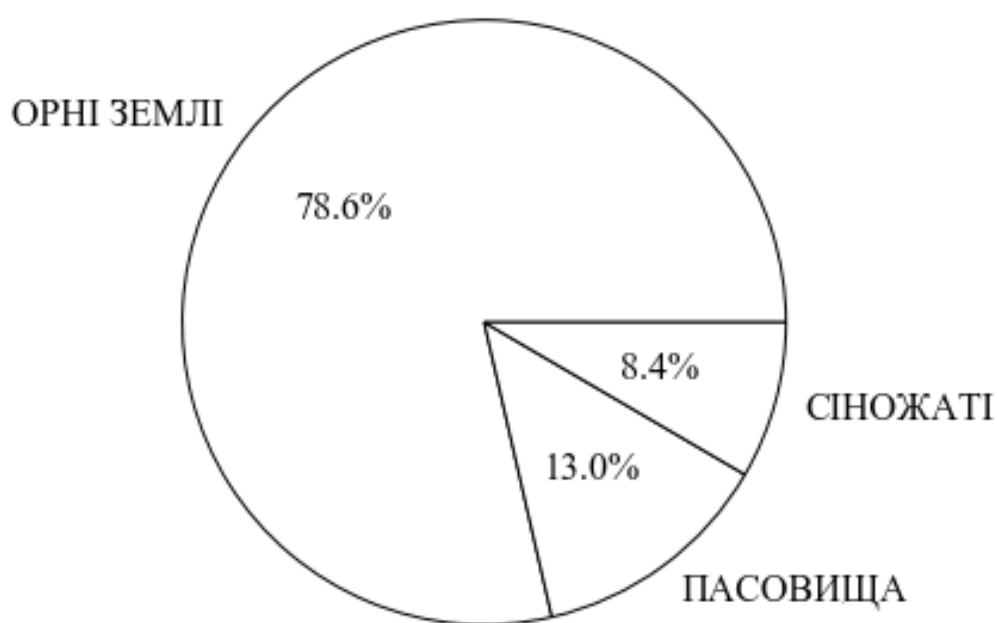


Рисунок 1.1 Розподіл земель під сільське господарство України

Разом з тим, уряд України надає дані, де, 17% валового внутрішнього продукту (ВВП) надається агросектором України і приносить 38% бюджету України у 2018 році[2].

Зрештою, сільське господарство складається не тільки із рослинництва, а і з тваринництва, але перше із них за вартістю відповідної продукції перевищує, тому є більш важливим. Беззаперечно, провідними рослинами в землеробстві є зернові. А саме - гречка, кукурудза, рис, озима і яра пшениця, жито, овес, озимий і ярий ячмінь, просо тощо.

У результаті, Україна займає одне із провідних місць у світі за виробництвом зерна. Наприклад, країна посідає 7 місце по вирощуванню ячменю та 11 по вирощуванню пшениці у світі. Такі можливості дають державі не тільки отримувати великі грошові надходження до бюджету, а й працевлаштовувати багатьох громадян країни.

Відповідно до статистики, наданої Міністерством аграрної політики і продовольства України, у 2016 році Україна змогла виробити понад 66 млн. тон зернових культур.

На рисунку 1.2 показано основні регіони вирощування деяких рослин.



Рисунок 1.2 Мапа вирощування деяких рослин в Україні

Але, на жаль, проблеми розвитку даної сфери сільського господарства також існують. До них можна віднести:

- Неправильне вирощування та догляд за рослинами;
- Погані погодні умови: висока температура чи надмірна вологість
- Пестициди;
- Шкідники;
- Неправильне зберігання рослин та зерна.

Також, рослини отримують певні пошкодження листяної поверхні чи коренів, погіршується якість їх плодів та насіння. Вони втрачають здатність до розвитку та родючості. І це все призводить до певних проблем у майбутньому.

До них можна віднести:

- Зменшення прибутку від вирощування сільськогосподарських культур;
- Проблема всесвітнього голоду;
- Втрата місць заробітку дрібних фермерів;
- Втрата цінних рослин;
- Інші проблеми, що негативно вплинуть на людство.

1.2 Типові хвороби сільськогосподарських культур

Хвороби рослин є однією із головних загроз для продовольчої безпеки не тільки у нашій країні, а й на планеті.

Якщо сільське господарство намагається підтримати життя зростаючого населення, то хвороби рослин знижують одночасно виробництво та якість не лише харчових продуктів і клітковини, але й біопалив.

За статистикою, понад 80 відсотків сільськогосподарського виробництва генерується дрібними фермерами, життєдіяльність яких залежить від здорових культур.

WWF повідомляє, що «Більш ніж половині наявних у світі видів рослин загрожує вимирання, у небезпеці кожен четвертий вид тваринного світу».

Взагалі, можна виділити найголовніші причини захворювань рослин, як показано на рисунку 1.3.

Відповідно до статистики, патогенні мікроорганізми та шкідники знижують врожайність п'яти основних продовольчих культур на 10-40%, відповідно до звіту Каліфорнійського університету в області сільського господарства [3].

Врожаї таких рослин, як пшениці, рису, кукурудзи, сої та картоплі зменшуються не тільки через патогени, а й через існування різних шкідливих тварин, включаючи комах, як було виявлено в масштабному опитуванні фахівців з охорони здоров'я рослин.

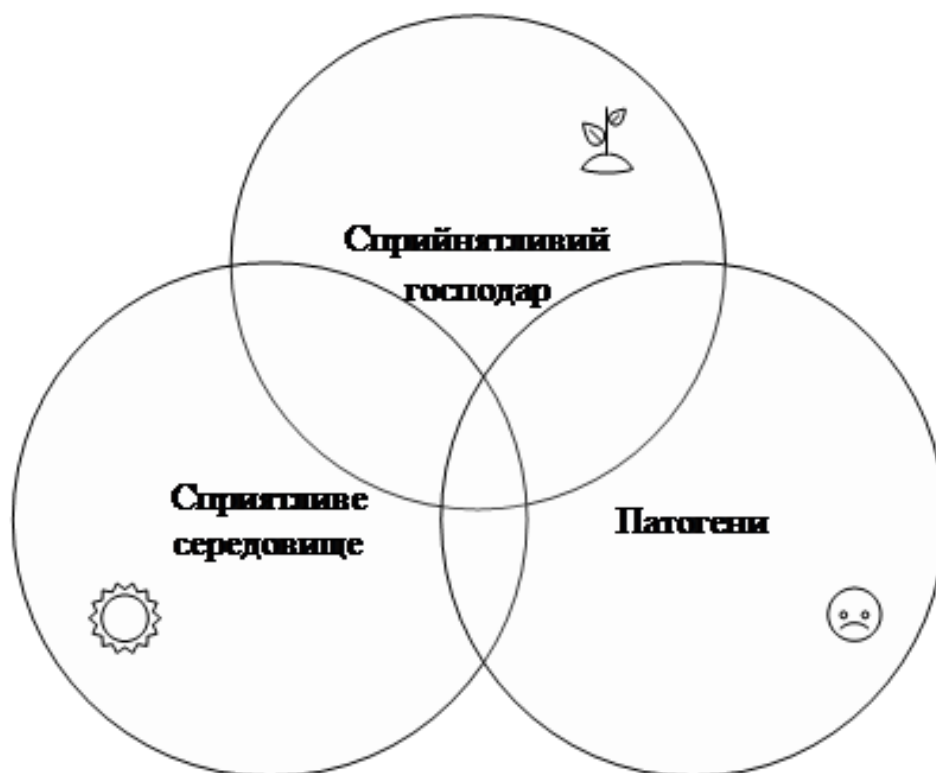


Рисунок 1.3 Трикутник хвороб рослин

Відомо, що збудниками хвороб рослин можуть бути вірусні, грибкові або бактеріальні патогени. Вони можуть пошкодити частини рослин над або під землею [4].

Відповідно до рейтингу, що складено джерелом [5], типовими хворобами можуть бути такі:

1. «Септоріоз листя пшениці» – вражає близько 400 тис. т. зерна. З'являються жовті та світлі плями на листях, із чорними дрібними вкрапленнями. Після чого листя починає всохати. Збудник – гриби із роду *Septoria*.
2. «Борошниста роса» – 370 тис. т. зернових культур. На листях можна помітити павутинний наліт. Збудником також є грибний патоген.
3. «Сітчаста плямистість ячменю» – 310 тис. т. зерна. З'являється сітковий візерунок та характерні бурі плями. Носій хвороби – гриб. Втрати можуть перевищувати 35% врожаю.
4. «Темно-бура плямистість» - 210 тис. т. зерна. Хвороба викликає гниль на коренях та плями на листях. Вражає, як правило, пшеницю та ячмінь. Збудник хвороби – гриб.
5. «Іржа злаків» - 100 тис. т. врожаю сільськогосподарських культур. Вражає жито, пшеницю та ячмінь. На листі з'являються характерні жовтуваті пустули. Збудник хвороби – гриб.

На рисунках 1.4-1.5 проілюстровані [5] деякі захворювання, про які було сказано вище.



Рисунок 1.4 Септоріоз листя пшениці



Рисунок 1.5 Сітчаста плямистість ячменю

Підсумовуючи, можна сказати, що на хвороби рослин треба завжди звертати увагу. Чим швидше буде виявлене захворювання, тим менші будуть втрати сільськогосподарських культур. Основними маркерами хвороб можна назвати такі, згідно з інформацією, що указана у джерелі [6]:

- Зів'янення листя;
- Некроз тканин;
- Знищення органів;
- Гниль;
- Пустули;
- Наліт;
- Деформація форми рослини;
- Нарости;
- Утворення камеді;
- Кремація органів.

1.3 Відомі методи дослідження хвороб рослин

У фітопатологічній практиці часто існують випадки, коли зовнішні ознаки кількох хвороб дуже схожі між собою, а збудники їх появи повністю різні. Тому, може виникнути проблема невірної класифікації хвороби, що призведе до неправильного лікування.

Через це, для максимально точного визначення типу захворювання, крім зовнішнього обстеження пошкодженої рослини, треба проводити певні дослідження та аналізи для того, щоб встановити причини і джерела різних типів інфекцій, що можуть виникнути.

У значній частині випадків явні ознаки хвороби поповнюються дослідженнями за допомогою мікроскопа і морфологічною інформацією про причини. Іноді використовуються різні методи закріплення та фарбування перерізів хворих тканин для різних хімічних чи досліджень. Тому, використовують, як правило, два відомих методи досліджень – біологічний та культуральний методи.

Біологічні експерименти виконують після вивчення диференціації збудника у враженій рослині і визначення рівня сприйнятливості до патогена. Для цих дослідів виконують штучне інфекціонування рослин, для того, щоб у майбутньому можна було порівнювати та вивчати різні симптоми ураження тканин рослин. Не можна не сказати, що знання біохімічних чи фізіологічних властивостей допомагають у лікуванні хвороби.

Необхідно також, як можна швидше, визначити основні фактори, що стимулюють розвиток захворювання.

Інший же метод дослідження - культуральний. Він полягає у виділенні патогенів в окрему культуру задля того, щоб можна було розглянути детальніше морфологічні, біохімічні та інші ознаки.

Таким чином, ці два методи використовувалися завжди під час виявлення хвороб у сільськогосподарських рослин. Але, з кожним разом, захворювання починають з'являтися все швидше і швидше, що зменшує швидкість діагностики і виявлення проблем на органах рослин.

Також, сільське господарство, а саме рослинництво, розвивається, тому площі під посіви стрімко ростуть, що також викликає проблему точної та ранньої діагностики.

1.4 Машинне навчання на службі у агробізнесу

Машинне навчання – це розділ штучного інтелекту, технологія, що отримує різні вхідні дані, потім аналізує їх, приймає рішення і дає відповідь людині. Комплекс таких методів машинного навчання, що взаємодіють із різноманітними наборами певних необроблених даних, а потім вже знаходять рішення проблеми, звуть глибинним навчанням [7]. Ця технологія широко застосовується у сільському господарстві. Наведемо декілька різноманітних прикладів її використання, що вказані у джерелі [7].

По-перше, селекціонери завжди хочуть знайти цікаві властивості, що зможуть дозволити рослинам ефективніше споживати воду чи поживні речовини, не давати розповсюджуватися хворобам чи пристосовуватися до зміни клімату. Для того, Щоб певна рослина отримала у майбутньому таку корисну властивість, науковці мають відшукати правильну послідовність генів. І дане завдання є важким, бо на початку роботи не відомо, яка сама послідовність має з'явитися в результаті.

Дослідники мають перебрати багато тисяч різних варіантів, щоб отримати новий сорт рослини. Але, такий пошук може зайняти десятки років і навіть більше. А глибинне навчання може спростити завдання. Для цього необхідно отримати дані, наприклад, як рослина поводить себе у різних погодних умовах, а потім використати їх для отримання ймовірнісної моделі. За допомогою штучного інтелекту обробка таких даних не займе багато часу. У результаті буде отримано перелік генів, які найкраще підійдуть рослині, перебравши мільйони комбінацій.

Також, машинне навчання може підвищити точність ведення рослинництва. Дослідник може оцінити набагато більший набір факторів. Так, комп'ютерне моделювання дозволяє дізнатися оцінку того, як різні культури будуть реагувати на різні типи ґрунтів чи на кліматичні умови. Штучний інтелект дозволяє селекціонерам прогнозувати із високою точністю продуктивність вирощування сільськогосподарських культур.

Більше того, штучний інтелект можна застосовувати і в сфері дослідження, створення та відбору найкращих насінин для посіву. Він може зробити ці процеси більш ефективними та підвищити їх точність. Таким чином, природні ресурси будуть краще використовуватися. Відомо, що над такими методами та алгоритмами працює вже декілька компаній, одна з них - BOLT-LMM [8].

Іще однією із відомих технологій є See & Spray [9]. Вона являє собою автоматизований обприскувач, який ідентифікує, де сільськогосподарська культура, а де бур'ян. Тому, знищується за допомогою гербіциду лише небажаних рослин. Такий спосіб дозволяє ефективно розподіляти хімікати і економити до 80% розчину.

Також, машинне навчання дозволяє ідентифікувати хвороби рослин, про які буде йти мова у наступному пункті.

1.5 Задача ідентифікації захворювання сільськогосподарських культур як задача штучного інтелекту

Для того, щоб прискорити та автоматизувати виявлення патогенів, можна використовувати новітні методи, що створені з використанням штучного інтелекту (ШІ).

До них можна віднести, як уже було сказано, машинне навчання та комп'ютерний зір [10], які можна використовувати і у сфері сільського господарства. Ці способи допомагають автоматизовано виявляти хворе листя, шляхом візуального огляду. Система, що використовує штучний інтелект, забезпечує швидке розпізнавання хворої рослини і дозволяє розпізнати проблему на ранній стадії. Захворювання рослин можна досліджувати різними способами. Перший із них – прогнозувати хвороби, використовуючи різноманітні дані про культури, наприклад, вміст хімічних речовин у рослині, вік рослини, її тип та сорт. Ця статистика потім може бути оброблена за допомогою методів штучного інтелекту.

Також, система може складатися із потужної обчислювальної техніки, набору камер, що встановлюється біля рослин. Основна система перевіряє потік зображень з камер, виявляє, класифікує хворобу. А самі результати перевірки надаються різними можливими способами. Цей спосіб використовує обробку зображень рослин. Останнім трендом у застосуванні для ідентифікації захворювання засад штучного інтелекту є розробка мобільних застосунків, що дозволяють оперативно виявляти та класифікувати захворювання культурних рослин за фотографіями. Задача визначення виду захворювання рослини за зображеннями відноситься до класу задач розпізнавання образів.

1.6 Методи розпізнавання образів

Розпізнавання образів (РО) — це процес віднесення об'єкта з певною фіксованою групою його властивостей до одного класу з множини образів за заздалегідь обумовленим правилом.

Теорія РО є одним із розділів кібернетики та інформатики, що досліджує методи розпізнавання та класифікації предметів, звуків, явищ [11].

Взагалі, розрізняють два різних типи завдань, а саме:

- завдання таксономії – це навчання без учителя;
- завдання розпізнавання – це навчання із учителем.

На рисунку 1.6 представлено класифікацію систем розпізнавання образів.

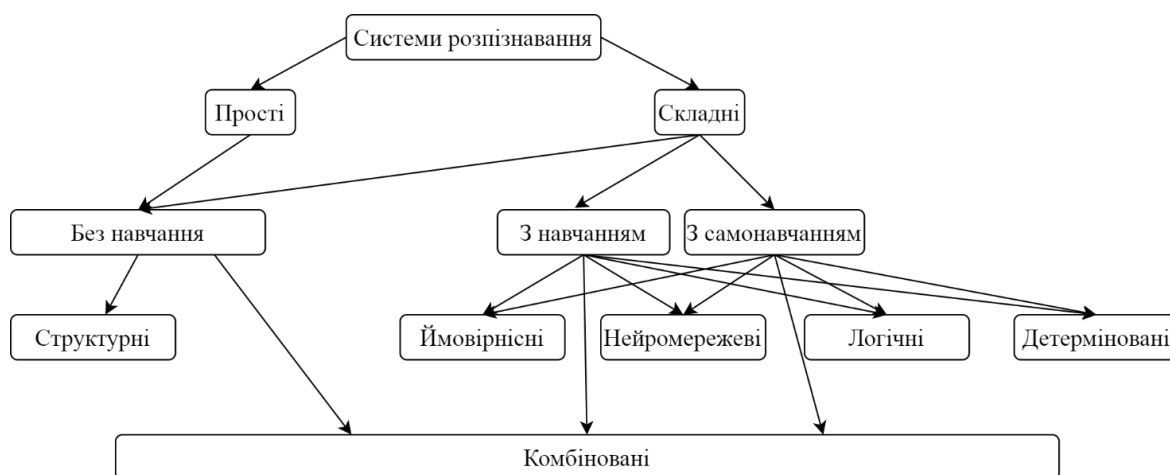


Рисунок 1.6 Класифікація систем розпізнавання

На даний момент, існує декілька напрямів у розпізнаванні образів. А саме:

- Порівняння зображення із деяким еталоном;
- Розпізнавання зображення, використовуючи характерні точки. Але спосіб отримання таких точок може бути різним.
- Розпізнавання за допомогою штучних нейронних мереж.

Також, виділяють чотири групи методів, які використовують у задачах розпізнавання образів. Це такі:

- Порівняння зі зразком. Тут застосовується геометрична нормалізацію та обчислюється відстань до прототипу. Цей метод найчастіше використовується при розпізнаванні тексту.
- Статистичні методи. Використовує елементи математичної статистики. Будується розподіл для всіх класів та застосовується класифікація за Байєсовським правилом.
- Структурні та синтаксичні методи. При використанні таких методів об'єкт розбивається на елементи. Будується певне правило, у залежності від входження окремих елементів і їх послідовностей.
- Нейронні мережі. Тут вибирається тип мережі, а потім налаштовуються вагові коефіцієнти. Після цього, на вхід штучної нейронної мережі подається певний об'єкт для подальшого розпізнавання. Група рецепторів мережі потім відповідає за прийом певної характеристичної властивості.

Виділяють наступні методи розпізнавання:

- Категорія 1:
 - Детерміністські;
 - Структурні.
- Категорія 2:
 - Ймовірнісні;
 - Детерміновані.
- Категорія 3:

- Математичні;
- Евристичні.

1.7 Приклади використання нейронних мереж для вирішення задачі розпізнавання захворювань рослин

Методи РО були застосовані деякими дослідниками. Перша така робота, що буде розглянута [12], була опублікована у 2016 році. Як правило, машинне навчання використовує штучні нейронні мережі (ШНМ) для створення працюючої системи. Автори кажуть, що їх було взято для створення інтелектуальної системи, а саме – згорткові нейронні мережі для проблеми, що була описана вище. Вони дослідили два типи архітектури ШНМ. Перша з них – це AlexNet [13], інша – GoogLeNet [14], які були створені в контексті “Large Scale Visual Recognition Challenge” (ILSVRC) [15] для набору даних ImageNet [16]. Архітектура нейронної мережі AlexNet мала такий же дизайн, як і архітектура LeNet-5 [17] 1990 року. Варіанти архітектури LeNet-5 складалися із шарів згортки з декількома повністю пов'язаними шарами. Їх архітектура AlexNet містила п'ять шарів згортки, також три повністю пов'язаних шари і ще один шар - softMax. Автори кажуть, що перші два шари згортки мали шар нормалізації та об'єднання, а за останнім шаром цього типу був один шар об'єднання. Останній повністю пов'язаний шар (fc8) мав 38 виходів у їх адаптованій версії AlexNet (як у кількість класів у їх наборі даних), що мав шар softMax. Рівень softMax експоненціально нормалізує вхід, який він отримував від (fc8), тим самим створювавши розподіл значень за 38 класами. Всі перші 7 шарів AlexNet мали пов'язаний з ними блок нелінійної активації ReLu, а перші два повністю з'єднані шари мали пов'язаний з ними шар випадання з коефіцієнтом випадання 0,5.

Наступна мережа, що була ними створена, як вже було сказано, - GoogLeNet. Ця архітектура була значно глибшою, бо містила 22 шари, але мала значно меншу кількість параметрів (близько п'яти млн. параметрів) у мережі, ніж

AlexNet (60 млн. параметрів). Автори наголошують, що застосування архітектури «network in network» [18] у вигляді початкових модулів є головною особливістю архітектури GoogleNet. Модуль Inception використовував паралельні згортки 1×1 , 3×3 та 5×5 разом із шаром максимального пулінгу паралельно, що дозволило йому паралельно фіксувати різноманітні ознаки. З їх практичної точки зору виконання, кількість пов'язаних обчислень треба завжди перевіряти, тому 1×1 згортки перед вищезгаданими 3×3 , 5×5 згортками (а також після шару максимального пулінгу) були додані для зменшення розмірності. Наприкінці, шар конкатенації фільтрів легко об'єднав всі виходи даних паралельних шарів. Вони використали, у загальному, дев'ять модулів Inception.

Після цього, автори проаналізували ефективність роботи цих двох архітектур на наборі даних під назвою PlantVillage, навчаючи модель з початку («Training from Scratch») в першому випадку, а потім за допомогою адаптації вже підготовлених моделей (навчених на наборі даних ImageNet), використовуючи трансферне навчання («Transfer Learning»).

Вони дослідили і знайшли ключову відмінність між навчанням з нуля і між трансферним навчанням. А саме – відомий початковий стан ваг декількох шарів, що дозволив підходу трансферного навчання використовувати великий об'єм візуальних знань, вже отриманих попередньо.

Також, не можна не сказати, що вони отримали 60 експериментальних конфігурацій, які мали зміни у наступних параметрах:

1. Вибір архітектури для навчання мережі:
 - AlexNet;
 - GoogLeNet.
2. Вибір механізму навчання:
 - «Transfer Learning»;
 - «Training from Scratch».
3. Вибір типу набору даних:
 - Кольоровий;

- Сірій.

4. Вибір відсоткового розподілу для навчання та тестування мережі:

- 80% / 20%;
- 60% / 40%;
- 50% / 50%;
- 40% / 60%;
- 20% / 80%.

Дослідники використали 30 епох для навчання мережі у всіх 60 експериментах. Саме така кількість була вибрана емпіричним шляхом для всіх досліджень. Також, необхідно зазначити, що для того, щоб було присутнє рівне та чесне порівняння результатів, вони намагалися стандартизувати деякі гіперпараметри у всіх експериментах. А саме:

- Тип рішення: стохастичний градієнт;
- Розмір партії (batch size): 24 (у випадку GoogLeNet), 100 (у випадку AlexNet);
- Базовий коефіцієнт навчання: 0.005;
- Швидкість навчання: крок, що зменшувався кожен епоху.

Наприкінці відзначимо, що їх нейромережа після навчання змогла розпізнавати та класифікувати хвороби рослин з великою точністю. Також, вони прийшли до висновку, що GoogLeNet працювала краще, ніж AlexNet і що трансферне навчання завжди дає кращі результати, чим навчання з нуля. Вони використали набір даних (датасет) PlantVillage. Їх мережа змогла правильно визначити 1 із 38 класів у 993 випадках із 1000. Сам набір даних містив 54'306 зображень 14 типів рослин і 26 різновидів хвороб.

Тепер, розглянемо дослідження інших науковців. Наприклад, роботу [19], що була виконана в 2018 році. Спочатку, вони підготували маленький набір даних, що складався із зображень розміром 256x256 пікселів. Він містив як здорові, так і хворі листя рослин, що мали певні ураження, наприклад, хлороз чи чорна гниль.

Вони, також, як і попередні дослідники, використовували модель трансферного навчання, щоб створити класифікатор зображень. Для того, щоб знайти найбільш пристосовану попередньо підготовлену мережу, вони порівнювали чотири відомих моделі, ваги яких були налаштовані для вирішення проблеми ILSVRC 2015 (ImageNet Large Scale Visual Recognition Challenge), як і в попередньому дослідженні, що було розглянуто вище. Такими моделями були: VGG19 [20], InceptionV3 [21], ResNet50 [22] та Xception [23].

Порівняльна схема кожного класифікатора полягала в тому, що бралися всі шари попередньо навченої мережі, окрім останнього шару та додавалася глобальна середня операція пулінгу на верхню частину кожної базової мережі для зменшення просторових розмірів тривимірного тензора виводу. А потім вони додали ще один шар із 256 виходами із коефіцієнтом виходу 0.5. Тобто, вони «заморозили» і зберегли всі шари в базовій нейромережі, а перенавчили лише три останні шари, використовуючи також стохастичний градієнтний спуск, як і автори у попередній роботі, але із 50 епохами навчання мережі.

Найкращі результати точності класифікації отримали після використання архітектури ResNet50. Але після застосування цієї моделі на тестовій підбірці було виявлено, що вона дала замалу точність розпізнавання. Тому вони вирішили, що треба перенавчити ще деякі шари штучної нейронної мережі. Дослідники «розморозили» всі шари, окрім перших 140 штук. І навчили ці шари, використовуючи оптимізатор Адама (Adam optimizer) [24]. Але точність була підвищення не на багато. Тому вони припустили, що проблема може бути у самому наборі даних, бо фотографії були зроблені на однорідному фоні, а не в природніх умовах вирощування рослин. Після цього вони вирішили, що зберуть власний датасет, що буде зроблений у природніх умовах. Автори знайшли зображення у відкритих джерелах, виділили основні та важливі частини, вирізали під заданий розмір, але проблем не вдалося уникнути, а також фотографій виявилось замало. Тому дослідники вирішили шукати нові методи вирішення задачі. Наступне, що вони розглянули – це сіамська нейронна мережа. Дана модель складалась із двох мереж-близнюків, з'єднаних шаром подібності із

енергетичною функцією зверху. Ваги «близнюків» були зв'язані, тому результат був інваріантним, а також гарантував, що дуже схожі зображення не могли бути у різних місцях у просторі ознак, оскільки кожна мережа обчислювала одну і ту ж функцію.

Шар подібності визначав деяку метрику відстані між вкладеннями, тобто представленнями на високому рівні ознак вхідних пар зображень. Тренування на парах означало, що існувало квадратично більше можливих зразків для навчання моделі, тим самим ускладнюючи їх надмірне вдосконалення.

Автори презентували з'єднання одного сигмоїдального нейрона із шаром відстані, що дозволило тренувати модель із бінарними втратами перехресної ентропії. Вони використали 32 згорткових фільтри і подвоєння кожного шару. Після останнього шару пулінга вони додали операцію згладжування, щоб стиснути згорткові елементи у вектор, після якого був шар, що містив 1024 сигмовидних нейронів. Також дослідники експериментували з додаванням регуляризації L2 до кожного із згорткових шарів, але не отримали видимого ефекту. Крім того, вони намагалися змінити розмір вкладеного шару від 256 до 4096.

Більше того, автори використали також і алгоритм k-найближчих сусідів для вирішення задачі класифікації на етапі тестування. Для метрики відстані, вони взяли абсолютну відстань у лямбда-шарі сіамської мережі. Після цього було вирішено збільшити розмір набору даних шляхом додавання поворотів на 75 градусів, випадкових зсувів. Також застосували масштабування і горизонтальне та вертикальне відображення.

Потім вони навчили сіамську мережу із використанням оптимізатора Адама. Дані для навчання та тестування вони розбили як 75% до 25%. І після цих всіх дій рівень класифікації у них значно виріс. Не можна не сказати, що вони спробували змішати зображення із живої природи разом із набором даних PlantVillage і проводити навчання і тестування.

Таким чином, вони значно покращили свої результати у розпізнаванні та класифікації хвороб.

Наступна наукова робота [25] була виконана в 2019 році. Авторка використала дві архітектури – AlexNet і свою, користувацьку. Також використала дві моделі навчання – з нуля та трансферне навчання. Але в неї кращі результати показало навчання з нуля, бо мережа була повністю, з самого початку, направлена на розпізнавання хвороб рослин.

Також вона розглянула три набори даних:

1. Кольоровий (RGB)
2. Сірий (Gray Scale)
3. Сегментований

У результаті, сегментовані зображення після навчання дали найкращу точність, бо всі шуми були видалені. Роботу вона виконувала у середовищі Matlab. Набір зображень авторка використала, як і всі попередні дослідники - PlantVillage. Початкова її мета була – дослідити, яка архітектура буде найкращою для розпізнавання хвороб рослин. Авторка створила 9 різних типів. Під час навчання вона виявила експоненціальне підвищення точності завдяки трансферному навчанню.

Таким чином, її вибрана до навчання та тестування штучна нейронна мережа досягла значних результатів, точність була достатньо висока. Також, було досліджено, що AlexNet стабільно перевершує архітектуру MyNet. Мережа спрацювала краще із кольоровим (RGB) набором даних, що вказує на те, що мережа використовувала кольорові показники, такі як аномальне забарвлення листя для класифікації. Незважаючи на те, що тренування її мережі займало кілька днів, сама класифікація окремого зображення займала менше 1 секунди. Без вилучення особливостей модель правильно класифікувала 38 класів із 14 культур та 26 захворювань.

Здійснений аналіз розглянутих деяких відомих робіт дає змогу дійти висновку, що найкращою архітектурою згорткової нейронної мережі у цих дослідженнях показала AlexNet, із використанням трансферного навчання чи навчання з нуля. Найкращі результати показав той набір даних, що містив кольорові зображення.

1.8 Існуючі набори даних для вирішення задачі

Тепер, детальніше розглянемо відомі набори даних, на яких можна навчати мережу. Як сказано в джерелі [26], існує три найпопулярніших набори даних, в залежності від повноти та інформативності. Розглянемо їх.

Перший – PlantVillage dataset [27], про який уже було згадано раніше. Тут зображення зроблені у штучному освітленні, на однорідному фоні, окремо від ґрунту. Це все спрощує процес аналізу даних зображень, так як відсутні інші об'єкти, які можуть бути пов'язані із зовнішніми природними умовами або будовою рослин, за для того, щоб зосередити увагу на самих симптомах. Даний набір містить 54'306 фотографій рослин із сільськогосподарськими культурами (26 хвороб). Розміри всіх зображень 256x256 пікселів. Є відкритим датасетом, був створений у рамках проекту PlantVillage [28]. Містить такі рослини, як:

- Яблуко;
- Вишня;
- Кукурудза;
- Виноград;
- Картопля;
- Помідор;
- Тощо.

На рисунку 1.7 наведено приклад зображення із даного набору даних.



Рисунок 1.7 Зображення із PlantVillage dataset

Наступний набір даних – більш комплексний. Фотографії були зроблені у неконтрольованих умовах – на природі, біля ґрунту, серед інших рослин. Але тут зображення сфокусовано на певному органі рослини, а саме – на листі, що містить уражену ділянку. У цьому датасеті зображення мають неоднорідний фон, але велика площа зайнята саме зображенням хвороби рослини. На рисунку 1.8 представлено приклад зображення із цього набору даних.



Рисунок 1.8 Приклад зображення із наступного набору даних

Нарешті, третій набір даних – також знятий у природніх умовах. Але зображення не фокусується на певному органі, а містить всю рослину. Даний датасет найкраще підходить для автоматизованого фітосанітарного застосунку, коли рослин забагато. Тобто, фотографії відображають те, що побачив би оператор у полі.

На рисунку 1.9 показано приклад зображення із цього набору даних.



Рисунок 1.9 Приклад зображення із третього набору даних

Наприкінці відзначмо, що найкращим датасетом для даних досліджень будемо вважати PlantVillage, бо він містить найбільше інформативних ознак та є одним із найбільших наборів даних, що є вільнодоступним у всьому світі.

1.9 Постановка задачі

Ціллю даного проекту є створення системи розпізнавання зображень, яка зможе виявляти та класифікувати хвороби сільськогосподарських рослин за їх зображенням.

Для вирішення поставленої задачі буде застосований метод штучного інтелекту, а саме, із використанням штучний нейронних мереж, про які було сказано раніше. Дана техніка являється однією із найбільш надійних для класифікації зображень.

До основних переваг цього рішення можна віднести високу швидкість обчислень та достатньо гарну точність класифікації.

Дана система розпізнавання різних захворювань рослин зможе працювати як універсальний та якісний детектор, розпізнаючи загальні проблеми на листках, такі як цвіль чи гниль.

Майбутнє дослідження пов'язане з точним виявленням певних захворювань. Після якісного навчання створена модель машинного навчання зможе розрізняти велику кількість різноманітних захворювань рослин.

Ми будемо використовувати набір даних PlantVillage, так як він є найбільш інформативним для поставленої нами задачі.

1.10 Системні вимоги

До мінімальних системних вимог можна віднести такі:

- Жорсткий диск: 5 ГБ вільного місця
- Оперативна пам'ять (ОЗП): 16 ГБ
- Операційна система (ОС): Windows 7 (x64) і вище
- Процесор: Intel i5, 1.60GHz (4CPUs), ~2.3GHz

- Відеокарта: Intel Graphics Family, NVIDIA GeForce 920M
- Аудіокарта: DirectX-сумісна
- DirectX: 11

1.11 Функціональні вимоги

Функціональними вимогами є такі:

- Навчання системи на вибірці зображень
- Відображення похибки обчислення у процесі навчання системи
- Розпізнавання зображень сільськогосподарських рослин
- Класифікація хвороб сільськогосподарських рослин
- Надання рекомендацій щодо лікування уражених ділянок рослин
- Ведення бази даних захворювань та препаратів для лікування
- Реалізація засобів завантаження знімків рослин
- Засоби збереження розпізнаних прикладів

1.12 Нефункціональні вимоги

До нефункціональних вимог можна віднести наступні:

- Розмір вхідного зображення 256x256 пікселів;
- Опрацювання та повернення результатів менше, чим за хвилину;
- Надійність системи 90%;
- Мова програмування – Python 3.6;
- Термін виготовлення готового продукту – травень 2020 року;
- Система повинна розроблятися в рамках діючого законодавства України;
- Система не порушуватиме етичних вимог, вона буде прийнятна для всіх користувачів;
- Вікова категорія: 0+.

1.13 Вхідна та вихідна інформація

Вхідна інформація:

- Набір даних зображень PlantVillage для навчання та тренування мережі
- Зображення рослини, що було зроблене користувачем

Вихідна інформація:

- Текстова інформація про результат класифікації хвороби рослини
- Текстові рекомендації щодо догляду за рослиною

2 ПРОЕКТУВАННЯ ІНТЕЛЕКТУАЛЬНОГО ДОДАТКА ІДЕНТИФІКАЦІЇ ЗАХВОРЮВАНЬ СІЛЬСЬКОГОСПОДАРСЬКИХ РОСЛИН

2.1 Розробка архітектури ІДІЗСР

2.1.1 Діаграми потоків даних ІДІЗСР

Побудуємо діаграму потоків даних (рисунок 2.1). Діаграми виконано у середовищі ERwin Process Modeler, нотація Гейна–Сарсона. Виділимо такі компоненти для контекстної діаграми потоків даних:

- Зовнішня сутність: користувач;
- Процес: ідентифікація захворювань с/г рослин;
- Потоки даних:
 - зображення с/г рослини;
 - дата;
 - звіт про класифікацію хвороби с/г рослини;
 - рекомендоване лікування с/г рослини;
 - історія випадків класифікації хвороб с/г рослин;
 - історія лікування с/г рослин;
 - статистичний звіт історії випадків класифікації захворювання с/г рослин.

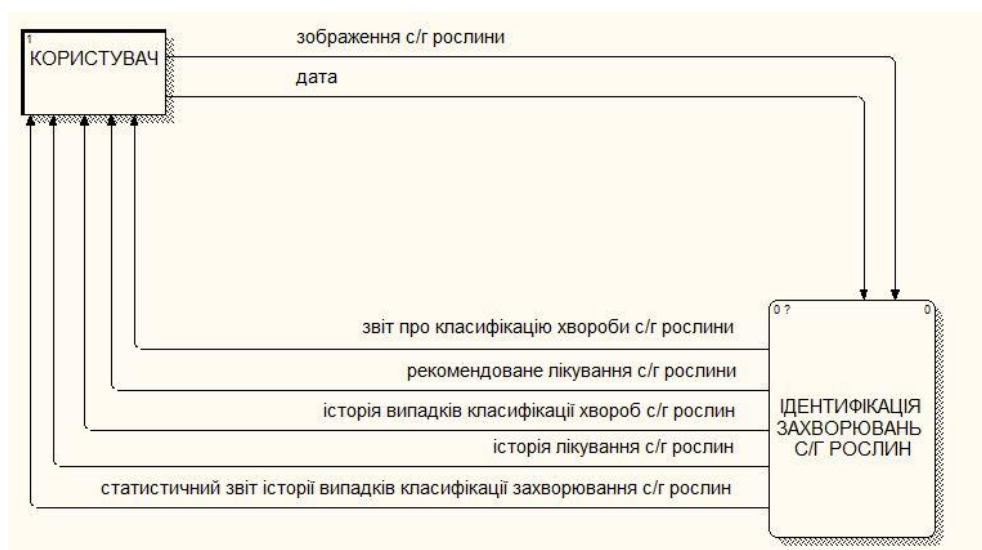


Рисунок 2.1 Контекстна діаграма потоків даних

Тепер, для того, щоб побудувати діаграму потоків даних першого рівня, необхідно її декомпонувати на менші частини. У результаті, отримаємо такі складові елементи даної діаграми (рисунок 2.2):

- Підпроцеси:
 - опрацювання вхідної інформації;
 - розпізнавання захворювань с/г рослин;
 - формування звітності.
- Накопичувачі:
 - Модель ШНМ, що буде містити навчену модель ШНМ, що використовується для розпізнавання зображень;
 - БД класів, хвороб та препаратів, що міститиме всю необхідну інформацію про класам, які отримуються у результаті роботи ШНМ, детальну інформацію про хвороби, та препарати, які можна застосовувати до даних хвороб;
 - БД історії випадків та лікувань, у якій буде зберігатися інформація про випадки класифікації зображення с/г рослин та який саме препарат/препарати застосовувались для рослин на ділянці (як історія, хронологія) та інша інформація;
- Потоки даних:
 - зображення с/г рослини;
 - дата;
 - звіт про класифікацію хвороби с/г рослини;
 - рекомендоване лікування с/г рослини;
 - історія випадків класифікації хвороб с/г рослин;
 - історія лікування с/г рослин;
 - статистичний звіт історії випадків класифікації захворювання с/г рослин;
 - запис випадку захворювання с/г рослини;
 - запит про історію випадків та лікувань;

- звіт про історію випадків та лікувань;
- результат класифікації;
- запит про модель ШНМ;
- дані про модель ШНМ;
- запит про класи, хвороби і препарати;
- звіт про класи, хвороби і препарати.

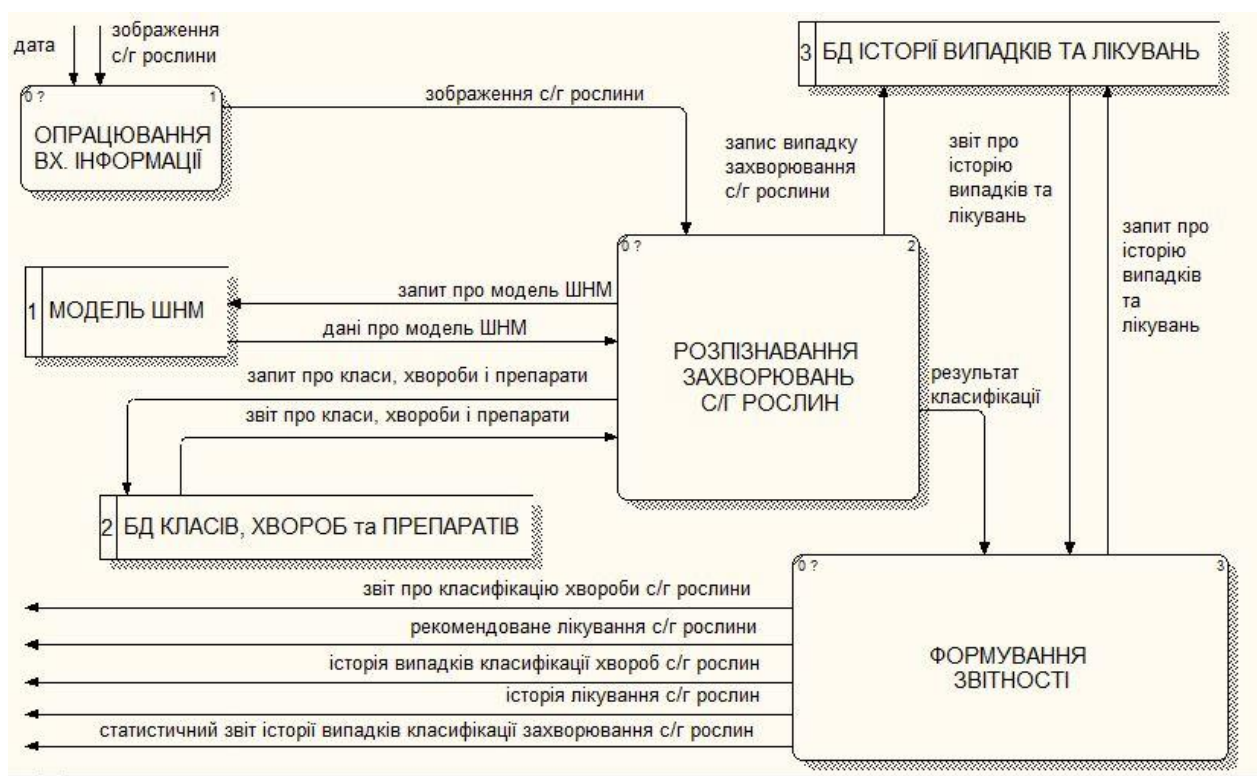


Рисунок 2.2 Діаграма потоків даних першого рівня

2.1.2 Структурна діаграма ІДІЗСР

Структурна діаграма (рис. 2.3), дерево функцій інформаційної технології, представляє собою ієрархічну структуру. Виділимо основні функції:

- Опрацювання вхідної інформації:
 - отримання зображення;
 - збереження зображення.
- Розпізнавання захворювань:
 - обробка зображення;

- класифікація зображення;
- ведення БД класів, хвороб та препаратів;
- ведення БД історії випадків.
- Формування звітності:
 - формування звіту про класифікацію хвороби;
 - формування списку рекомендованого лікування;
 - ведення БД історії лікувань;
 - формування історії випадків класифікації хвороб;
 - формування історії лікування;
 - формування статистичної діаграми випадків захворювання.

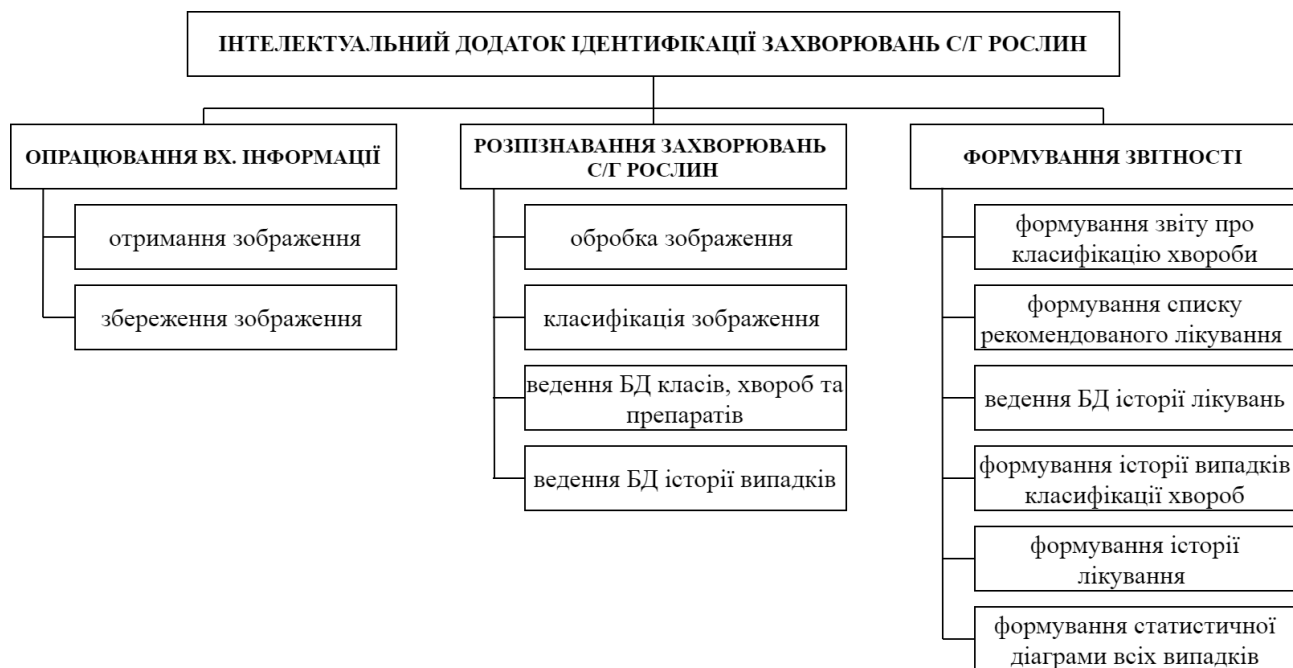


Рисунок 2.3 Структурна діаграма

2.2 Проектування нейромережних засобів розпізнавання захворювання рослин

Нейронні мережі, як відомо, гарно справляються із вирішенням важкоформалізованих задач.

Але, різновидів ШНМ існує дуже багато і всі вони використовуються у різних цілях.

Наприклад, для розпізнавання зображень, що є основним у даній науковій роботі, використовуються наступні типи:

- Звичайні нейронні мережі (повнозв'язні). У них кожен нейрон зв'язаний один із одним, сигнал напрямлений лише із початку в кінець.
- Згорткові нейронні мережі. Вони вважаються найкращими для вирішення задачі розпізнавання зображення. Тому, саме вони будуть використанні для поставленої задачі у кваліфікаційній роботі. Розглянемо їх детальніше.

2.2.1 Згорткові нейронні мережі

Згорткові нейронні мережі належать до класу глибинних ШНМ прямого поширення. Як вже було сказано вище, вони застосовуються до аналізу візуальних зображень [32]. ЗНМ можуть забезпечити певну стійкість до змін та спотворень. Таких як поворот, зсув, зміна ракурсу зображення тощо [33]. Дана архітектура була запропонована французьким вченим Яном Лекуном у 1988 році [34]. Ідея будови ЗНМ лежить у слідуванні згорткових шарів (С-шар), і декількох шарів підвибірки (S-шар). А на виході лежить повнозв'язний шар нейронів. Вони утворюють ансамбль типізованих ШНМ. Згорткові нейронні мережі мають у собі три основні механізми, які використовуються для досягнення інваріантності(незмінності) до спотворень різних зображень. Виділимо їх:

- Локальна виїмка ознак. Це означає, що будь-який нейрон приймає сигнал від рецептивного локального поля у попередньому шарі. І у такий спосіб добуваючи його локальні прикмети. Коли ж ознаку було знайдено, то його місцеположення уже не грає ролі, бо відомо його розташування у порівнянні з іншими ознаками [35];
- Утворення шарів, що мають вигляд комплекту деяких карт ознак. Це означає, що всякий обчислювальний шар містить в собі множину карт ознак. Це такі площини, де усі нейрони використовують одну і ту ж саму множину синаптичних ваг. Але, така форма представлення робить

складнішою будову мережі. Проте, вона має дві позитивних риси. Перша з них - інваріантність(незмінність) відносно зсуву, що може бути досягнена за допомогою певної згортки з ядром маленького розміру. Наступна - це зменшення кількості вільних параметрів, що може бути досягнене за допомогою сукупного застосування синаптичних ваг всіма нейронами спільної карти [35];

- І останнім механізмом є підвибірка. Це означає, що за всяким шаром згортки наступним буде шар із обчисленнями, що виконує локальне усереднення, а також підвибірку. Більше того, така дія зумовлює зменшення чутливості шуканого вихідного показника оператора відтворення ознак до мізерного зсуву чи до інших видів деформації. Відомо, що подібним оператором є одна із сигмоїдальних функцій, що використовується у створенні нейронних мереж [35].

На рисунку 2.4 [35] представлена показана архітектура згорткової нейронної мережі, що реалізує таку можливість, як розпізнавання зображень.

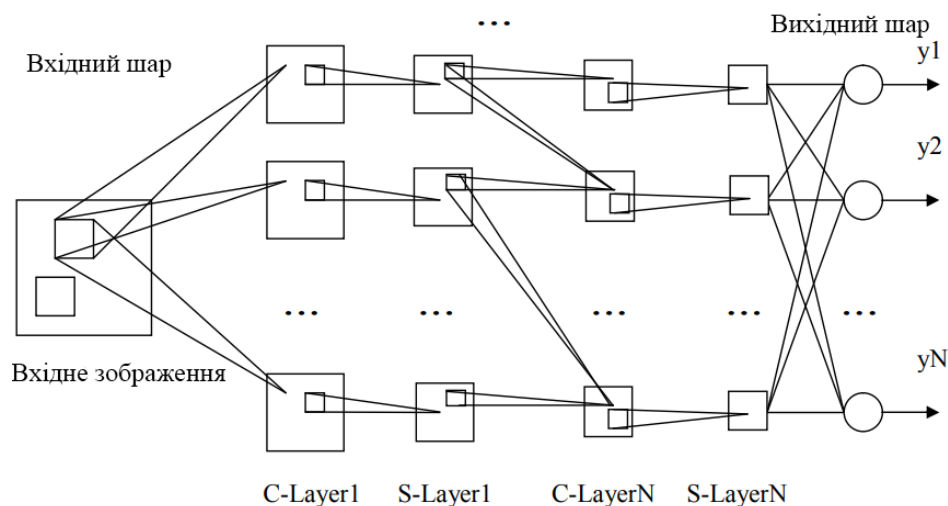


Рисунок 2.4 Згорткова нейронна мережа

Необхідно додати, що головна особливість роботи згорткових мереж пов'язана, як вже було сказано раніше, із застосуванням певних карт ознак.

Спочатку, зображення починає скануватися малим вікном квадратного розміру. Також, розташування вікна на фотографії можуть частково перекриватися. Тоді, у квадратному вікні кожному об'єкту сканування пікселя зіставляється своя деяка вага. Після цього на вихід вікно видає зсумоване зважене значення всіх обрахованих пікселів. Але так як таких положень вікна чимала кількість, то у результаті отримується двовимірна сітка таких знайдених значень. Таким чином, таке вікно детектує наявність поєднання декількох пікселів на зображенні.

У кінцевому результаті на вихідній сітці отримуються великі кінцеві значення там, де є необхідні поєднання пікселів, та маленькі там, де знаходиться інформація зовсім не та, що шукається.

Така сітка(решітка) називається картою ознак, про яку вже було сказано раніше.

Також, знаходиться не одна якась певна ознака, а багато різних. Для кожної із таких знак є певне своє вікно, що містить свій набір ваг. Тому, у результаті, на виході отримується декілька карт ознак, що були знайдені у підсумку. Кожна карта ознак (вже як зображення) відправляється на вхід наступного шару розпізнавача і також отримує на виході ще кілька карт ознак.

Далі, автори роботи [35] кажуть, що у вихідному шарі нейронної мережі всі карти подаються на вхід деякого підсумкового розпізнавача, в якості якого можна застосовувати, наприклад, багатошаровий перцептрон.

Відомо, що модель згорткової мережі також можна розглянути також і у теоретико-множинному представленні. Воно задається таким кортежем [35]:

$$CNN = \langle M, \{N^\eta\}, \{ \langle \{C_{(kl),m}^{(\eta,p^n,(ij))}\}, f^\eta, \{S_{(qt),n}^{(\eta,p^n,(i,j))}\}, \varphi^\eta \rangle \}, N, \{w_i^j\}, \psi \rangle,$$

де M - кількість шарів у мережі, η - номер шару і $1 \leq \eta \leq M$; N^η - число карт ознак в η -му шарі; $C_{(kl),m}^{(\eta,p^n,(ij))}$ - ваговий коефіцієнт зв'язку m -того входу (ij) нейрону p -площини η -го шару із (kl) -м виходом його рецептивної області(тобто, вхід нейрону – це матриця певних рецептивних зв'язків);

$1 \leq i \leq I^{(\eta,c)}, 1 \leq j \leq J^{(\eta,c)}, I^{(\eta,c)}, J^{(\eta,c)}$ - розміри карти ознак η -го шару C -того підшару; $a_{ij}^{\eta,c} \leq k \leq b_{ij}^{\eta,c}, e_{ij}^{\eta,c} \leq l \leq d_{ij}^{\eta,c}$, де $a_{ij}^{\eta,c}, b_{ij}^{\eta,c}, e_{ij}^{\eta,c}, d_{ij}^{\eta,c}$ - координати межі області зв'язку (ij) -го елемента нейрона η -го шару C -того підшару.

Також, автори [35] зазначають, що $1 \leq m \leq \theta, \theta = (b_{ij}^{\eta,c} - a_{ij}^{\eta,c}) \cdot (d_{ij}^{\eta,c} - e_{ij}^{\eta,c})$, де θ - розмір вектору вхідних сигналів (ij) -го нейрону η -го шару C -того підшару.

Інші позначення за словами авторів [35] мають такі пояснення: $S_{(qt),n}^{(\eta,p,(i',j'))}$ - ваговий коефіцієнт зв'язку n -го входу (i',j') нейрона p -тої площини η -го шару із (qt) -м виходом його рецептивної області (тобто, вхід нейрону – це матриця рецептивних полів); $1 \leq i' \leq I^{(\eta,s)}, 1 \leq j' \leq J^{(\eta,s)}, I^{(\eta,s)}, J^{(\eta,s)}$ - розміри карти ознак η -го шару S -того підшару; $a_{ij}^{\eta,s} \leq q \leq b_{ij}^{\eta,s}, e_{ij}^{\eta,s} \leq t \leq d_{ij}^{\eta,s}$, де $a_{ij}^{\eta,s}, b_{ij}^{\eta,s}, e_{ij}^{\eta,s}, d_{ij}^{\eta,s}$ - координати межі області зв'язку (i',j') -го елемента нейрона η -го шару S -того підшару.

Також, вони зазначають [35], що $1 \leq n \leq \theta, \theta = (b_{ij}^{\eta,s} - a_{ij}^{\eta,s}) \cdot (d_{ij}^{\eta,s} - e_{ij}^{\eta,s})$, де θ - розмір вектору вхідних сигналів (ij) -го нейрону η -го шару S -того підшару; f^η, ϕ^η - функції активації нейронів η -го шару, C і S шарів відповідно.

Більше того, дослідники описали параметри вихідного шару [35] мережі: N - число нейронів останнього шару ШНМ; $w_i^{j''}$ - матриця вагових коефіцієнтів вихідного шару; ψ - функція активації нейронів вихідного шару.

Відомо, що у загальному випадкові шарів може бути велика кількість, але навіть занадто велика кількість шарів у мережі може призвести до того, що мережа втратить здатність розпізнавати зображення з високою точністю.

Отже, згортка – це процес обчислення нового значення певного пікселя, при врахуванні значень пікселів, що його оточують.

Для проведення даних обрахунків використовують матрицю, яку називають ядром згортки. Як правило, воно має розмір $a * a$, де a - непарне

число. Іноді зустрічаються випадки, коли матриця згортки є не квадратною, а прямокутною. Популярними ядрами згортки можна назвати, які використовуються у наступних цілях: розмиття, виділення границь, підвищення чіткості тощо (рисунок 2.5).

Розмиття			Виділення границь			Підвищення чіткості		
1/9	1/9	1/9	0	-1	0	0	-1	0
1/9	1/9	1/9	-1	4	-1	-1	5	-1
1/9	1/9	1/9	0	-1	0	0	-1	0

Рисунок 2.5 Ядра згортки

На рисунку 2.6 зображено схематично карта ознак, ядро згортання і наступна карта ознак.

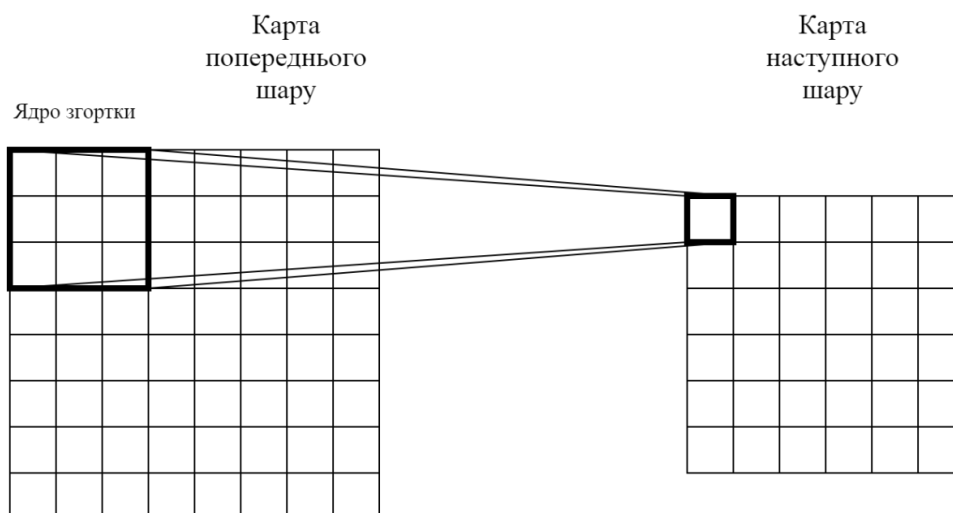


Рисунок 2.6 Згортання карти ознак у меншу карту

Як зазначає автор статті [36], що операція згортки має гарну властивість - інваріантність, про яку вже було сказано раніше, щодо перенесення. Це означає, що кожен згортковий фільтр відображає деяку множину властивостей,

наприклад, очі, вуха, тіло і тому подібне. А сам алгоритм згорткової нейронної мережі вчиться визначати, який набір властивостей відповідає зображенню, наприклад, kota чи іншої тварини.

Тобто, інтенсивність вихідного сигналу залежить не від місця розташування властивостей, а від їх наявності. Отже, кіт може бути зображений в різних позах, але алгоритм все одно зможе його розпізнати.

Також, автор статті [36] зазначає, що вчені, дослідивши принцип роботи головного мозку, змогли створити математичну модель для вилучення певних властивостей.

Але потім, краще оцінивши загальну кількість існуючих шарів і властивостей, які необхідно проаналізувати для дослідження різних геометричних форм, науковці дійшли до висновку, що для зберігання всієї інформації вистачить пам'яті навіть у надпотужних комп'ютерах.

Більше того, кількість необхідних потужних обчислювальних ресурсів починає експоненціально зростати разом зі збільшенням кількості всіх властивостей, що досліджуються.

Для вирішення цієї проблеми була розроблена методика пулінга, який використовується на шарі підвибірки або, як його іще називають, шарі пулінга чи субдискретизації.

Процес пулінга простий - якщо деяка область зображення містить яскраво виражені певні властивості, то можливо відмовитися від пошуку властивостей у даній області дослідження. Дана операція робить не лише економію зайнятої пам'яті чи обчислювальної потужності, але і дозволяє очистити зображення від різноманітного шуму.

Як правило, за фільтр беруть матрицю, розміром 2×2 , що застосовується із кроком, рівним 2.

Також, існують різні варіанти нелінійних функцій, що використовуються для операції пулінгу, серед яких найбільш популярна конструкція максимального пулінгу, яка для кожного регіону знаходить і повертає максимальне значення (рисунок 2.7).

Як і було сказано вище, головна ідея полягає у тому, що точне положення певної ознаки у просторі не так важливе, як її положення щодо інших ознак.



Рисунок 2.7 Операція максимального пулінгу

Також, у будові ЗНМ, як правило, шар пулінгу вставляють між деякими послідовними шарами згортки. Але на даному етапі мережа може лише визначати множини ознак. Дані, що були отримані в результаті згортки і пулінга вже стають більш абстрактними, що були отримані після обробки вхідного зображення. Більше того, необхідно щоб мережа могла і класифікувати зображення якимось чином, а не просто виділяти певні ознаки. Для цього застосовують повнозв'язний шар. Отримані дані передають на звичайну мережу, яка також може складатися із декількох частин. Цей останній шар (рисунок 2.8) буде надавати ймовірнісні значення належності зображення до кожного із існуючих класів. І звідси вже можна буде отримати якісь висновки щодо задачі класифікації.

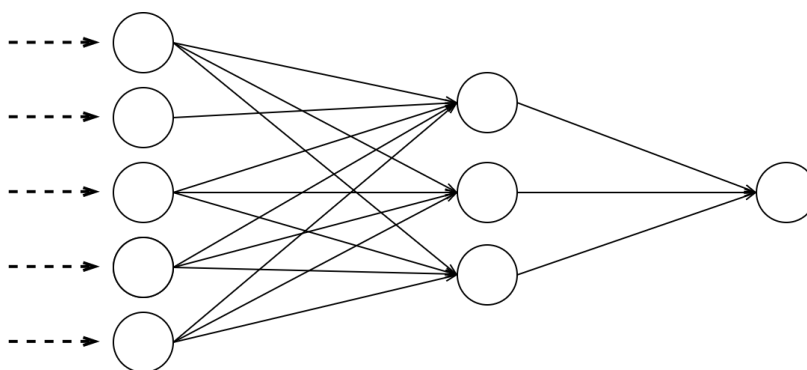


Рисунок 2.8 Структура повнозв'язної частини ЗНМ

До переваг згорткових мереж можна віднести наступні:

- Один із найкращих відомих алгоритмів розпізнавання зображень;
- Має меншу кількість ваг, що треба налаштовувати, у порівнянні із повнозв'язною мережею. Завдяки цьому інформація узагальнюється і спрощується, а не зберігається для кожного пікселя окремо;
- Інваріантність до здвигу та повороту картинки;
- Зручне розпаралелення обчислень, що дозволяє реалізувати алгоритм на графічних процесорах.

Також, можна виділити наступні недоліки:

- Велика кількість параметрів мережі, що треба налаштовувати;
- Не завжди зрозуміло, як правильно підібрати параметри для кожної задачі, що потрібно вирішити;
- Налаштування параметрів мережі істотно впливають на результат;
- Існують різні можливі конфігурації згорткової мережі, але нема рекомендацій, яку все таки краще обрати для конкретної задачі.

Отже, вище була наведена і розглянута будова простої згорткової нейронної мережі. Але, як відомо, їх існує багато різних типів, кожна з яких має свої переваги та певні недоліки.

Тому, у наступному підрозділі розглянемо саме різновиди більш складних згорткових нейронних мереж.

2.2.2 Вибір архітектури нейронної мережі для ІДІЗСР

2.2.2.1 Неокогнітрон

Неокогнітрон вважають попереником всіх згорткових нейронних мереж. Це багатошарова нейронна мережа, що була вперше представлена у 1979-1980 роках японським вченим Кунікіхо Фукусімою [37-38]. Дана мережа використовувалася для розпізнавання рукописних символів і не тільки. У майбутньому стала натхненням для створення згорткових нейронних мереж.

Неокогнітрон складається із S-шарів та C-шарів. Це аналоги до шарів згортки і пулінга у ЗНМ. Він не має повнозв'язних шарів. Натомість, шари S та C розміщені таким чином, що операція пулінгу, що виконується лише в останньому шарі, покриває все зображення і має лише один вихід на клітинну площину.

Так як у ті роки іще методу зворотного поширення похибки не було винайдено, Фукусіма визначив свої особисті функції для навчання мережі. Більше того, неокогнітрон використовує так звану інгібіторну клітинку, щоб зменшити вплив неактуального виходу.

Вважатимемо цю модель наближеною до згорткових мереж, опускаючи те, що вона використовує інші користувацькі функції, а не зворотнього поширення похибки і те, що тут є також і інгібіторні клітини, про які уже було сказано раніше.

Таким чином, модель неокогнітрона має таку будову:

- Згортково-підвибірковий шар номер 1. Він складається із 32 фільтрів, що мають розміри ядра 7×7 . Для підвибірки використовується операція максимального пулінгу із матрицею розміру 2×2 із кроком, рівним 2. Вхідний розмір дорівнює 28×28 , а вихідний - 14×14 .
- Згортково-підвибірковий шар номер 2. За функціональністю він нагадує попередній шар, але має менший розмір на вході. Тут також 32 фільтри, що мають розміри ядра 7×7 для згортки. Розмір підвибірки рівний також 2×2 із кроком 2. Вхідний розмір дорівнює 14×14 , а вихідний - 7×7 .
- Згортково-підвибірковий шар номер 3. Функціонально подібний до двох попередніх шарів. Містить в собі 32 фільтри із розміром ядра 7×7 для згортки. Для операції пулінгу використовується також матриця 2×2 із кроком, рівний 2. Але вхідний розмір дорівнює 7×7 , а вихідний - 4×4 .
- Згортково-підвибірковий шар номер 3 (він також є і вихідним шаром). Тут шар згортки має лише десять фільтрів. Тому, ми отримуємо

лише десять можливих виходів, тобто, вони відповідають десяти різноманітним класам. Зауважимо, що тут також можливо було зробити 32 фільтри і просто поставити їх у відповідність десятком класам. Також, вхідний розмір для шару підвибірki рівний 4×4 . Отже, використовується матриця для пулінгу 4×4 із кроком, рівним 4. Це означає, що береться одиничне значення для кожного фільтру.

Таким чином, дана будова мережі була найпростішою моделлю, яка дала поштовх у розвитку наступних типів згорткових мереж.

2.2.2.2 Згорткова мережа LeNet

Така згорткова 7-рівнева штучна нейронна мережа, що називається LeNet [40], вперше була запропонована Яном Лекуном у 1998 році. Вона є однією із перших ЗНМ, що була створена після багатьох спроб і різноманітних досліджень, що відбувалися, починаючи з 1988 року.

Спочатку Ян Лекун у 1989 році вперше застосував метод зворотного поширення помилки для навчання згорткового ядра задля того, щоб розпізнавати рукописні цифри.

Таке навчання було повністю автоматизоване і було кращим за ручне налаштування коефіцієнтів.

Згодом, це дослідження було успішно застосоване до розпізнавання ZIP-кодів – системи поштових індексів, що застосовується поштою США і послужило прототипом майбутній мережі LeNet.

І вже у 1998 році науковці [40] представили різноманітні методи розпізнавання рукописних цифр.

Вони проводили дослідження і порівнювали знайдені моделі. Ці дослідження мали величезний успіх і зібрали багато зацікавлених вчених навколо.

Хоча, на даний момент, дана будова мережі не має найкращі виконавчі результати, вона стала початковою точкою для багатьох майбутніх архітектур нейронних мереж, принісши у це поле досліджень великий науковий інтерес.

Розглянемо її будову.

- Перший шар - згортковий шар C1. На вхід подається зображення розміром 32×32 . Він складається із шести карт ознак, що мають ядро розміром 5×5 , крок рівний 1. На виході отримуємо карти ознак, розміром 28×28 . Шар містить 156 параметрів та 122'304 з'єднань;
- Другий – шар підвибірки S2. Він складається із 6 карт ознак, на виході вони будуть розміру 14×14 . Кожна структурна одиниця у кожній карті ознак приєднана до попереднього шару C1 за допомогою фільтру 2×2 із кроком 2. Даний шар має 12 параметрів, що можна навчити та 5'880 з'єднань;
- Третій шар – шар згортки C3. Він містить 16 карт ознак, що містять ядра розміром 5×5 і кроком, рівним 1. На виході отримується карти ознак розміром 10×10 . Шар містить 1'516 параметрів та 151'600 з'єднань;
- Четвертий – шар підвибірки S4. Подібний до шару S2. Містить 16 карт ознак. Кожен фільтр має розміри 2×2 і крок, рівний 2. На виході отримуються карти ознак розміром 5×5 . Шар має 32 параметрів та 2'000 з'єднань;
- П'ятий шар – шар згортки C5. Містить в собі 120 карт ознак. Розмір ядра - 5×5 із кроком 1. На виході отримуються карти ознак розміром 1×1 . Шар містить 48'120 з'єднань;
- Шостий – це звичайний повнозв'язний шар F6. Містить в собі 84 вузли. Також цей шар має 10'164 параметрів;
- Сьомий – повнозв'язний шар F7. Це останній шар даної мережі. Має в собі 10 виходів, так як результатом буде виділення 10 класів, що відповідають 10 цифрам [40].

На рисунку 2.9 представлена повна будова даної мережі, що була опублікована Яном Лекуном в його науковій роботі [40].

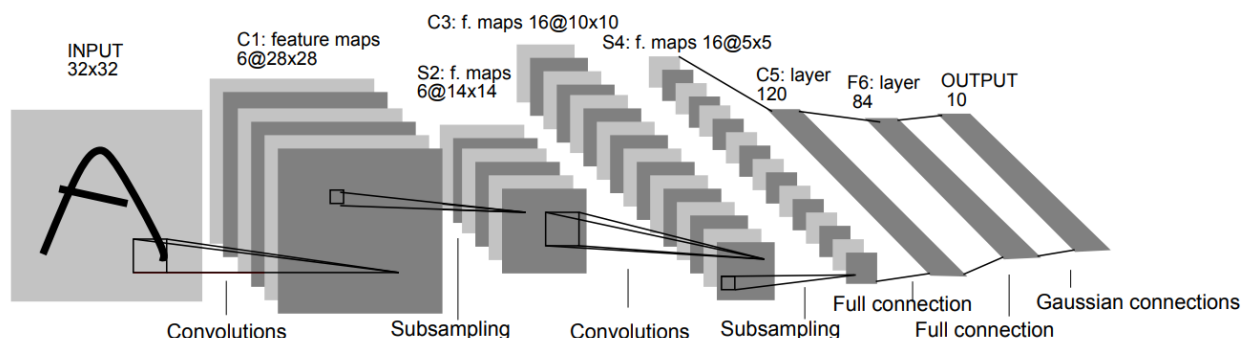


Рисунок 2.9 Будова мережі LeNet

2.2.2.3 AlexNet

У період із 1998 по 2010 роки розвиток згорткових мереж значно сповільнився. Більшість науковців не бачили у них сенсу чи просто робили повільні темпи у дослідженнях.

Але, з іншої сторони, ставало більше доступної інформації та даних, на яких можна було навчати мережу, також зростала потужність обчислювальної техніки. Вже у 2012 році група дослідників, на чолі із Алексом Крижевським розробила свою архітектуру згорткової нейронної мережі [41]. Була представлена під час змагань ILSVRC (ImageNet Large Scale Visual Recognition Challenge) [42], де вони і посіли перше місце.

І через рік після публікації даної статті всі учасники конкурсу почали використовувати згорткову нейронну мережу для вирішення задачі класифікації.

Будова даної мережі схожа на будову LeNet, але має більшу кількість фільтрів, у порівнянні із LeNet і тому вона була здатна класифікувати значно більшу кількість об'єктів. Розглянемо її.

- Перший шар – згортковий. Він містить фільтри розміром $224 \times 224 \times 3$ і використовує 96 ядер розміром $11 \times 11 \times 3$ з кроком, рівним 4 пікселям;
- Другий – також згортковий шар. Він приймає на вході результат, отриманий попереднім шаром. Також фільтрує отримані дані 256 ядрами, що мають розмір $5 \times 5 \times 48$;

- Третій – шар згортки. Містить в собі 384 ядер, розміром $3 \times 3 \times 256$;
- Четвертий – згортковий. Має 384 ядер, розміром $3 \times 3 \times 192$;
- П'ятий шар – згортковий. Містить в собі 256 ядер, розміром $3 \times 3 \times 192$;
- Шостий шар – повнозв'язний. Містить в собі 4096 нейронів;
- Сьомий шар – повнозв'язний. Містить в собі також 4096 нейронів;
- Восьмий шар – останній. Містить 1000 виходів, так як задано 1000 класів для розпізнавання.

На рисунку 2.10 представлена будова AlexNet, що була опублікована в науковій роботі Алекса Крижевського і його команди [41].

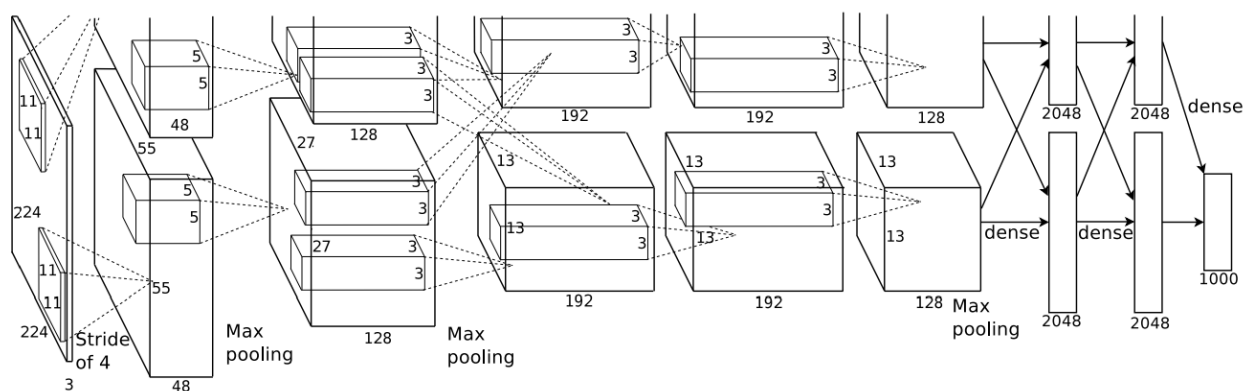


Рисунок 2.10 Будова AlexNet.

2.2.2.4 GoogLeNet (Inception)

Наступна мережа, що буде розглянута – це GoogLeNet [43]. Вона також, як і AlexNet, перемогла у конкурсі ILSVRC, але вже у 2014 році.

Дослідники поставили собі за мету зменшення обчислювального навантаження нейронних мереж, створивши GoogLeNet, першу версію ЗНМ Inception. Моделі глибинного навчання стали дуже корисними у вирішенні задачі класифікації різноманітних зображень чи відео після 2014 року.

З врахування таких нових можливостей, така велика корпорація, як Google, почала також задумуватися про створення своєї нейронної мережі.

Тому, автори [43], які були співробітниками Google, почали розробляти свою архітектуру. Спочатку, вони створили модуль, який назвали Inception (рисунок 2.11) [43].

Основна ідея полягала у тому, що замість того, щоб застосовувати згорткові шари із різними параметрами у різних частинах мережі, створити один загальний блок, який буде виконувати операції згортки.

Після цього, вони створили GoogLeNet, яка містила у собі таких дев'ять блоків Inception.

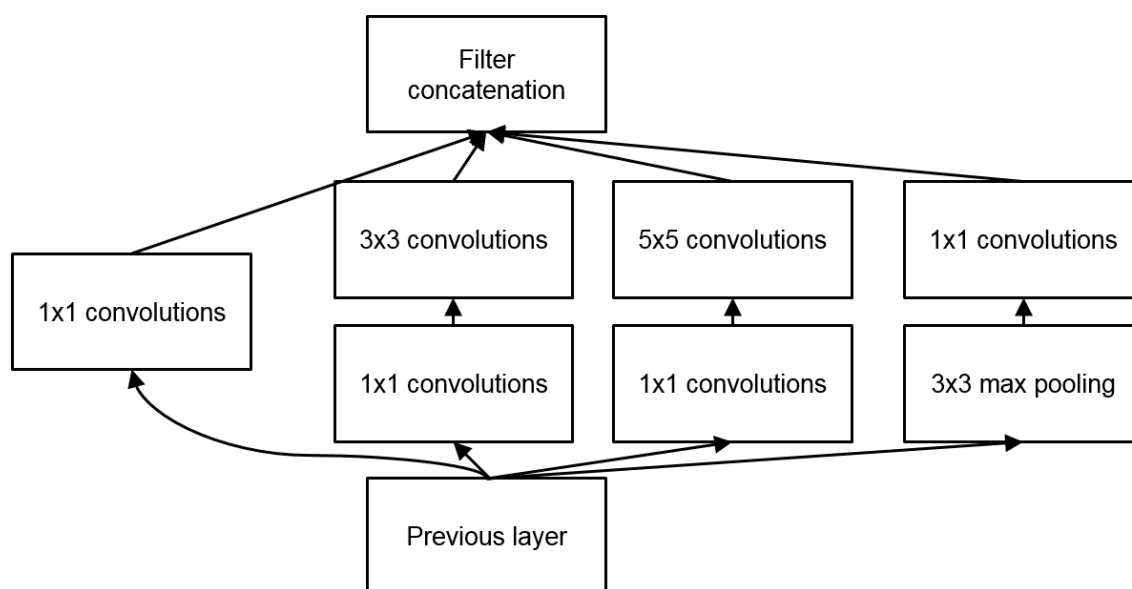


Рисунок 2.12 Модуль Inception

Не можна також не сказати, хоч ця мережа і складається із 22 шарів, але вона має меншу кількість параметрів, ніж AlexNet (4 мільйони проти 60 мільйонів).

2.2.2.5 Інші згорткові мережі

Також, існують і інші будови згорткових нейронних мереж. Серед них можна назвати такі:

- ZFNet (2013) [44]. Ця згорткова мережа стала переможцем конкурсу ILSVRC у 2013 році. Це була також вдосконалена версія AlexNet, яка

використовувала інші параметри, збільшивши розмір середнього згорткового шару і зменшивши крок та розмір фільтру на першому шарові. На рисунку 2.13 представлена її будова [44].

- VGGNet (2014) [45]. Ця мережа посіла друге місце на конкурсі ILSVRC у 2014 році. Кінцева версія мала 16 пар згорткових і повнозв'язних шарів та мала однакові розміри фільтрів для згортки - 3×3 і 2×2 для операції пулінгу.
- ResNet (2015) [46]. А ця мережа стала переможцем конкурсу ILSVRC у 2015 році. У даній архітектурі відсутні повнозв'язні шари у кінці мережі.

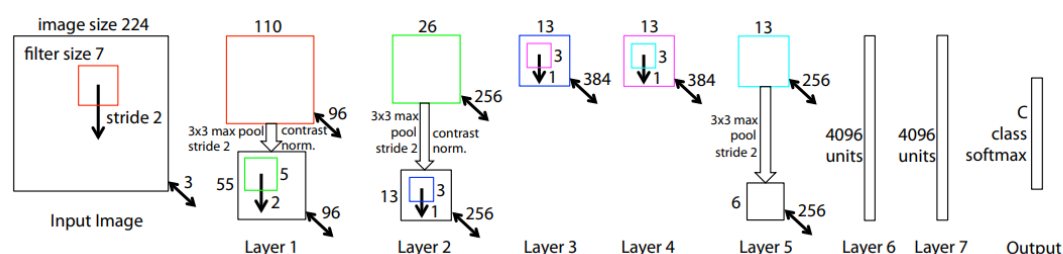


Рисунок 2.13 ZFNet

Можна зробити висновок, що для розпізнавання захворювань рослин буде обрана архітектура AlexNet через декілька причин. По-перше, вона є одним із лідерів у використанні для вирішення задач розпізнавання. Також, вона є «піонером» серед інших архітектур, що були створені для даної проблеми. Більше того, має відносно інших нових ЗНМ просту архітектуру. Тому саме ця архітектура буде реалізована в створюваному додатку

2.3 Проектування БД

2.3.1 Аналіз особливостей предметної області

Перед тим, як розробити повну БД, що буде містити всю необхідну інформацію, потрібно детальніше дослідити предметну область (ПрО). Як вже відомо, що набір даних містить 38 різних класів, серед яких 26 типів хворих та

14 типів здорових рослин. БД матиме інформацію про всі 38 класів, про хвороби, які там зустрічаються, про препарати, якими можна буде лікувати с/г культури.

Кожне захворювання може лікуватися декількома видами препаратів. Так само, як і один і той же препарат може використовуватися до різних хвороб, але відповідно до інструкції застосування.

Також, БД міститиме записи про історію випадків класифікації та історію лікувань хворих рослин певними препаратами.

Користувач інтелектуального додатку повинен мати змогу переглянути всі випадки класифікації рослин, отримати статистичний звіт щодо випадків захворювань для подальшого аналізу.

Також, він має робити певні записи до БД про лікування хвороб певними препаратами. Більше того, користувач має отримувати доступ до всієї історії лікування, знову ж таки, для подальшого аналізу свого сільського господарства.

Такі можливості надає MySQL – система управління базами даних (СУБД), яка дуже часто вживана у поєднанні із Python.

2.3.2 MySQL - середовище реалізації БД

Для розробки БД було вибране середовище MySQL. Це вільна система управління базами даних, яка є дуже швидкою, надійною та легкою у використанні. Проектування БД відбувалося у середовищі Workbench. Це спеціальний офіційний інструмент для кращого візуального бачення БД при її створенні.

Для проектування БД було обрано версію MySQL Workbench 8.0.20 та MySQL Server 8.0.20 для архітектури комп'ютера x64.

2.3.3 Фізична модель БД

Тепер, розглянемо фізичну модель БД, що була створена у середовищі MySQL Workbench (рисунок 2.14). Для цього опишемо таблиці.

- DRUG (препарат). Ця таблиця містить необхідну інформацію про препарати, які можна застосовувати для лікування с/г рослин;

- DISEASE (хвороба). Таблиця містить інформацію про різновиди захворювань, що можуть бути класифіковані;
- DRUG_and_DISEASE (препарат та хвороба). Ця таблиця містить інформацію про відповідність хвороб та препаратів, якими можна скористатися для лікування с/г рослини;
- CASE_OF_THE_DISEASE (випадок захворювання). У даній таблиці створюються записи всіх випадків класифікації с/г рослини (як здорові, так і хворі культури);
- TREATMENT_HISTORY (історія лікування). Ця таблиця містить інформацію про те, як і яка рослина була пролікована різними препаратами.
- CLASS (клас). Тут знаходиться список можливих класів при розпізнаванні рослини (як здорові, так і хворі культури).

Тепер, опишемо складові кожної із таблиць. Результат представлено у таблиці 2.1. У таблиці прийняті такі позначення ключів: РК – первинний ключ (primary key); FK – зовнішній ключ (foreign key).

Також є наступні типи даних: int – ціле число, varchar() – символічне значення, text() – текстове значення, datetime() – дата та час, float – дійсне число.

Таблиця 2.1 Склад та характеристики атрибутів таблиць

Атрибут	Опис українською	Тип даних	Ключ	Обов'язкове значення	Унікальний індекс	Автоінкрементне поле
Таблиця DRUG						
drug_id	ІД препарату	int	РК	+	+	+
drug_name	назва препарату	varchar(90)		+		
pharma_company_name	назва виробника препарату	varchar(100)		+		
drug_description	опис препарату	text(20000)		+		

Продовження таблиці 2.1

Таблиця DISEASE						
disease_id	ІД хвороби	int	PK	+	+	+
disease_name	назва хвороби	varchar(90)		+		
plant_pathogen	збудник хвороби	varchar(100)		+		
symptoms	симптоми	text(20000)		+		
Таблиця DRUG_and_DISEASE						
DRUG_drug_id	ІД препарату	int	FK	+		
DISEASE_disease_id	ІД хвороби	int	FK	+		
instruction_description	опис застосування препарату під певну хворобу	text(20000)		+		
Таблиця CLASS						
class_id	ІД класу	int	PK	+	+	
class_name	назва класу	varchar(90)		+		
alias_class_name	спрощена назва класу	varchar(90)		+		
DISEASE_disease_id	ІД хвороби	int	FK		+	
Таблиця CASE_OF_THE_DISEASE						
case_id	ІД випадку	int	PK	+	+	+
case_date_time	час класифікованого випадку	datetime(6)		+		
probability	ймовірність належності до класу	float		+		
CLASS_class_id	ІД класу	int	FK	+		
Таблиця TREATMENT_HISTORY						
treatment_id	ІД лікування	int	PK	+		+
treatment_date_time	час лікування	datetime(6)		+		
CASE_OF_THE_DISEASE_case_id	ІД випадку захворювання	int	FK	+		
DRUG_drug_id	ІД препарату	int	FK	+		
treatment_result	результат лікування	text(20000)		+		

Також, розглянемо таблицю опису зв'язків для даної схеми, що були позначені на фізичній моделі як R1-R6. Результати представлені у таблиці 2.2.

Таблиця 2.2 Опис зв'язків схеми

Назва зв'язку	Відношення	Тип відношення	Опис
R1	1:n	ідентифікуюче	Один препарат, але застосовувати можна застосовувати до багатьох хвороб.
R2	1:n	ідентифікуюче	Одна хвороба, але можна лікувати багатьма препаратами.
R3	1:1	неідентифікуюче	Один клас – одна хвороба (або її відсутність).
R4	1:n	ідентифікуюче	Один клас може зустрічатися у багатьох випадках класифікації.
R5	1:n	ідентифікуюче	Один випадок захворювання можна лікувати по різному.
R6	1:n	ідентифікуюче	Один препарат можна застосовувати до багатьох випадків.

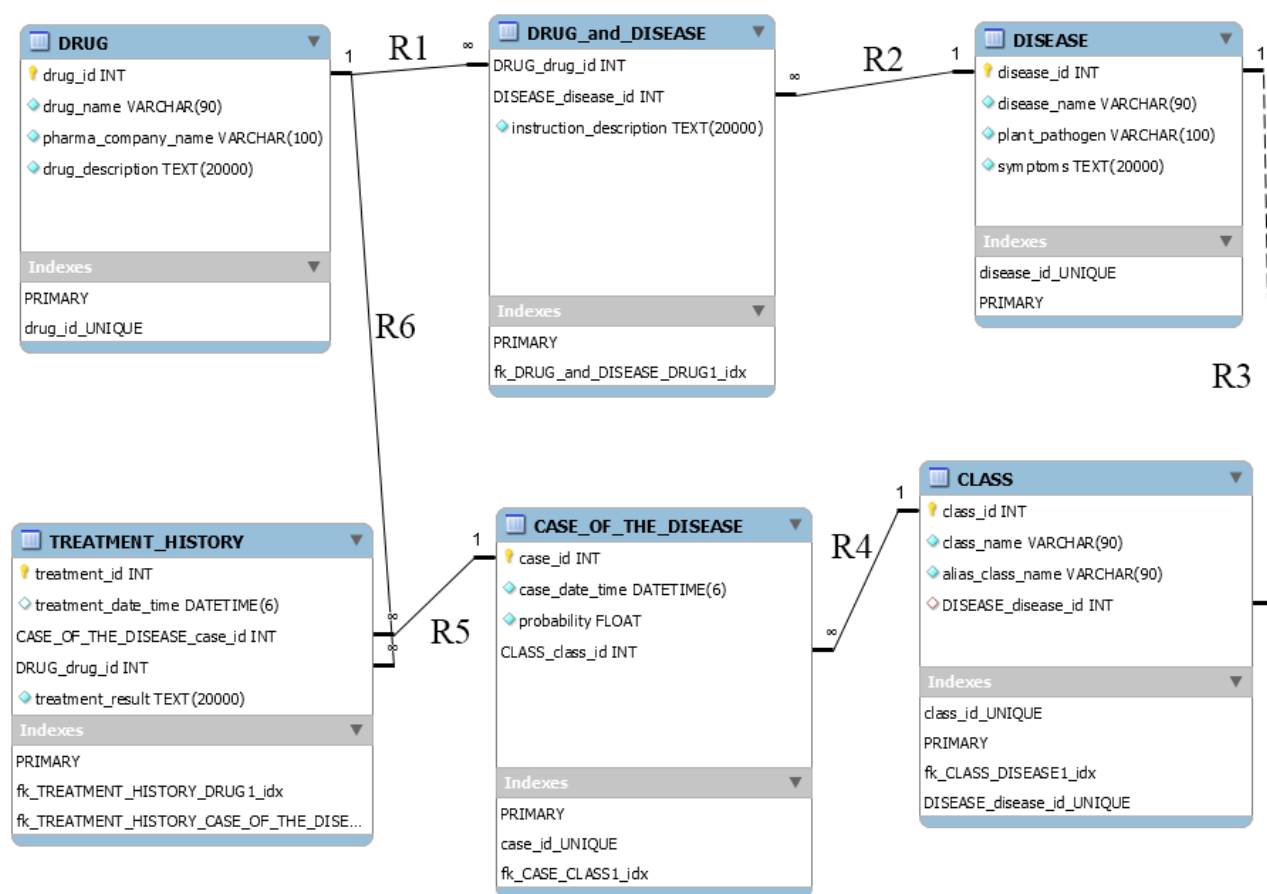


Рисунок 2.14 Фізична модель БД

Після цього, таблиці у MySQL Workbench були заповнені відповідною інформацією. Для того, щоб можна було взаємодіяти із БД (робити запити, чи створювати нові записи), необхідно створити БД на сервері та скопіювати уже готову БД та всі її дані таблиць. Як вже було вказано вище, це буде MySQL Server (рисунок 2.15).

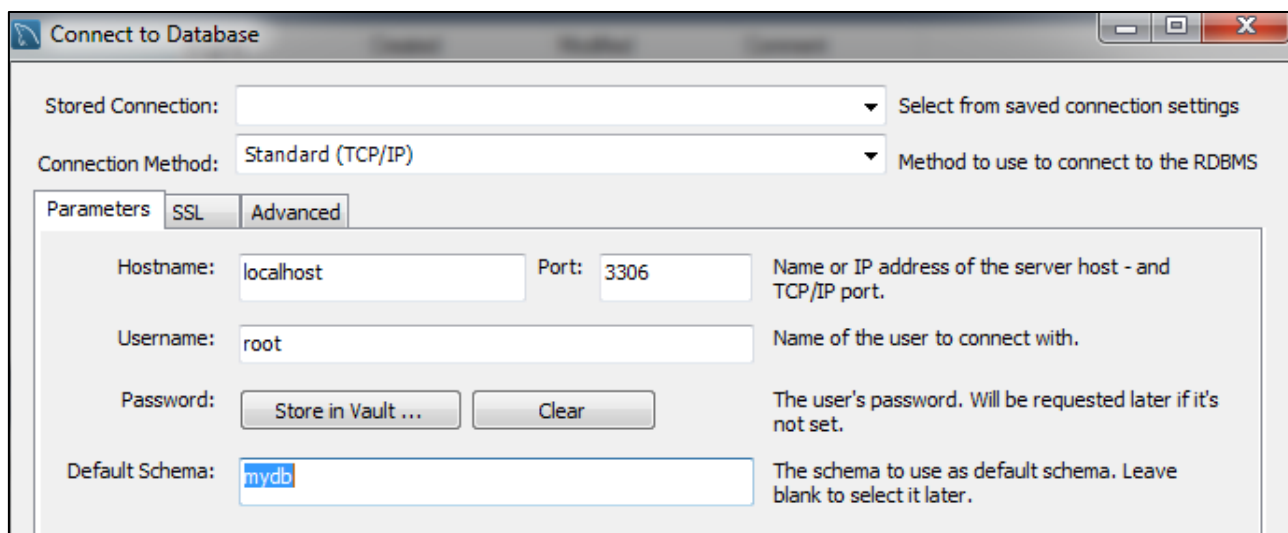


Рисунок 2.15 Підключення до БД на сервері

Після того, як відбудеться успішне створення БД на сервері, синхронізація вже раніше створеної моделі із тою, що зберігається на сервері, отримаємо кінцеву готову БД (рисунок 2.16):

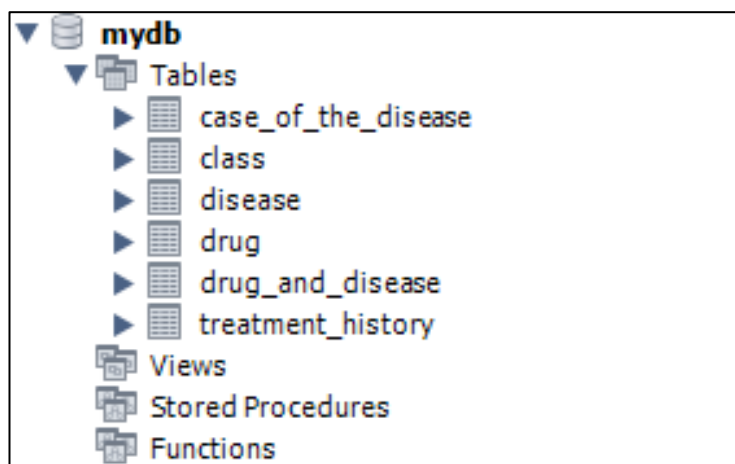


Рисунок 2.16 Готова БД на сервері MySQL Server

2.4 Розробка інтерфейсу інтелектуального додатку

Виділимо наступні сторінки, що матиме додаток (рисунки 2.17 – 2.25):

- «Домашня сторінка» (PLANT DISEASE AI DETECTION),
- «Головна сторінка» (MAIN-1, MAIN-21, MAIN-22),
- «Історія випадків» (CASE HISTORY),
- «Додати запис про лікування» (ADD TREATMENT),
- «Історія лікування» (TREATMENT HISTORY),
- «Статистика» (STATISTICS),
- «Про додаток» (ABOUT).

На рисунку 2.15 зображено макет сторінки PLANT DISEASE AI DETECTION. Вона містить основну інформацію про створену інтелектуальну систему.

Далі, на рисунку 2.16, показано макет сторінки MAIN-1. На ній розміщені дві кнопки (CHOOSE A PHOTO та ANALYZE), за допомогою яких можна буде вибрати фото та відправити зображення с/г рослини на класифікацію відповідно. Також, сторінка містить попередній вигляд зображення, який буде розпізнаватися, щоб користувачу було краще зрозуміло, інформацію про яке фото саме він хоче отримати. Після того, як зображення буде проаналізовано, буде отримано результат класифікації (MAIN-21 чи MAIN-22), рисунки 2.17 – 2.18. Якщо зображення буде таким, що міститиме хвору рослину, то користувач побачить всю необхідну інформацію: ймовірність хвороби, назву збудника, опис хвороби та система запропонує йому можливі препарати, за допомогою яких можна буде лікувати рослину (назва препаратів, фірма-виробник, опис препарату та інструкція щодо застосування препарату саме для даного захворювання). Якщо ж зображення містить здорову рослину, то користувач побачить ймовірність того, що це певна здорова рослина. І, тому, для такої с/г культури лікування не потрібне. Також, дані сторінки, як MAIN-21, так і MAIN-22 містять кнопку «TRY AGAIN» для того, щоб можна було проаналізувати нову рослину чи, наприклад, повторити класифікацію знову тієї рослини, що була щойно

проаналізована. Більше того, після того, як зображення рослини буде класифіковане, відповідний запис буде записано до таблиці CASE_OF_THE_DISEASE у БД MySQL Server.

На рисунку 2.19 представлено макет сторінки CASE HISTORY, що буде відображати саме цю інформацію про попередні випадки, що були класифіковані системою, що буде взято із таблиці CASE_OF_THE_DISEASE.

Далі, на рисунку 2.20, представлено макет сторінки ADD TREATMENT, на якій можна буде додати запис про випадок захворювання та відповідний препарат, який був застосовано.

Для того, щоб мати більш детальну інформацію про результат лікування, користувач зможе записати додаткову інформацію у текстовому полі. Запис відбувається до таблиці TREATMENT_HISTORY у БД MySQL Server.

Також, на рисунку 2.21 представлено макет сторінки, що буде містити всю необхідну інформацію про результат історії лікування. Вона відображає всі записи із таблиці TREATMENT_HISTORY.

Для кращого сприйняття класифікованих випадків, буде побудовано гістограму, що буде відображатися на сторінці STATISTICS (макет на рисунку 2.22).

Більше, на рисунку 2.23 представлено макет сторінки ABOUT, що буде містити більш детальну інформацію про систему, про те, які рослини можна досліджувати тощо. Також, там будуть кнопки (CONTACT VIA E-MAIL, UNIVERSITY FACULTY, PRIVACY POLICY), за допомогою яких можна буде написати лист розробнику, перейти на веб-сторінку факультету інформаційних технологій та можна буде ознайомитися із Політикою конфіденційності відповідно.

Також, на рисунку 2.31 представлена діаграма переходів із сторінки на сторінку. Користувач матиме змогу пересуватися із сторінки на будь-яку сторінку. За винятком MAIN-21, MAIN-22 – туди можна буде потрапити лише тоді, коли буде отримане і класифіковане зображення (чи хворої, чи здорової рослини).

PLANT DISEASE AI DETECTION		
MAIN	MORE ▼	ABOUT
CASE HISTORY ADD TREATMENT TREATMENT HISTORY STATISTICS TEXT		

Рисунок 2.23 PLANT DISEASE AI DETECTION

PLANT DISEASE AI DETECTION		
MAIN	MORE ▼	ABOUT
IMAGE PREVIEW CHOOSE A PHOTO ANALYZE		

Рисунок 2.24 MAIN-1

PLANT DISEASE AI DETECTION

MAIN

MORE ▼

ABOUT

IMAGE PREVIEW

CLASSIFICATION RESULT	CLASSIFICATION RESULT												
PATHOGEN	PATHOGEN												
DISEASE DESCRIPTION	DISEASE DESCRIPTION												
POSSIBLE TREATMENT	<table><tr><td>DRUG NAME</td><td>PHARMA COMPANY NAME</td><td>DRUG DESCRIPTION</td><td>INSTRUCTION DESCRIPTION</td></tr><tr><td>DRUG NAME</td><td>PHARMA COMPANY NAME</td><td>DRUG DESCRIPTION</td><td>INSTRUCTION DESCRIPTION</td></tr><tr><td>DRUG NAME</td><td>PHARMA COMPANY NAME</td><td>DRUG DESCRIPTION</td><td>INSTRUCTION DESCRIPTION</td></tr></table>	DRUG NAME	PHARMA COMPANY NAME	DRUG DESCRIPTION	INSTRUCTION DESCRIPTION	DRUG NAME	PHARMA COMPANY NAME	DRUG DESCRIPTION	INSTRUCTION DESCRIPTION	DRUG NAME	PHARMA COMPANY NAME	DRUG DESCRIPTION	INSTRUCTION DESCRIPTION
DRUG NAME	PHARMA COMPANY NAME	DRUG DESCRIPTION	INSTRUCTION DESCRIPTION										
DRUG NAME	PHARMA COMPANY NAME	DRUG DESCRIPTION	INSTRUCTION DESCRIPTION										
DRUG NAME	PHARMA COMPANY NAME	DRUG DESCRIPTION	INSTRUCTION DESCRIPTION										

TRY AGAIN

Рисунок 2.25 MAIN-21

PLANT DISEASE AI DETECTION

MAIN

MORE ▼

ABOUT

IMAGE PREVIEW

CLASSIFICATION RESULT	CLASSIFICATION RESULT
-----------------------	-----------------------

TRY AGAIN

Рисунок 2.26 MAIN-22

PLANT DISEASE AI DETECTION				
MAIN		MORE ▼	ABOUT	
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CASE (№)	DATE	PROBABILITY	CLASSIFICATION RESULT
	CONTACT VIA E-MAIL		UNIVERCITY FACULTY	PRIVACY POLICY

Рисунок 2.27 CASE HISTORY

PLANT DISEASE AI DETECTION		
MAIN	MORE ▼	ABOUT
CASE OF THE DISEASE CASE 1 CASE 2 DRUG NAME DRUG 1 DRUG 2 ADD A RECORD TO DB GO TO TREATMENT HISTORY		
		INPUT TEXT

Рисунок 2.28 ADD TREATMENT

[illegible]

Рисунок 2.29 TREATMENT HISTORY

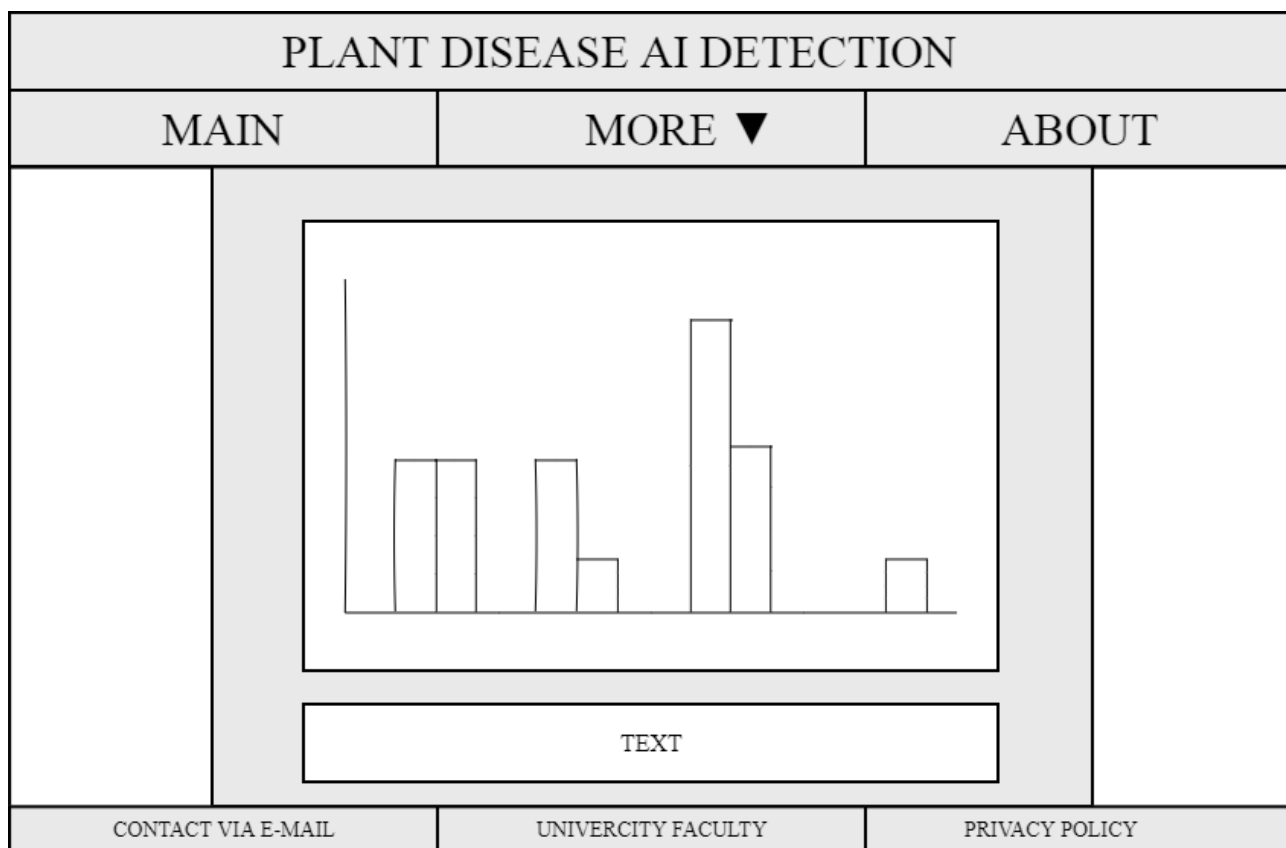


Рисунок 2.30 STATISTICS

PLANT DISEASE AI DETECTION		
MAIN	MORE ▼	ABOUT
TEXT		
CONTACT VIA E-MAIL	UNIVERCITY FACULTY	PRIVACY POLICY

Рисунок 2.31 ABOUT

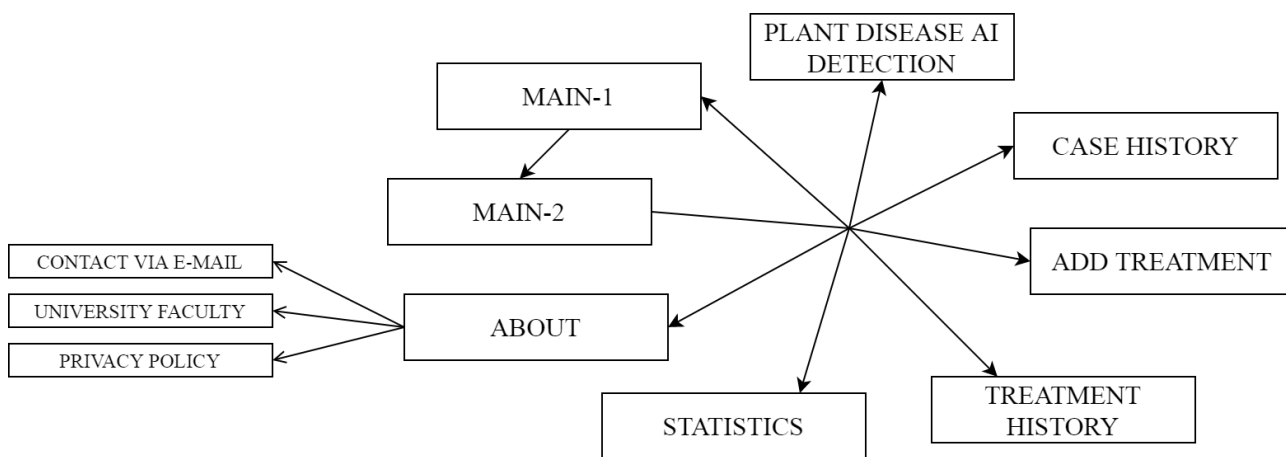


Рисунок 2.31 Діаграма переходів із сторінки на сторінку

3 РЕАЛІЗАЦІЯ ІНТЕЛЕКТУАЛЬНОГО ДОДАТКУ, ТЕСТОВИЙ ПРИКЛАД

3.1 Вибір середовища реалізації проекту

Для розробки програмного коду апарату розпізнавання зображень можна використовувати різноманітні мови програмування, якщо розуміти загальну теорію нейронних мереж. У даній роботі було створено нейронну мережу із застосуванням Python[47]. Як відомо, вона належить до інтерпретованих об'єктно-орієнтованих мов програмування [48]. До переваг Python можна віднести [49]:

- Низький поріг входження. Синтаксис Python більш зрозумілий для вивчення мов програмування, чим, наприклад, C++;
- Широке застосування. Python може використовуватися для розробки веб-додатків, цікавих ігор, для різних автоматизації застосунків, математичних обчислень, машинного навчання тощо;
- Python, на даний момент, є дуже популярною мовою програмування;
- Python взаємодіє із багатьма різними застосунками: MySQL, Excel тощо;
- Тут існує багато різноматіних бібліотек, які можна використовувати для статистичних обчислень та візуалізації даних чи для машинного навчання. Наприклад, таких як:
 - Matplotlib [50]. Використовується для візуалізації даних, побудови графіків та різних діаграм;
 - Numpy [51], для обробки великих об'ємів даних, масивів та таблиць;
 - TensorFlow [52]. Це бібліотека, що використовується для машинного навчання;
 - Keras [53]. Створена для проектів, що містять нейронні мережі та глибинне навчання. Може використовуватися поверх TensorFlow;

- PyTorch [54]. Ця відкрита бібліотека також створена для машинного навчання;
- Django [55]. Це Python-фреймворк для створення веб-додатків;
- Flask [56]. Також фреймворк для створення веб-застосунків за допомогою Python;
- тощо.

Для створення класифікатора було розглянуто три бібліотеки для Python – Keras, Tensorflow та PyTorch. Якщо їх всі порівнювати, то можна сказати, що продуктивність менша у бібліотеки Keras, у порівнянні із Tensorflow та PyTorch. Із точки зору відладки коду, то PyTorch має кращі можливості, чим інші дві бібліотеки.

Також, Keras використовується для маленьких наборів даних, оскільки він порівняно повільніше. Тим часом, TensorFlow та PyTorch використовуються програмістами для різних високопродуктивних моделей та великих наборів даних, які потребують швидкого виконання. І так як PyTorch містить більше зручних засобів для створення, використання та навчання моделі ЗНМ AlexNet, було вирішено використовувати саме її.

Для організації серверної сторони додатку було розглянуто дві бібліотеки, а саме – Django та Flask для порівняння. Наприклад, Flask має модульну структуру, тоді як Django поєднує все разом. Щодо їх продуктивності, то можна сказати, що тут вони мають приблизно однакові значення. Django був розроблений для швидкого розвитку складних веб-додатків. Він надає розробникам необхідні інструменти для реалізації масштабованого та підтримуваного функціоналу. З іншого боку, простота Flask дозволяє розробникам швидше створювати менші додатки. У кінці все ж було обрано використовувати Flask для реалізації серверної сторони додатку.

3.2 Python-реалізація згорткової мережі

Для того, щоб навчити ЗНМ на датасеті, як вже було сказано раніше, існує два можливих варіанти – навчання з нуля та трансферне навчання. Було вирішено, що буде використовуватися саме трансферне навчання, адже, як показали дослідження деяких попередніх робіт, воно показало найкращі результати у розпізнаванні. Тому, використовувана модель AlexNet уже попередньо навчена на 1000-класовому наборі даних ImageNet, про який уже була згадка раніше. Отже, будова AlexNet, яка була описана вже раніше, у PyTorch має наступний вигляд (рисунок 3.1):

```
AlexNet(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ReLU(inplace=True)
    (10): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): ReLU(inplace=True)
    (12): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(6, 6))
  (classifier): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=9216, out_features=4096, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=4096, out_features=4096, bias=True)
    (5): ReLU(inplace=True)
    (6): Linear(in_features=4096, out_features=38, bias=True)
  )
)
```

Рисунок 3.1 – будова мережі AlexNet

Взагалі, всі попередньо навчені згорткові моделі у даній бібліотеці, мають отримувати на вхід однаково нормалізовані зображення, тобто кожне фото має бути представлено у 3-канальному RGB вигляді, розмірами $3 \times H \times P$, де H та W мають бути, як мінімум, рівними 224.

Зображення мають бути завантажені у діапазон $[0,1]$ і тоді нормалізовані, використовуючи конкретні значення $\text{mean} = [0.485, 0.456, 0.406]$ та $\text{std} = [0.229, 0.224, 0.225]$, де mean та std – це середнє значення та стандартне відхилення відповідно.

3.3 Опис структури датасету

Перед тим, як навчати ШНМ для класифікації зображень, повернемося до набору даних Plant Village та розглянемо його детальніше. Датасет містить три різноманітних категорії зображень однакового змісту: кольорові зображення, сірі зображення та сегментовані (рисунок 3.2):

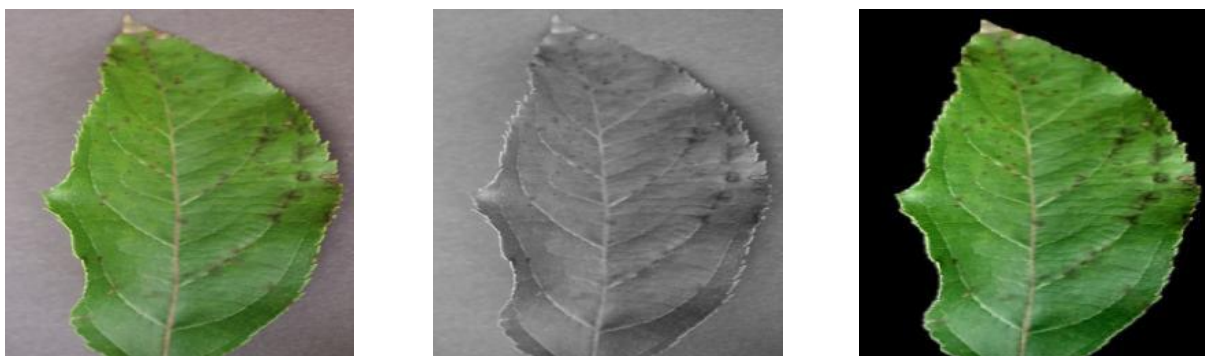


Рисунок 3.2 Різновиди фото у наборі даних

Було вирішено, що для навчання мережі буде використовуватися саме сегментована категорія зображень, так як вона містить лише листок рослини без непотрібного фону, який може, у подальшому, при тестуванні не вірно розпізнаватися. Категорія із сірими зображеннями також не дуже підходить, адже коли різні захворювання просто можуть мати однакову форму плям, саме колір рослини може вказувати на тип захворювання. Як вже відомо, весь набір даних містить 54'306 зображень. Для створення навчальної та тестової вибірок було вирішено поділити датасет у відношенні 80% до 20% відповідно за допомогою нескладного Python-коду.

Набір даних містить наступний перелік різних класів (рисунок 3.3):

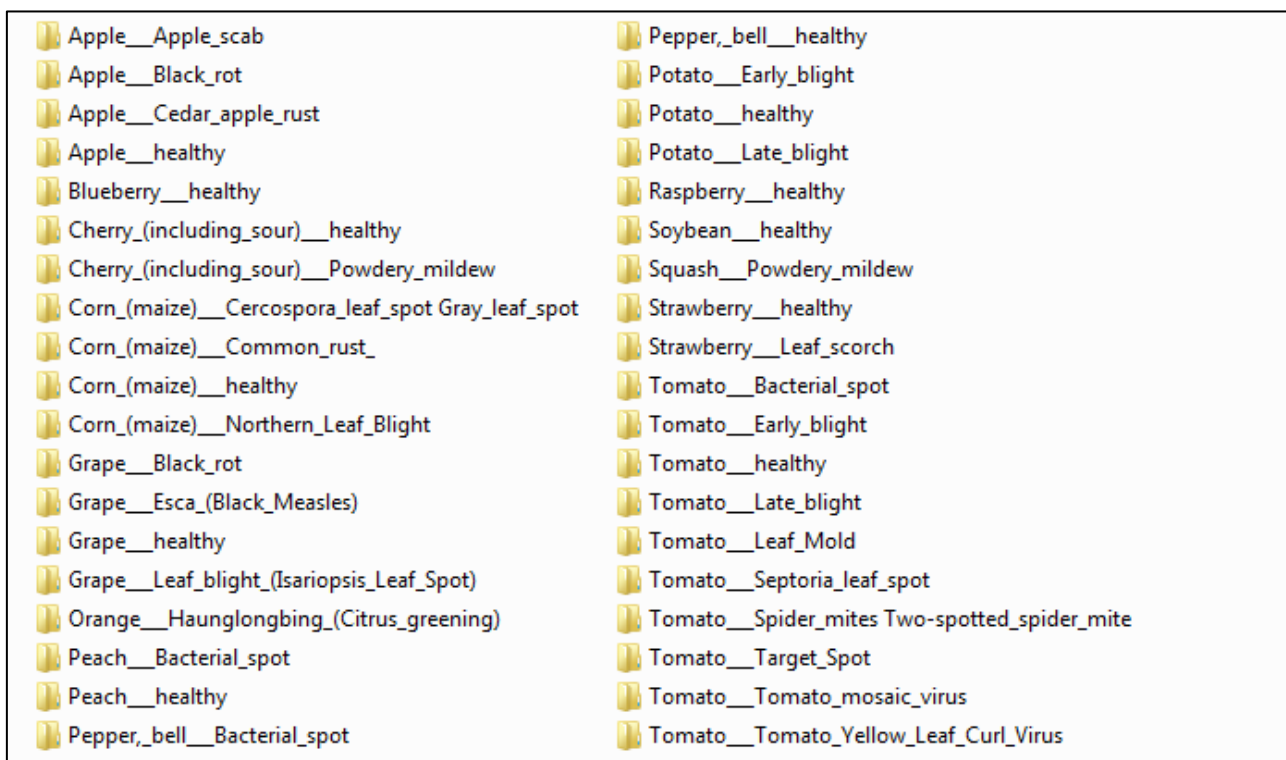


Рисунок 3.3 Всі теки із зображеннями рослин у датасеті

3.4 Вибір оптимальної моделі мережі

У даній роботі буде розглянуто два типи трансферного навчання – «тонке налаштування» (finetuning) та «вилучення ознак» (feature extracting). Як відомо, у процесі «тонкого налаштування» починаємо з попередньо навченої моделі і оновлюємо всі її параметри для нової задачі, тобто, перенавчаємо всю модель. А під час «вилучення ознак» починаємо з попередньо навченої моделі і оновлюємо лише ваги остаточного шару, з яких отримуємо прогнози та результати класифікації. Цей метод так називається, тому що використовується попередньо навчена ЗНМ як фіксований об'єкт для вилучення певних ознак і у результаті, змінюється тільки вихідний шар.

Як правило, обидва способи трансферного навчання виконують декілька однакових кроків:

- Ініціалізація попередньо навченої моделі;
- Зміна розміру вихідного шару (шарів), щоб кількість виходів співпадала із кількістю досліджуваних класів;

- Визначення, які параметри моделі треба перенавчити;
- Навчання мережі.

Для навчання мережі було розглянуто чотири різноманітних варіанти (Моделі 1-4), вхідні налаштування представлені у Таблиці 3.1. Всі моделі запускалися на центральному процесорі (CPU). Далі окремо буде розглянуто їх детальний опис.

Таблиця 3.1 Порівняльна характеристика деяких налаштувань ЗНМ AlexNet

	Модель-1	Модель-2	Модель-3	Модель-4
ОЗП (ГБ)	8	8	16	16
PyTorch Version	1.4.0	1.4.0	1.5.0	1.5.0
Torchvision Version	0.5.0	0.5.0	0.6.0	0.6.0
Процесор	CPU	CPU	CPU	CPU
Кількість епох	15	13	20	40
Розмір партії зображень (batch size)	64	64	64	128
Метод	feature extracting	feature extracting	feature extracting	finetuning
Параметри, що навчаються	classifier.6.weight classifier.6.bias	classifier.6.weight classifier.6.bias	classifier.6.weight classifier.6.bias	features.0.weight features.0.bias features.3.weight features.3.bias features.6.weight features.6.bias features.8.weight features.8.bias features.10.weight features.10.bias classifier.1.weight classifier.1.bias classifier.4.weight classifier.4.bias classifier.6.weight classifier.6.bias
Час навчання	близько 600 хв	494 хв 59 сек	237 хв 6 сек	950 хв 47сек
Результат навчання (точність)	-	0.884045	0.897594	0.987833

3.4.1 Модель-1

Перша модель AlexNet мала 15 епох та використовує метод feature extracting, розмір партії зображень – 64, ОЗП 8 ГБ. На рисунках 3.4 – 3.5 представлена функції втрат та точності для даної моделі (як для навчальної, так і для тестової вибірки):

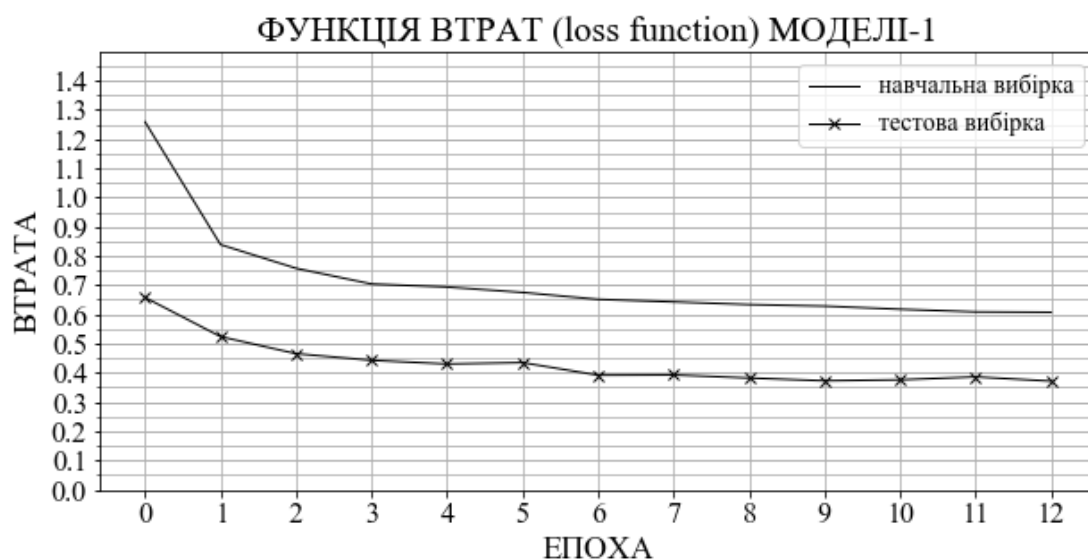


Рисунок 3.4 Функція втрат Моделі-1

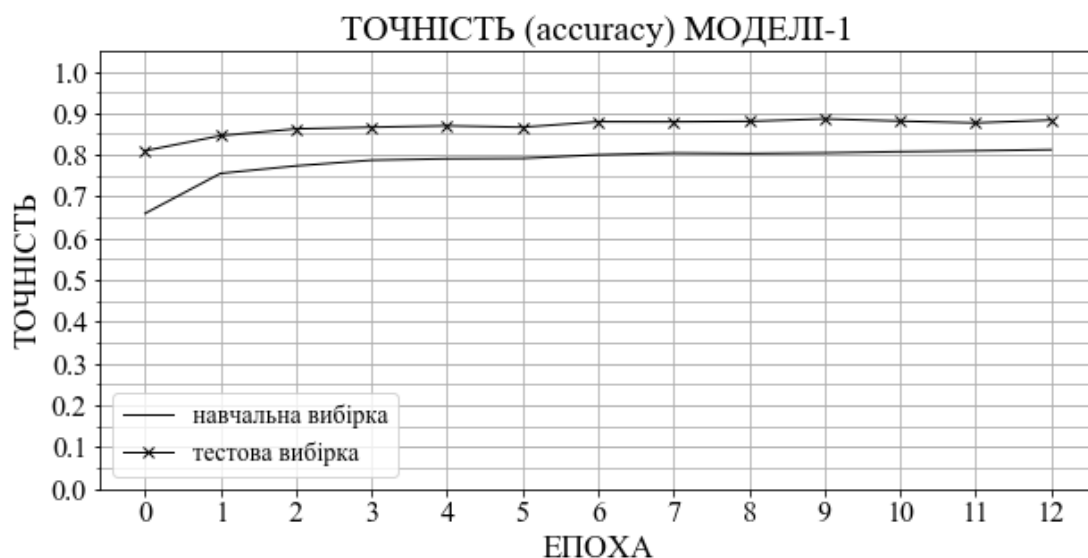


Рисунок 3.5 Точність Моделі-1

Як ми бачимо, із заявлених 15 епох, лише 0-12 були навчені, а далі програма перестала працювати через недостатчу ОЗП на ноутбучі. Найкраща точність = 0.8862.

3.4.2 Модель-2

У наступний запуск моделі було вирішено зменшити кількість епох до 13, все інше залишилось без змін. Результати представлено на рисунках 3.6 – 3.7.

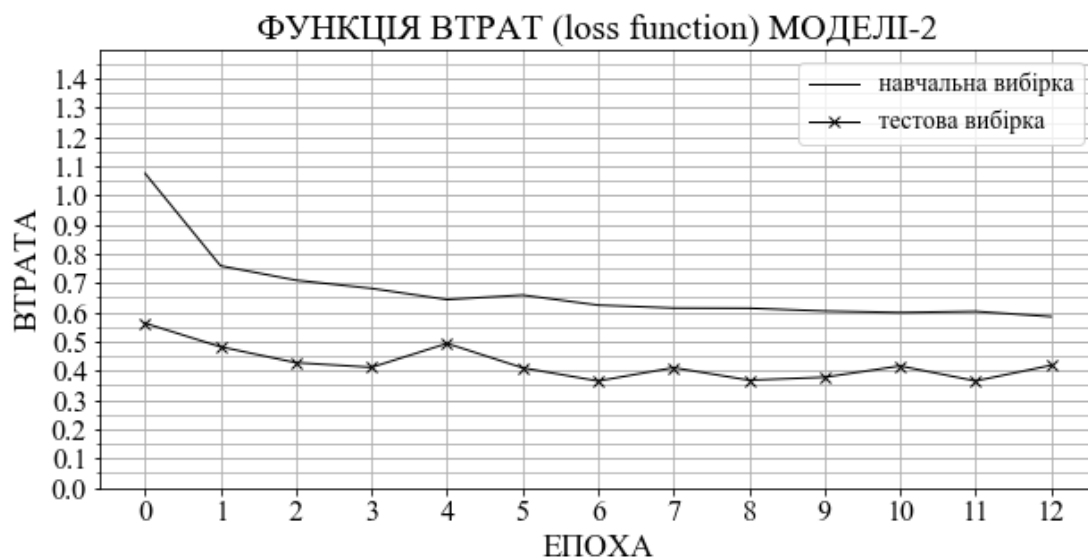


Рисунок 3.6 Функція втрат Моделі-2

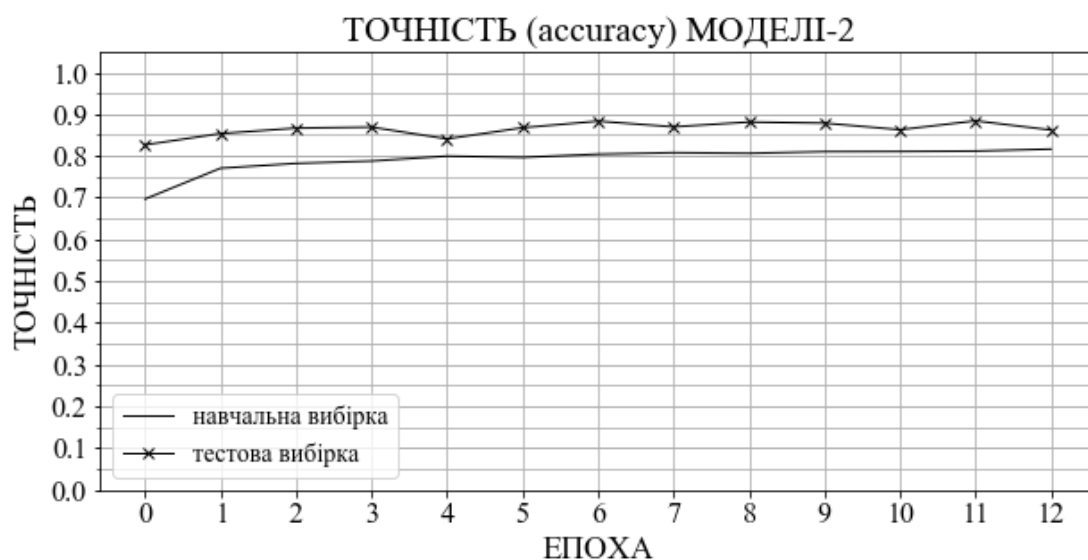


Рисунок 3.7 Точність Моделі-2

Навчання даної моделі завершилося успішно через 494 хв 59 сек. Найкраща точність = 0.884045.

Так як така точність не задовільнила бажаний результат, було вирішено запустити ще раз модель, але з іншими налаштуваннями комп'ютера.

3.4.3 Модель-3

Для третьої спроби був обраний інший комп'ютер, який мав 16 ГБ ОЗП. Також, кількість епох було вирішено збільшити до 20. Результати на рисунках 3.8 – 3.9.

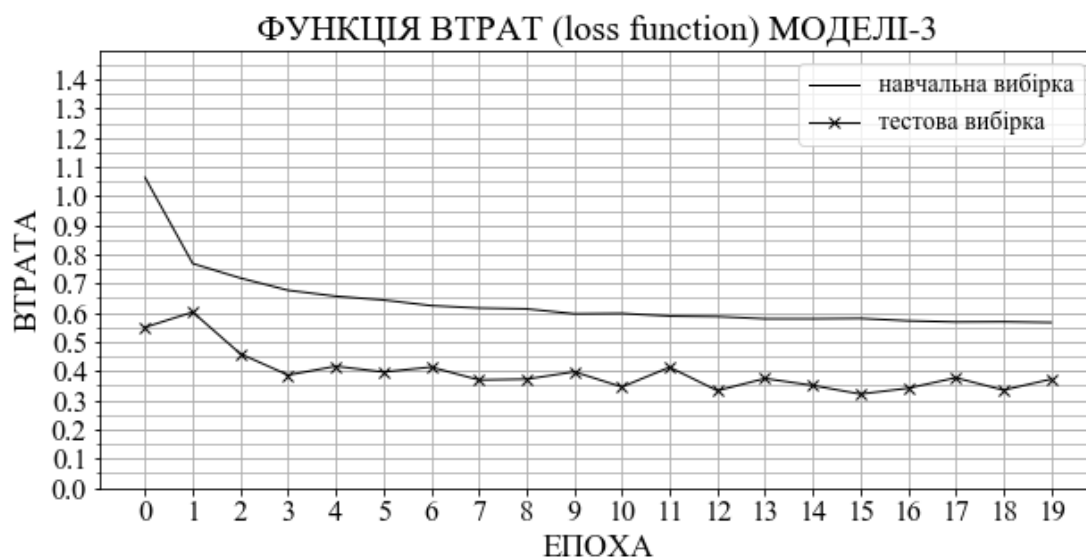


Рисунок 3.8 Функція втрат Моделі-3

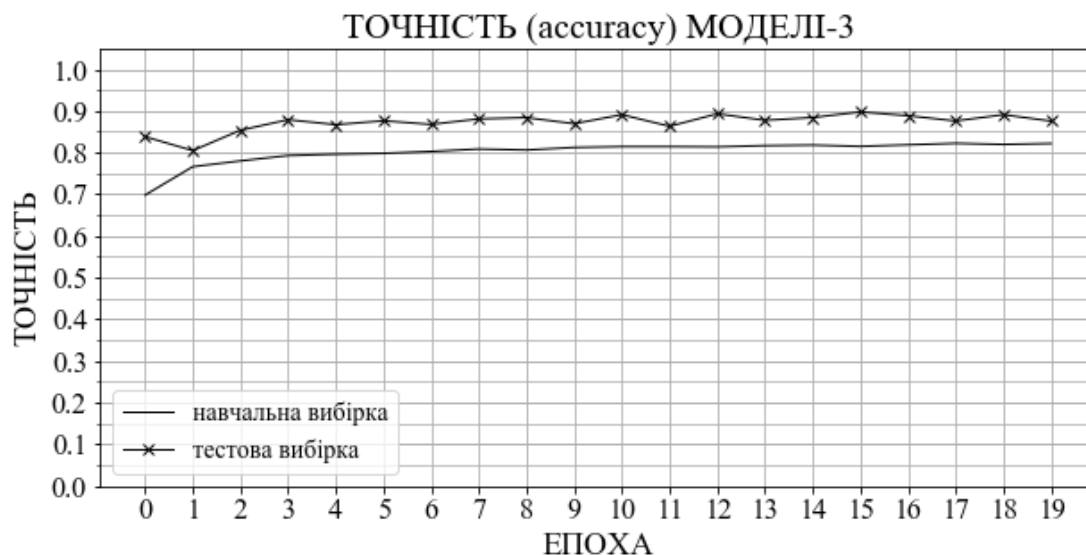


Рисунок 3.9 Точність Моделі-3

Дана модель навчилася за 237 хв 6 сек. Найкраща точність = 0.897594. Час зменшився за рахунок вдвічі більшого значення ОЗП. І знову, так як така точність не задовільнила бажаний результат, було вирішено запустити ще раз модель, але з іншими налаштуваннями самого типу навчання мережі на цьому ж комп'ютері.

3.4.4 Модель-4

Четверта модель мала тип навчання – finetuning (Додаток А). Кількість епох – 40, розмір партії зображень – 128. Результати на рисунках 3.10 – 3.11.

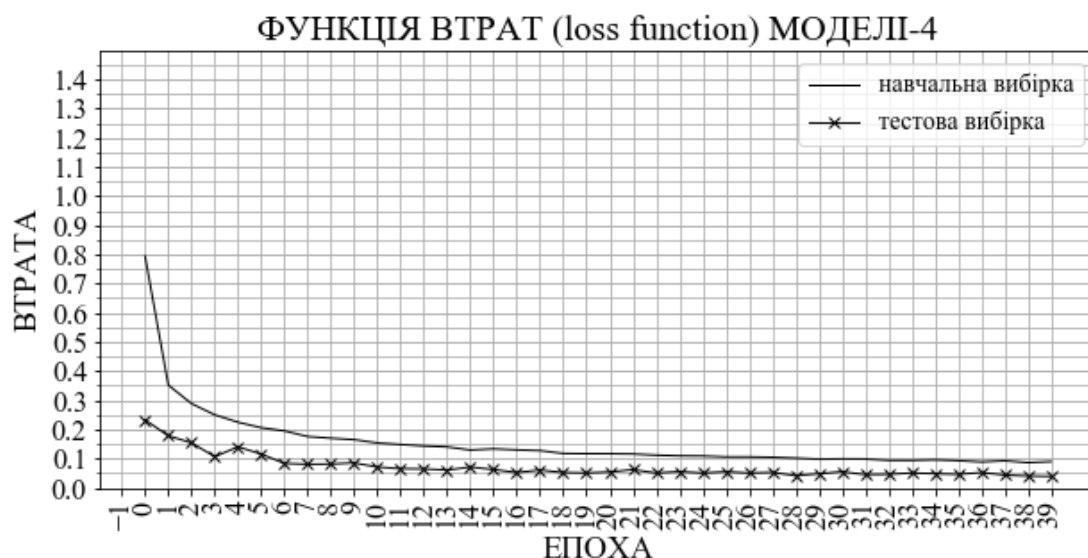


Рисунок 3.10 Функція втрат Моделі-4

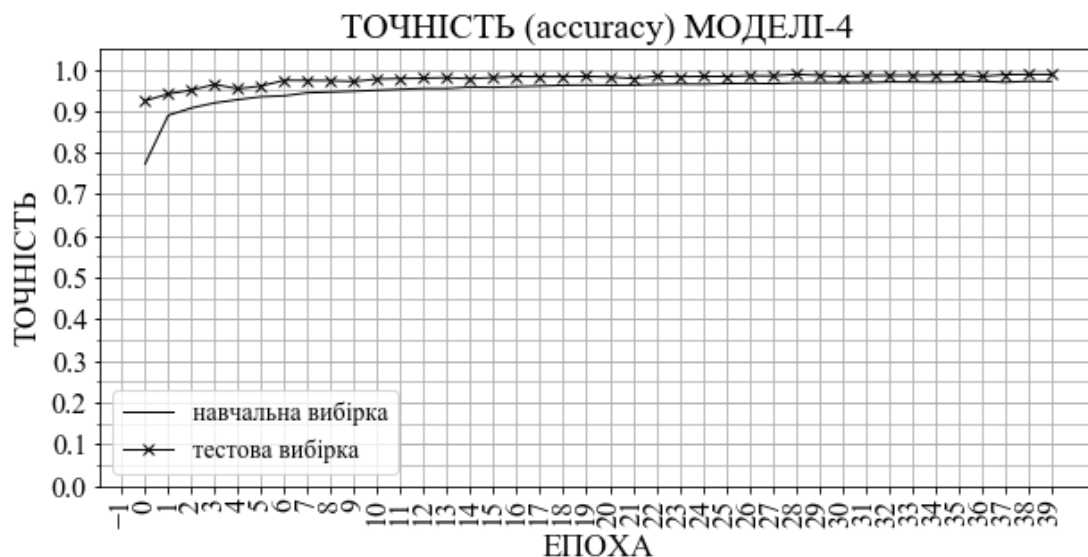


Рисунок 3.11 Точність Моделі-4

Дана модель показала точність = 0.987833, хоч і виконувалася 950 хв 47 сек.

Даний результат виявився не тільки найкращим серед всіх чотирьох випадків, а й ще дуже близько до 1, тому, було вирішено, що саме ця навчена модель буде використовуватися у інтелектуальному додаткові.

3.5 Аналіз коректності роботи застосунку

Тепер, розглянемо роботу інтелектуального додатку, що був створений за допомогою веб-складових HTML/CSS/JavaScript, серверної сторони Python Flask та бази даних MySQL. Запуск відбувається на локальному комп'ютері (localhost).

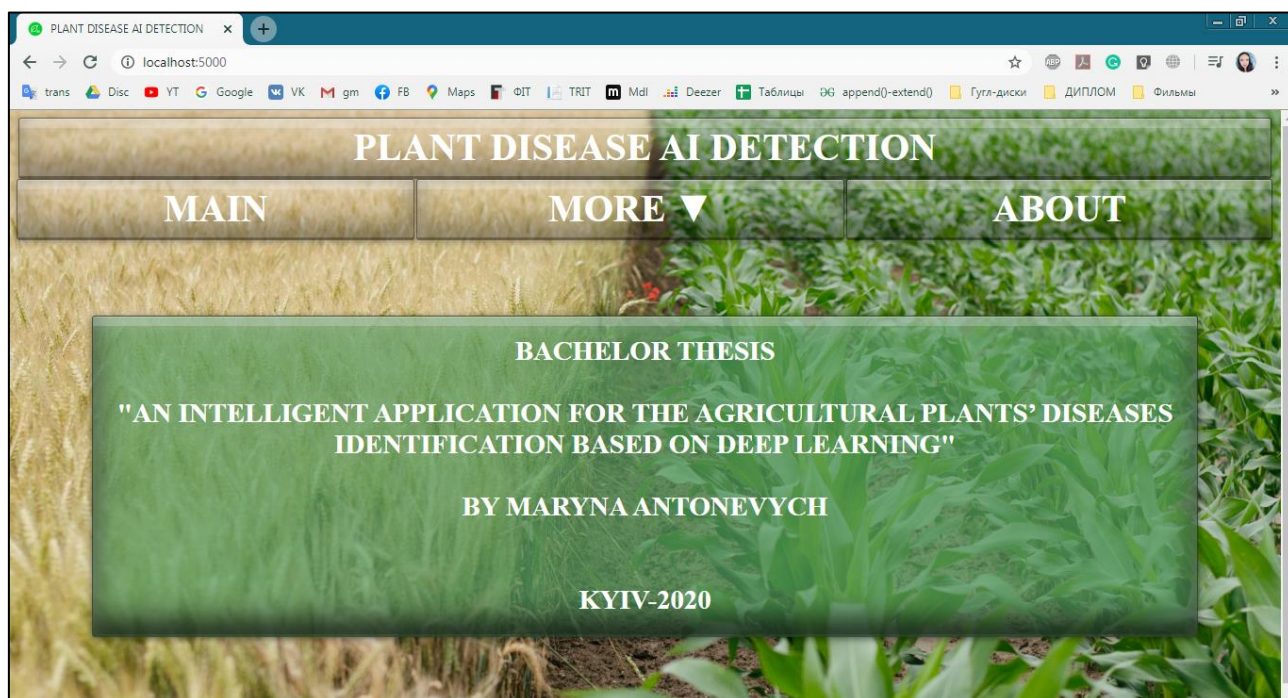


Рисунок 3.12 Сторінка PLANT DISEASE AI DETECTION

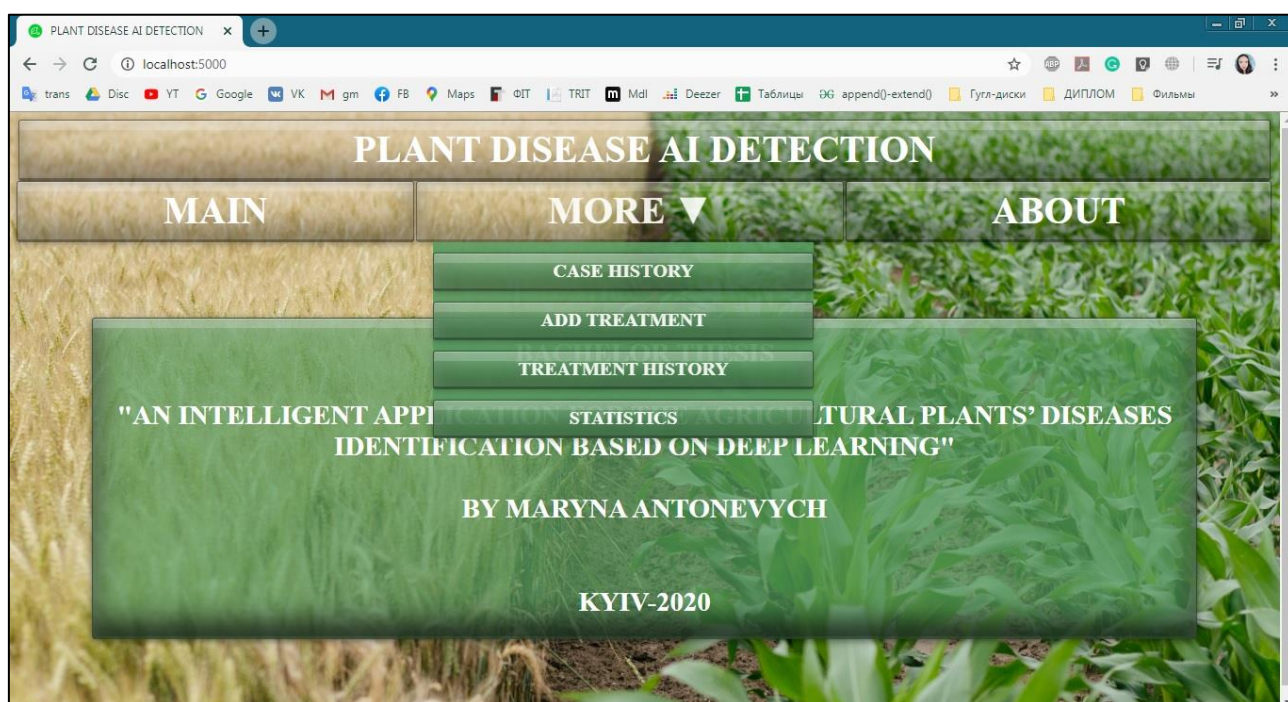


Рисунок 3.13 Випадаюче меню

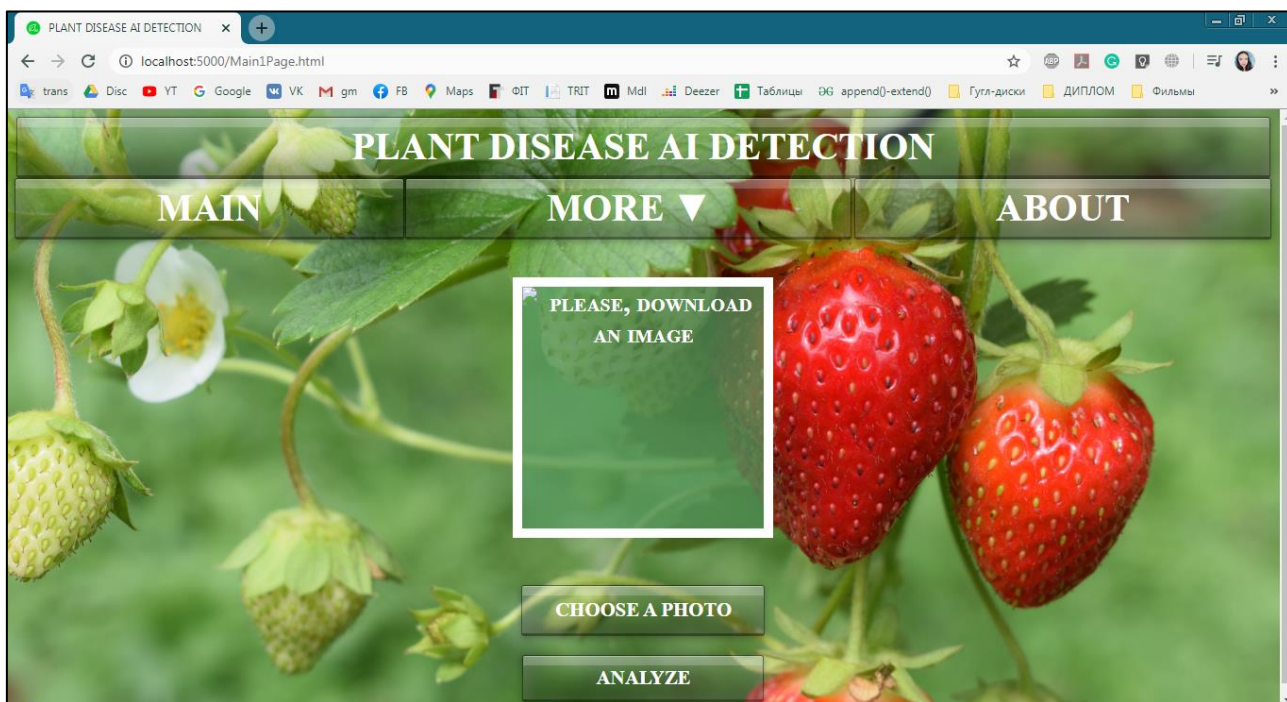


Рисунок 3.14 Сторінка MAIN-1

На рисунку 3.12 представлено знімок екрану сторінки PLANT DISEASE AI DETECTION. Це – титульна сторінка сайту. Також, як можна спостерігати, кнопка MORE має випадаюче меню, яке містить інші сторінки веб-додатку (рисунок 3.13).

Наступний рисунок 3.14 відображає головну сторінку MAIN, на якій відбувається вибір та завантаження фото для класифікації. Якщо користувач не вибрав зображення, а лише натиснув кнопку ANALYZE, то він отримає попередження про те, що зображення не було обране (рисунок 3.14), яке легко можна закрити, натиснувши на відповідну кнопку CLOSE на жовтому вікні.

Після того, як користувач обрав фото (рисунок 3.15), відбувається завантаження зображення до локального сховища та починається його обробка. Спочатку, отримане зображення перейменовується та зберігається під спеціальним іменем, яке залежить від того, коли було виконане завантаження фото, для кращої організації даних на комп'ютері. Після цього, зображення потрапляє на класифікатор, виконується його попередня обробка, зміна розміру (за потреби) та нормалізація. А вже потім і класифікація зображення.

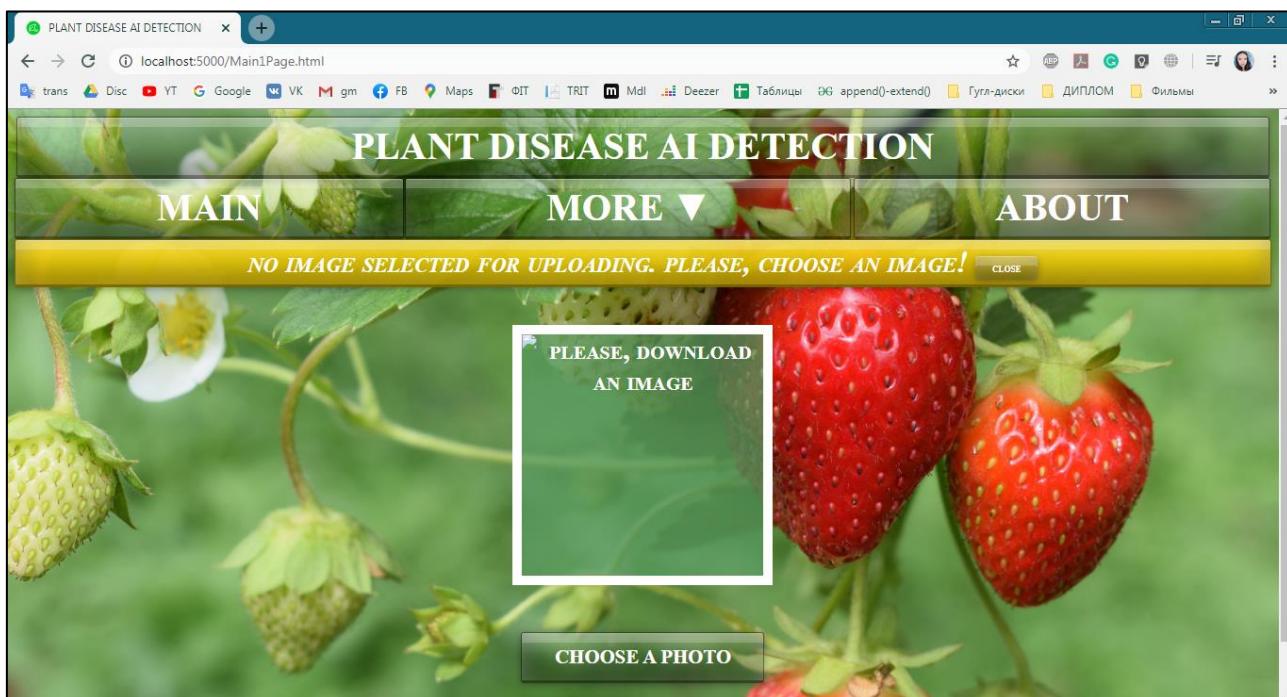


Рисунок 3.14 Попередження на сторінці MAIN-1

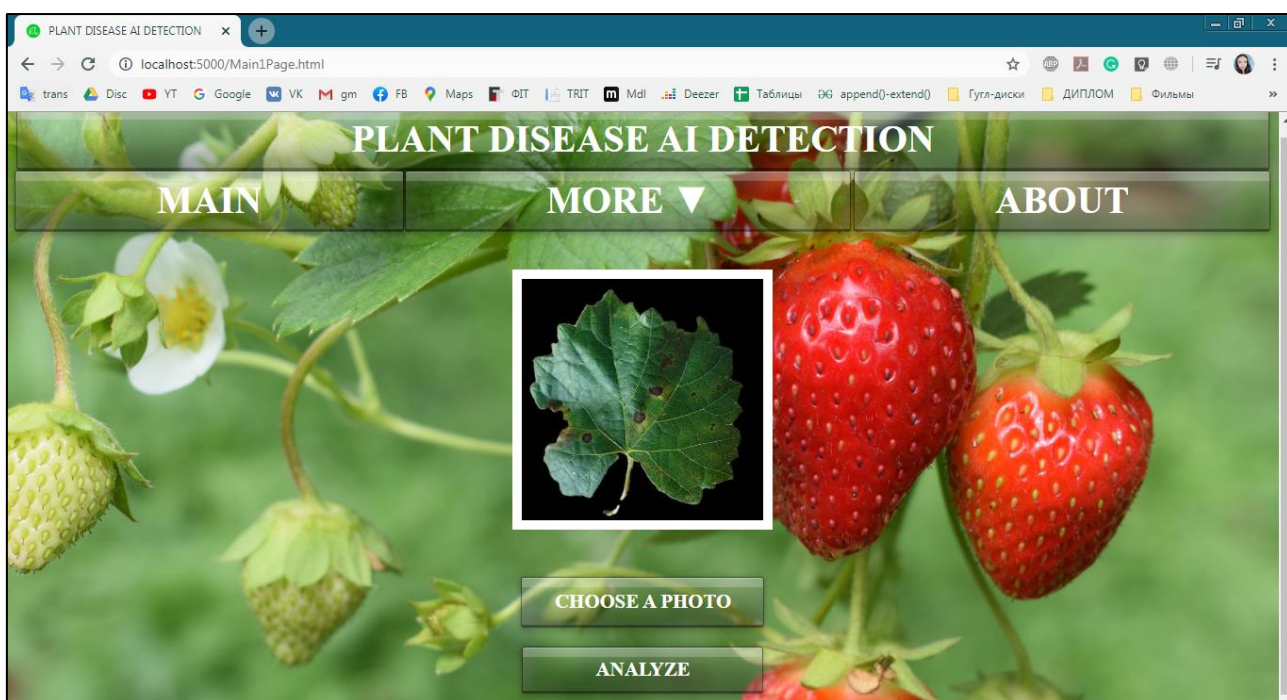


Рисунок 3.15 Попередній перегляд обраного зображення (хворий виноград)

Якщо дана рослина виявилася такою, що має хворобу, яку розпізнав нейромережний класифікатор, то на сторінці MAIN-2 з'явиться вся необхідна інформація про хворобу (випадок MAIN-21). Тут розміщено клас та ймовірність

належності до класу, збудник хвороби та можливі препарати, якими можна лікувати уражену рослину, про які детальніше можна почитати, натиснувши кнопку CLICK TO READ MORE (рисунок 3.16).

PLANT DISEASE AI DETECTION

MAIN MORE ▼ ABOUT

CLASSIFICATION RESULT with 99.997% it is a/an Grape Black Rot

PATHOGEN Guignardia bidwellii

DISEASE DESCRIPTION Grape black rot originated in eastern North America, but now occurs in portions of Europe, South America, and Asia. Relatively small, brown circular lesions develop on infected leaves and within a few days tiny black spherical fruiting bodies (pycnidia) protrude from them. Elongated black lesions on the petiole may eventually girdle these organs, causing the affected leaves to wilt. Shoot infection results in large black elliptical lesions. These lesions may contribute to breakage of shoots by wind, or in severe cases, may girdle and kill young shoots altogether. This fungus bides its time. Most plants show very little signs of infection until its too late. They will look very healthy until fruit sets. Even flowering will be normal. Infection of the fruit is the most serious phase of the disease and may result in substantial economic loss. Infected berries first appear light or chocolate brown; it will have a spot that looks very round, like the eye of a bird. That spot will get larger and infect more of the fruit bunch and more of the plant. This creates masses of black pycnidia developing on the surface. Finally, infected berries shrivel and turn into hard black raisin-like bodies that are called mummies.

POSSIBLE TREATMENT

DRUG NAME	PHARMA COMPANY NAME	DRUG DESCRIPTION	INSTRUCTION DESCRIPTION
SOVRAN®	BASF	► <i>click to read more</i>	▼ <i>click to read more</i> Apply at 240 g/ha at a 14-day interval. Sovran is excellent for control of black rot.
Pristine®	BASF	▼ <i>click to read more</i> Pristine fungicide is the leading fungicide used in	► <i>click to read more</i>

TRY AGAIN

Рисунок 3.16 Сторінка MAIN-2, випадок MAIN-21

Якщо ж було обране зображення здорової рослини (рисунок 3.17), то користувач просто отримає інформацію (ймовірність та клас) про те, що рослина здорова (MAIN-2, випадок MAIN-22). Очевидно, що така рослина не потребує лікування, тому інша інформація відсутня (рисунок 3.18).

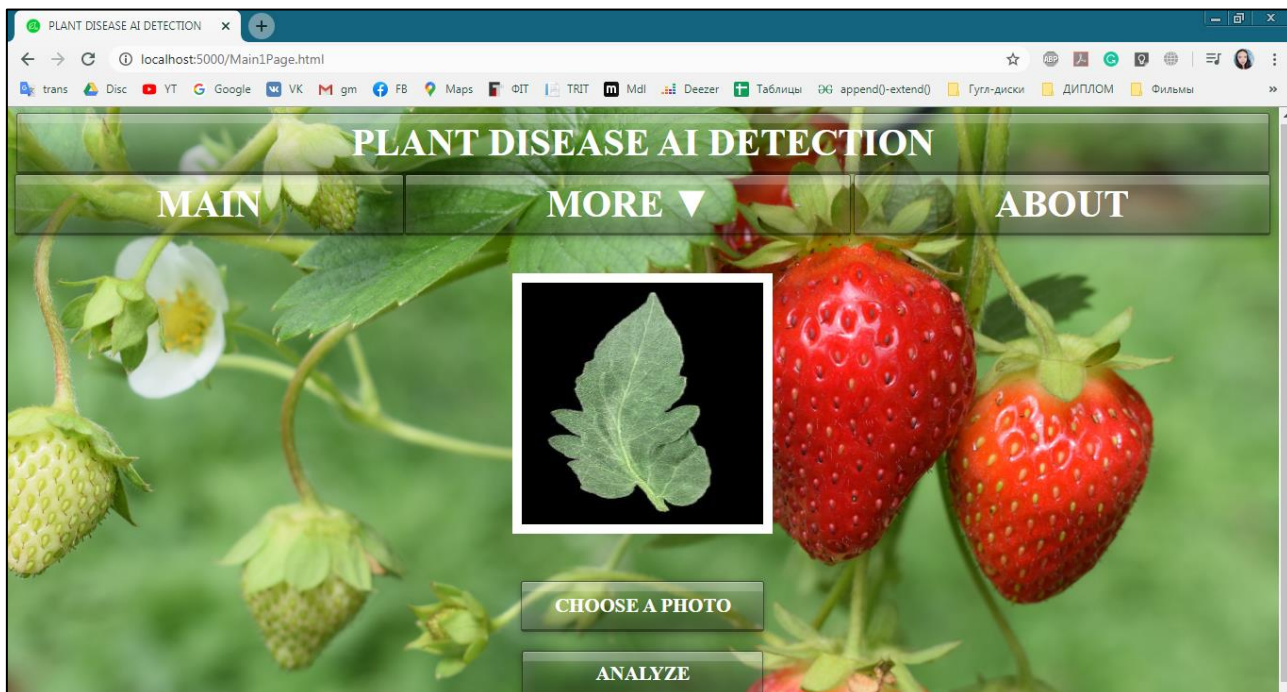


Рисунок 3.17 Попередній перегляд обраного зображення (здоровий помідор)

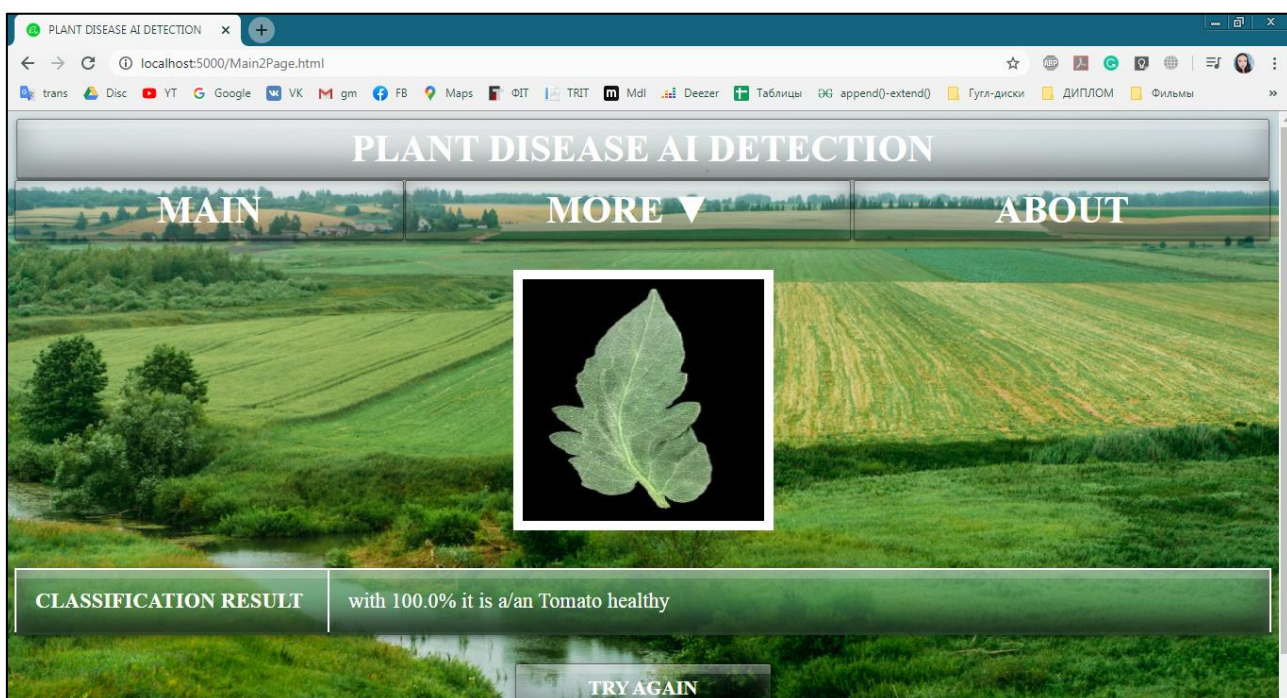
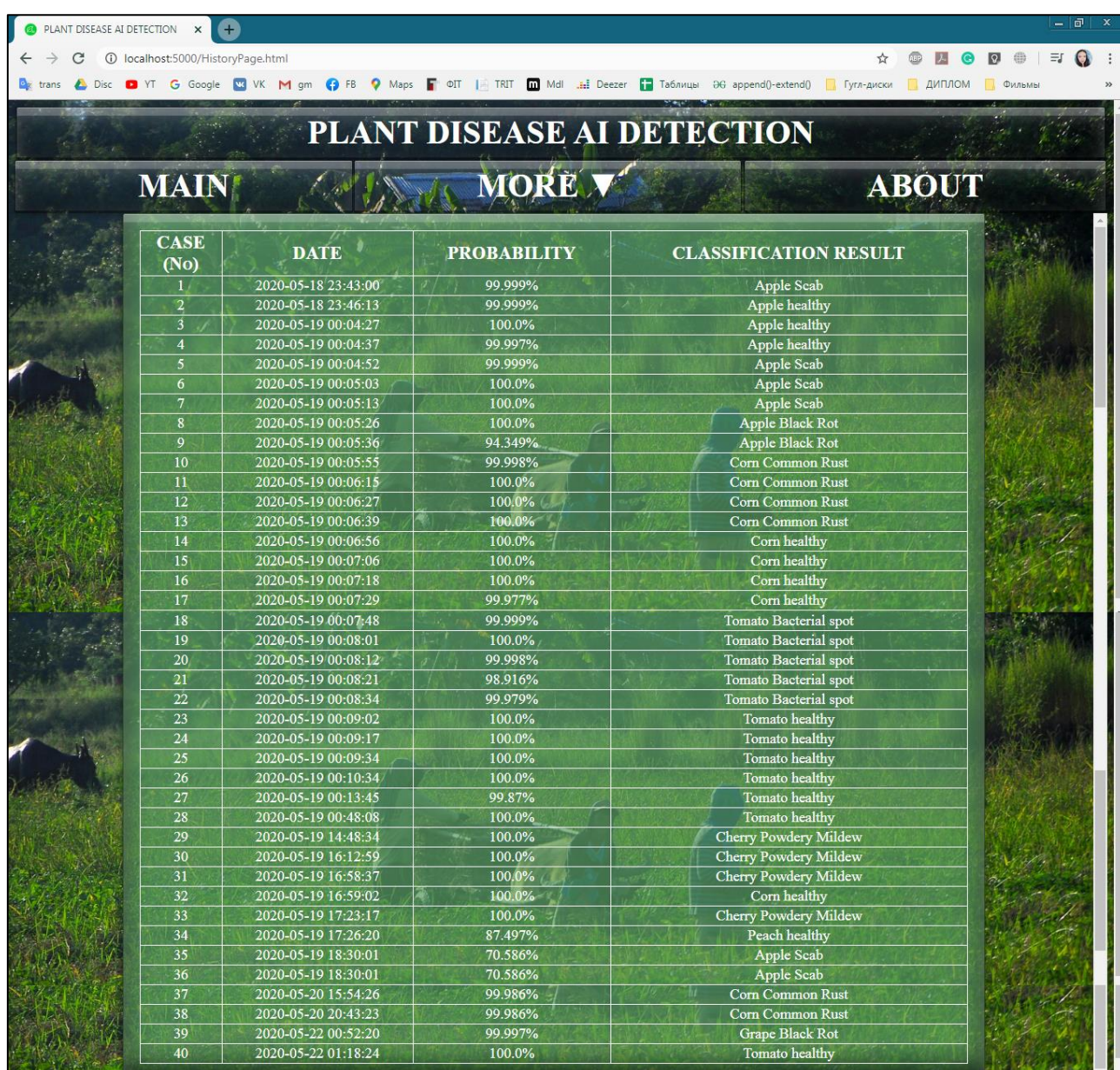


Рисунок 3.18 Сторінка MAIN-2, випадок MAIN-22

Для перегляду всієї історії класифікації рослин, можливо перейти на сторінку CASE HISTORY (рисунок 3.19). Всю таблицю можна прокручувати окремо, але для зручності був створений один рисунок із декількох знімків екрану. Можна спостерігати, що останні два записи у таблиці, це якраз саме ті дві рослини, що були щойно класифіковані і додані у таблицю CASE_OF_THE_DISEASE БД MySQL (рисунок 3.20). Вона містить всі необхідні дані, які потім користувач зможе додатково проаналізувати і зробити певні висновки.



CASE (No)	DATE	PROBABILITY	CLASSIFICATION RESULT
1	2020-05-18 23:43:00	99.999%	Apple Scab
2	2020-05-18 23:46:13	99.999%	Apple healthy
3	2020-05-19 00:04:27	100.0%	Apple healthy
4	2020-05-19 00:04:37	99.997%	Apple healthy
5	2020-05-19 00:04:52	99.999%	Apple Scab
6	2020-05-19 00:05:03	100.0%	Apple Scab
7	2020-05-19 00:05:13	100.0%	Apple Scab
8	2020-05-19 00:05:26	100.0%	Apple Black Rot
9	2020-05-19 00:05:36	94.349%	Apple Black Rot
10	2020-05-19 00:05:55	99.998%	Corn Common Rust
11	2020-05-19 00:06:15	100.0%	Corn Common Rust
12	2020-05-19 00:06:27	100.0%	Corn Common Rust
13	2020-05-19 00:06:39	100.0%	Corn Common Rust
14	2020-05-19 00:06:56	100.0%	Corn healthy
15	2020-05-19 00:07:06	100.0%	Corn healthy
16	2020-05-19 00:07:18	100.0%	Corn healthy
17	2020-05-19 00:07:29	99.977%	Corn healthy
18	2020-05-19 00:07:48	99.999%	Tomato Bacterial spot
19	2020-05-19 00:08:01	100.0%	Tomato Bacterial spot
20	2020-05-19 00:08:12	99.998%	Tomato Bacterial spot
21	2020-05-19 00:08:21	98.916%	Tomato Bacterial spot
22	2020-05-19 00:08:34	99.979%	Tomato Bacterial spot
23	2020-05-19 00:09:02	100.0%	Tomato healthy
24	2020-05-19 00:09:17	100.0%	Tomato healthy
25	2020-05-19 00:09:34	100.0%	Tomato healthy
26	2020-05-19 00:10:34	100.0%	Tomato healthy
27	2020-05-19 00:13:45	99.87%	Tomato healthy
28	2020-05-19 00:48:08	100.0%	Tomato healthy
29	2020-05-19 14:48:34	100.0%	Cherry Powdery Mildew
30	2020-05-19 16:12:59	100.0%	Cherry Powdery Mildew
31	2020-05-19 16:58:37	100.0%	Cherry Powdery Mildew
32	2020-05-19 16:59:02	100.0%	Corn healthy
33	2020-05-19 17:23:17	100.0%	Cherry Powdery Mildew
34	2020-05-19 17:26:20	87.497%	Peach healthy
35	2020-05-19 18:30:01	70.586%	Apple Scab
36	2020-05-19 18:30:01	70.586%	Apple Scab
37	2020-05-20 15:54:26	99.986%	Corn Common Rust
38	2020-05-20 20:43:23	99.986%	Corn Common Rust
39	2020-05-22 00:52:20	99.997%	Grape Black Rot
40	2020-05-22 01:18:24	100.0%	Tomato healthy

Рисунок 3.19 Сторінка CASE HISTORY

case_id	case_date_time	probability	CLASS_class_id
31	2020-05-19 16:58:37	100	5
32	2020-05-19 16:59:02	100	10
33	2020-05-19 17:23:17	100	5
34	2020-05-19 17:26:20	87.497	17
35	2020-05-19 18:30:01	70.586	0
36	2020-05-19 18:30:01	70.586	0
37	2020-05-20 15:54:26	99.986	8
38	2020-05-20 20:43:23	99.986	8
39	2020-05-22 00:52:20	99.997	11
40	2020-05-22 01:18:24	100	37
NULL	NULL	NULL	NULL

Рисунок 3.20 Таблиця із новими записами на MySQL Server

Для того, щоб додати запис про лікування певної рослини, необхідно перейти на сторінку ADD TREATMENT (рисунки 3.21 – 3.22), обрати необхідний випадок захворювання та препарат, яким було здійснено лікування. Також можливо ввести додаткову інформацію про перебіг лікування.

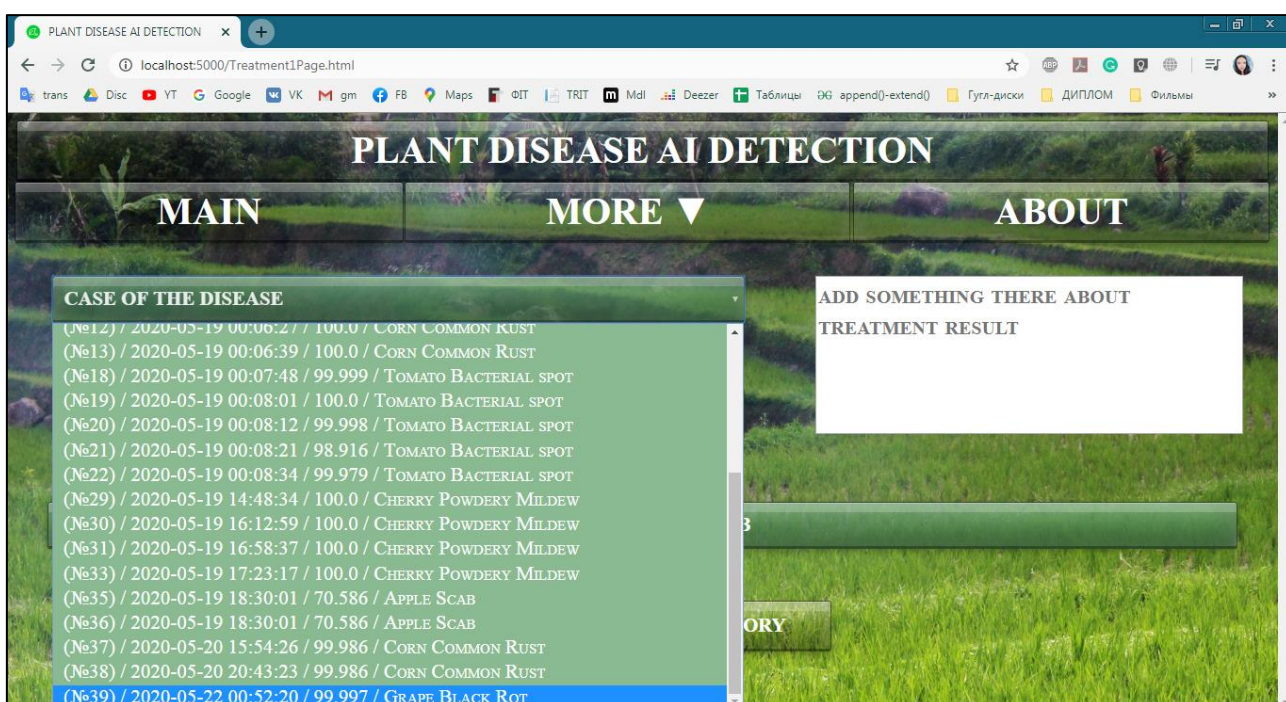


Рисунок 3.21 Вибір випадку захворювання

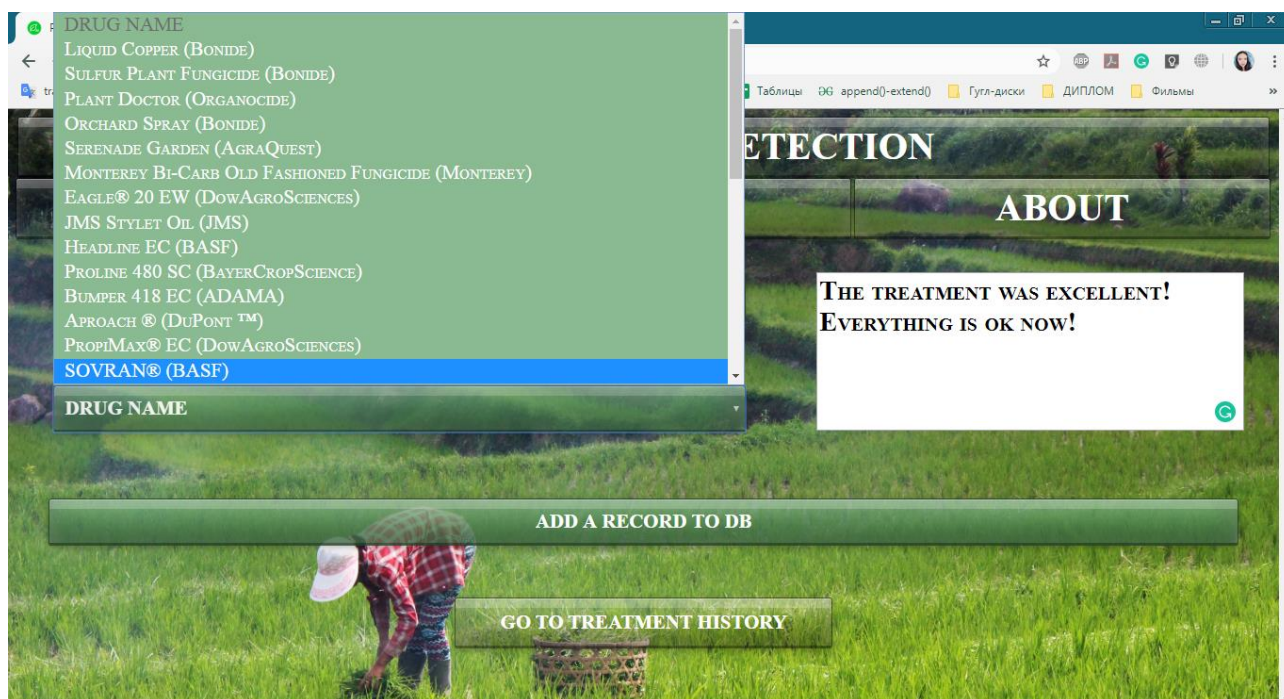


Рисунок 3.22 Вибір препарату для лікування

Після того, як всі необхідні поля обрані та заповнені (рисунок 3.23), інформація записується до таблиці БД MySQL TREATMENT_HISTORY (рисунок 3.24). Також, користувач бачить повідомлення про створення запису.

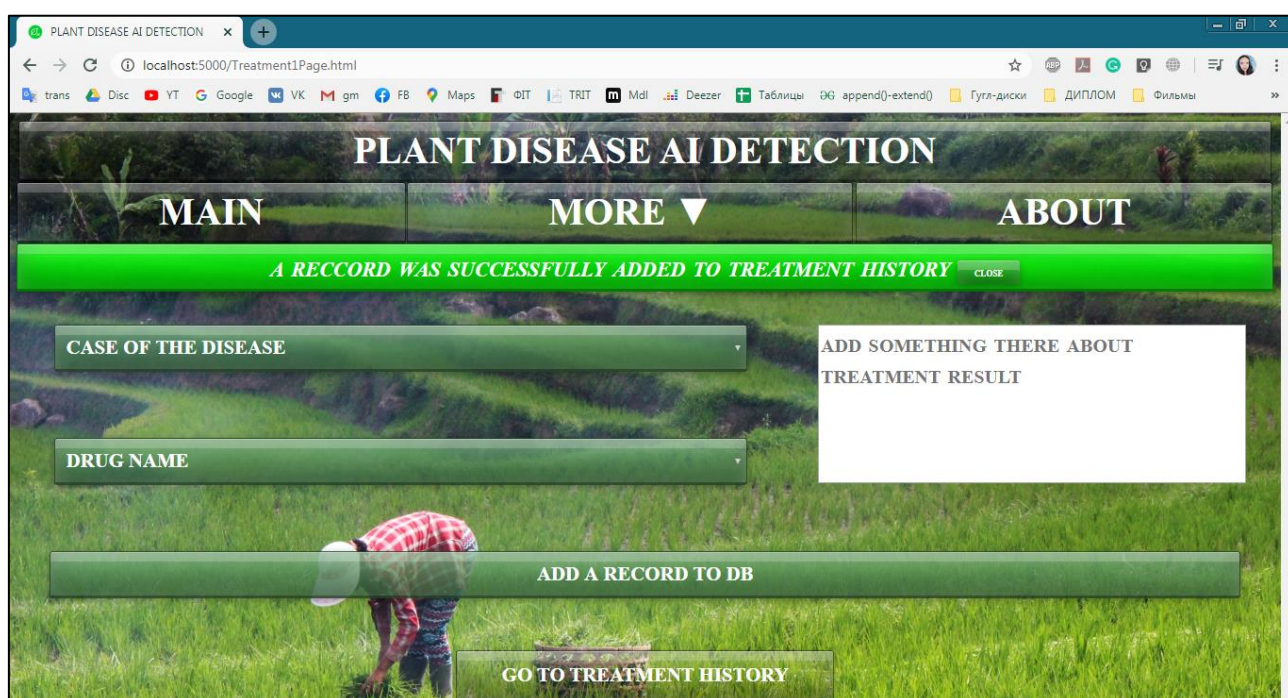
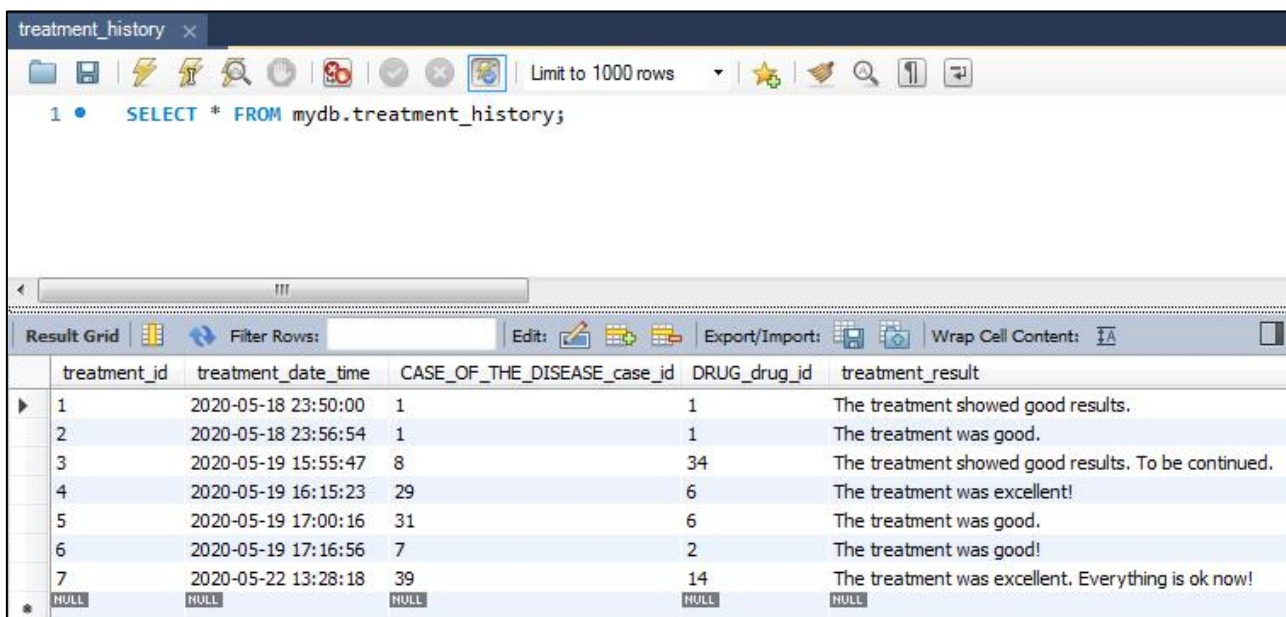


Рисунок 3.23 Запис успішно додано до БД



treatment_id	treatment_date_time	CASE_OF_THE_DISEASE_case_id	DRUG_drug_id	treatment_result
1	2020-05-18 23:50:00	1	1	The treatment showed good results.
2	2020-05-18 23:56:54	1	1	The treatment was good.
3	2020-05-19 15:55:47	8	34	The treatment showed good results. To be continued.
4	2020-05-19 16:15:23	29	6	The treatment was excellent!
5	2020-05-19 17:00:16	31	6	The treatment was good.
6	2020-05-19 17:16:56	7	2	The treatment was good!
7	2020-05-22 13:28:18	39	14	The treatment was excellent. Everything is ok now!
NULL	NULL	NULL	NULL	NULL

Рисунок 3.24 Таблиця із новими записами на MySQL Server

Якщо, наприклад, був обраний невірний препарат, який відповідає даному захворюванню (рисунок 3.25), то користувач отримає повідомлення на жовтому фоні про те, що необхідно повторити спробу, адже відбулася помилка (рисунок 3.26). Тому, такий запис не створиться у таблиці БД (рисунок 3.27).

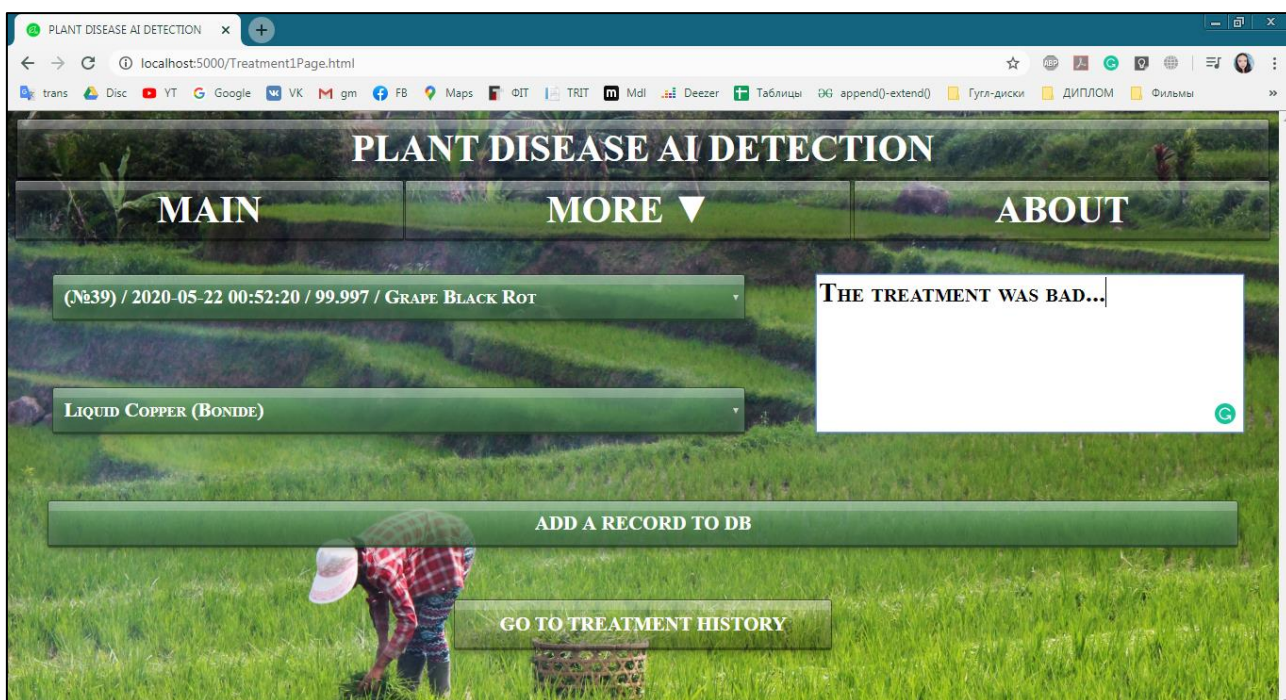


Рисунок 3.25 Невірне поєднання препарату та випадку захворювання

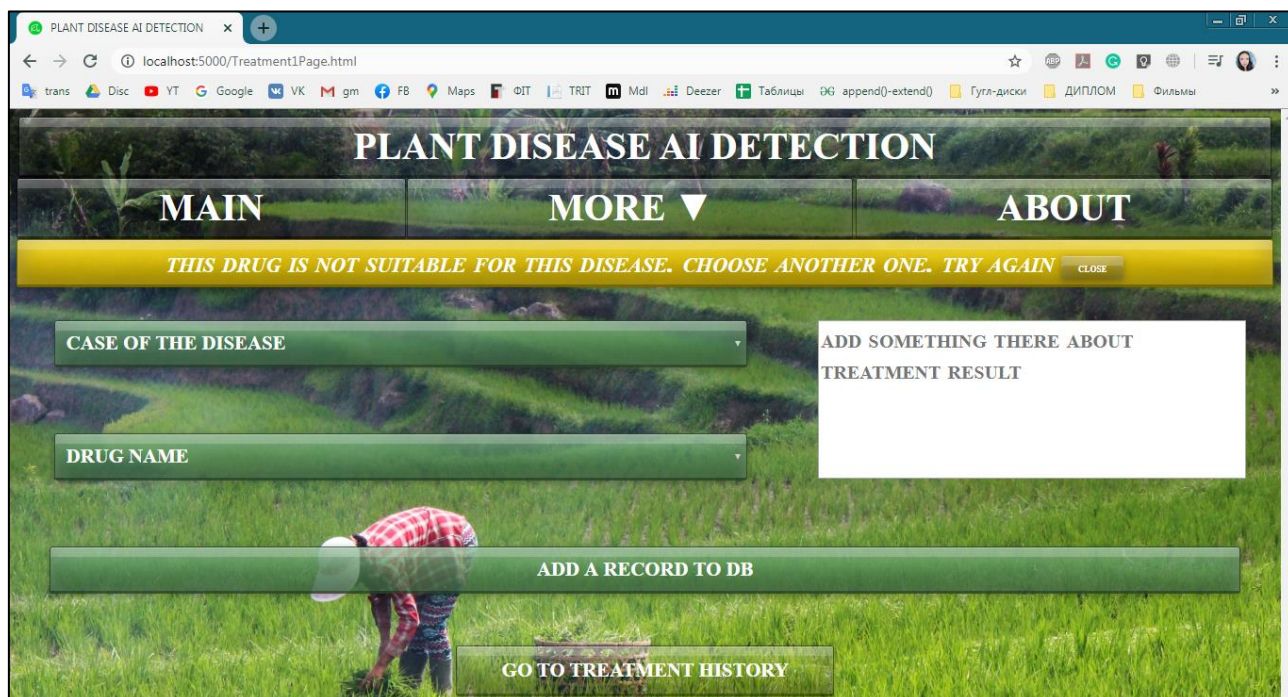


Рисунок 3.26 Жовте попередження про помилку

	treatment_id	treatment_date_time	CASE_OF_THE_DISEASE_case_id	DRUG_drug_id	treatment_result
1	1	2020-05-18 23:50:00	1	1	The treatment showed good results.
2	2	2020-05-18 23:56:54	1	1	The treatment was good.
3	3	2020-05-19 15:55:47	8	34	The treatment showed good results. To be continued.
4	4	2020-05-19 16:15:23	29	6	The treatment was excellent!
5	5	2020-05-19 17:00:16	31	6	The treatment was good.
6	6	2020-05-19 17:16:56	7	2	The treatment was good!
7	7	2020-05-22 13:28:18	39	14	The treatment was excellent. Everything is ok now!
*	NULL	NULL	NULL	NULL	NULL

Рисунок 3.27 Таблиця без нових записів на MySQL Server

Якщо ж користувач не заповнив поля, то він також отримає відповідне попередження (рисунок 3.28) і у нього буде нова спроба ввести всі необхідні дані з самого початку.

Очевидно, що запис у таблиці БД MySQL TREATMENT_HISTORY не створиться (рисунок 3.29).

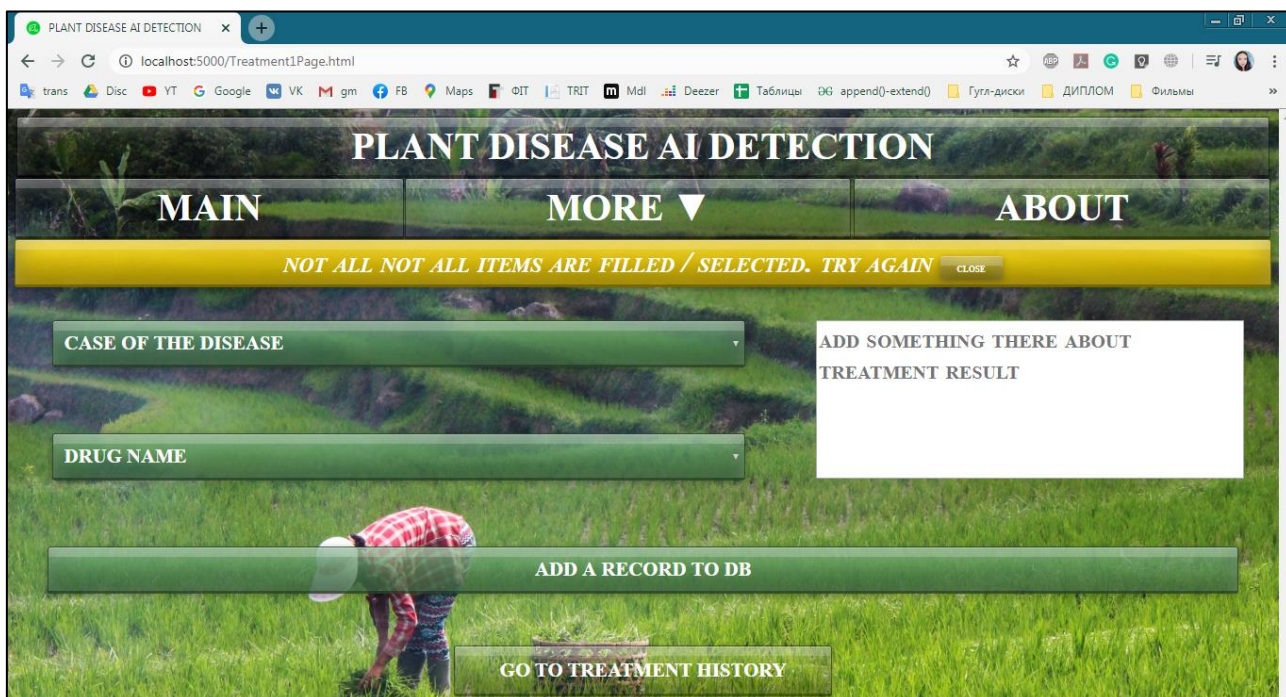


Рисунок 3.28 Попередження про порожні поля

	treatment_id	treatment_date_time	CASE_OF_THE_DISEASE_case_id	DRUG_drug_id	treatment_result
1	1	2020-05-18 23:50:00	1	1	The treatment showed good results.
2	2	2020-05-18 23:56:54	1	1	The treatment was good.
3	3	2020-05-19 15:55:47	8	34	The treatment showed good results. To be continued.
4	4	2020-05-19 16:15:23	29	6	The treatment was excellent!
5	5	2020-05-19 17:00:16	31	6	The treatment was good.
6	6	2020-05-19 17:16:56	7	2	The treatment was good!
7	7	2020-05-22 13:28:18	39	14	The treatment was excellent. Everything is ok now!
*	NULL	NULL	NULL	NULL	NULL

Рисунок 3.29 Таблиця без нових записів на MySQL Server

Для того, щоб переглянути всю історію лікувань с/г рослин, можливо перейти на сторінку TREATMENT HISTORY (рисунок 3.30). Там відображається вся необхідна інформація про лікування, що береться із таблиці БД MySQL TREATMENT_HISTORY. Останнім можна спостерігати запис, який

щойно вище був створений. Всю таблицю можна прокручувати окремо, але для зручності був створений один рисунок із декількох знімків екрану.



TREATMENT (No)	TREATMENT RECORD DATE	CASE (No)	CASE DATE	PROBABILITY	CLASSIFICATION RESULT	DRUG NAME	PHARMA COMPANY NAME	TREATMENT RESULT
1	2020-05-18 23:50:00	1	2020-05-18 23:43:00	99.999	Apple Scab	Liquid Copper	Bonide	The treatment showed good results.
2	2020-05-18 23:56:54	1	2020-05-18 23:43:00	99.999	Apple Scab	Liquid Copper	Bonide	The treatment was good.
3	2020-05-19 15:55:47	8	2020-05-19 00:05:26	100.0	Apple Black Rot	TOPSIN® M 70WP	Cerexagri-Nisso LLC	The treatment showed good results. To be continued.
4	2020-05-19 16:15:23	29	2020-05-19 14:48:34	100.0	Cherry Powdery Mildew	Monterey Bi-Carb Old Fashioned Fungicide	Monterey	The treatment was excellent!
5	2020-05-19 17:00:16	31	2020-05-19 16:58:37	100.0	Cherry Powdery Mildew	Monterey Bi-Carb Old Fashioned Fungicide	Monterey	The treatment was good.
6	2020-05-19 17:16:56	7	2020-05-19 00:05:13	100.0	Apple Scab	Sulfur Plant Fungicide	Bonide	The treatment was good!
7	2020-05-22 13:28:18	39	2020-05-22 00:52:20	99.997	Grape Black Rot	SOVRAN®	BASF	The treatment was excellent. Everything is ok now!

Рисунок 3.30 сторінка TREATMENT HISTORY

Наступна сторінка, яка буде розглянута, це STATISTICS (рисунок 3.31). Вона відображає статистичні дані випадків захворювання. Побудована гістограма частот для кожного класу.

Тобто, скільки було розпізнано різноманітних класів за всю історію випадків (сюди включені як здорові, так і хворі рослини). По вісі ОХ відображається номер класу, по вісі ОУ – значення частот.

Знизу можна ознайомитися за допомогою списку, що прокручується, відповідність номеру класу, що був позначений на вісі абсцис до назви класу, щоб користувачу було краще орієнтуватися у даних.

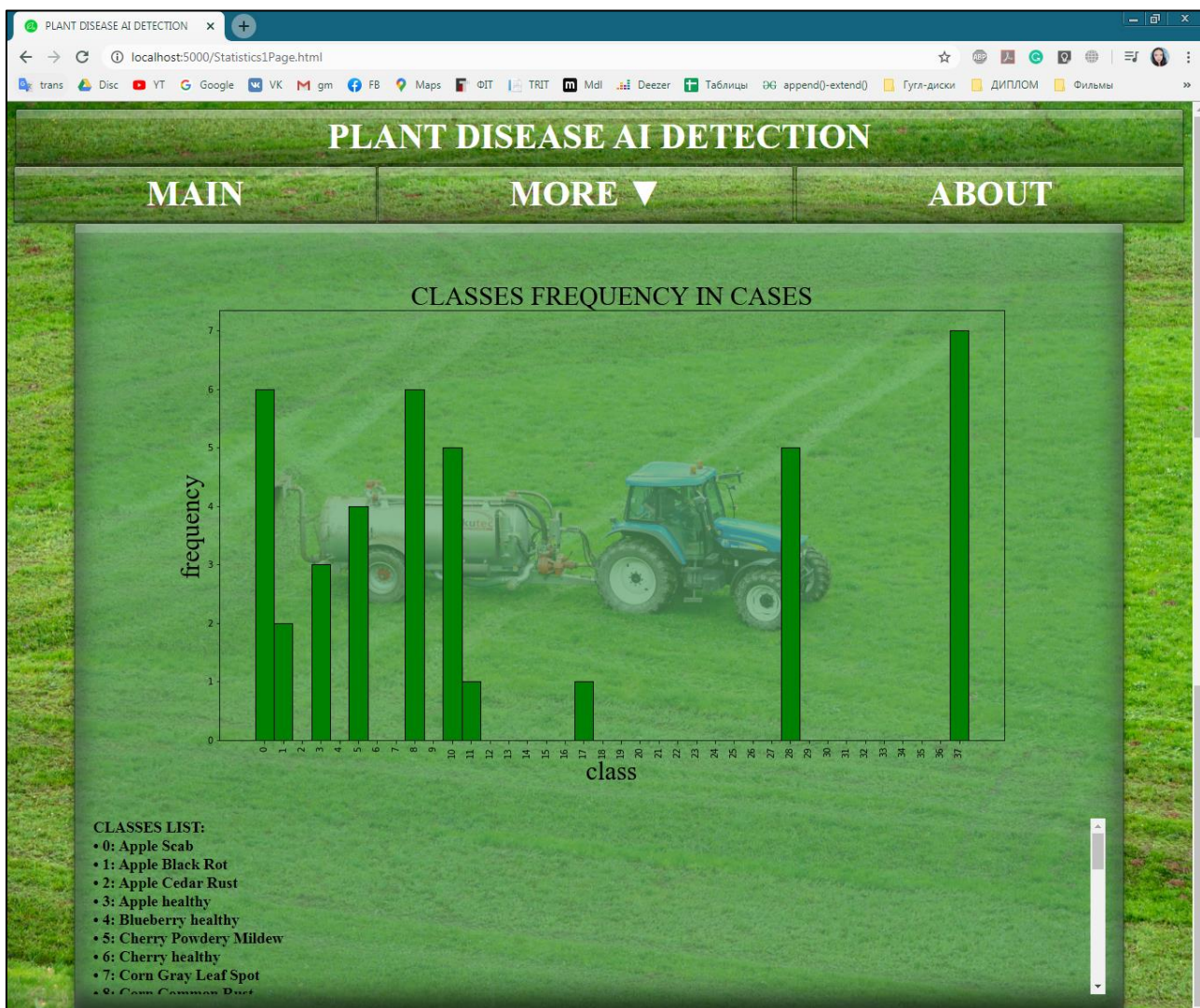


Рисунок 3.31 Сторінка STATISTICS

На наступному рисунку (3.32) представлена сторінка ABOUT. Вона містить всю необхідну інформацію про інтелектуальний додаток, перелік с/г культур, що можна перевіряти на захворювання. Також, існує можливість зв'язатися із розробниками (кнопка CONTACT VIA E-MAIL, рисунок 3.33), перейти на сторінку факультету інформаційних технологій (кнопка UNIVERSITY FACULTY, рисунок 3.34). Якщо додаток буде використовуватися великим колом людей, необхідно буде мати Політику конфіденційності (кнопка PRIVACY POLICY, рисунок 3.35), щоб користувачі могли ознайомитися із правилами збереження та використання їх особистих даних тощо.

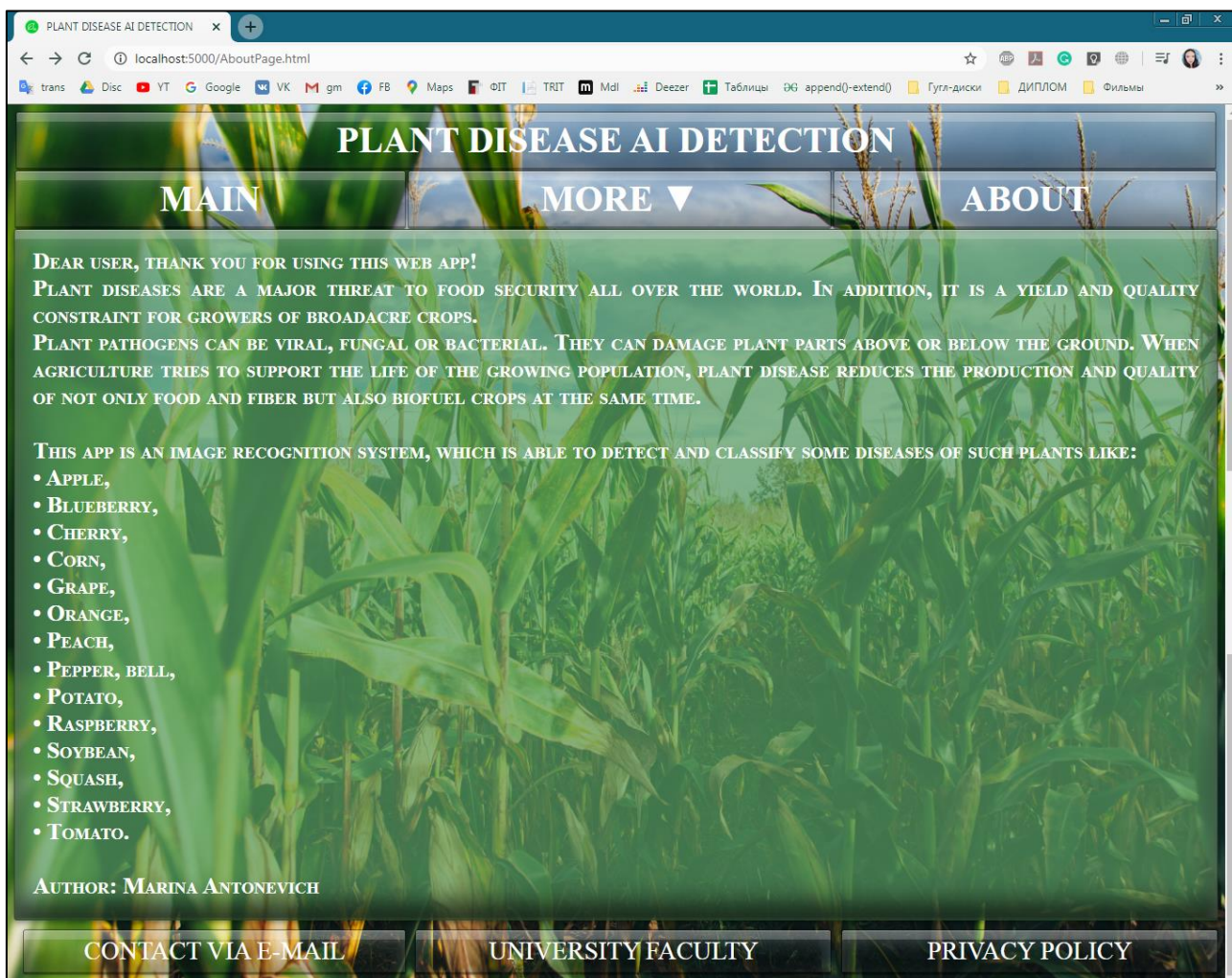


Рисунок 3.32 Сторінка ABOUT

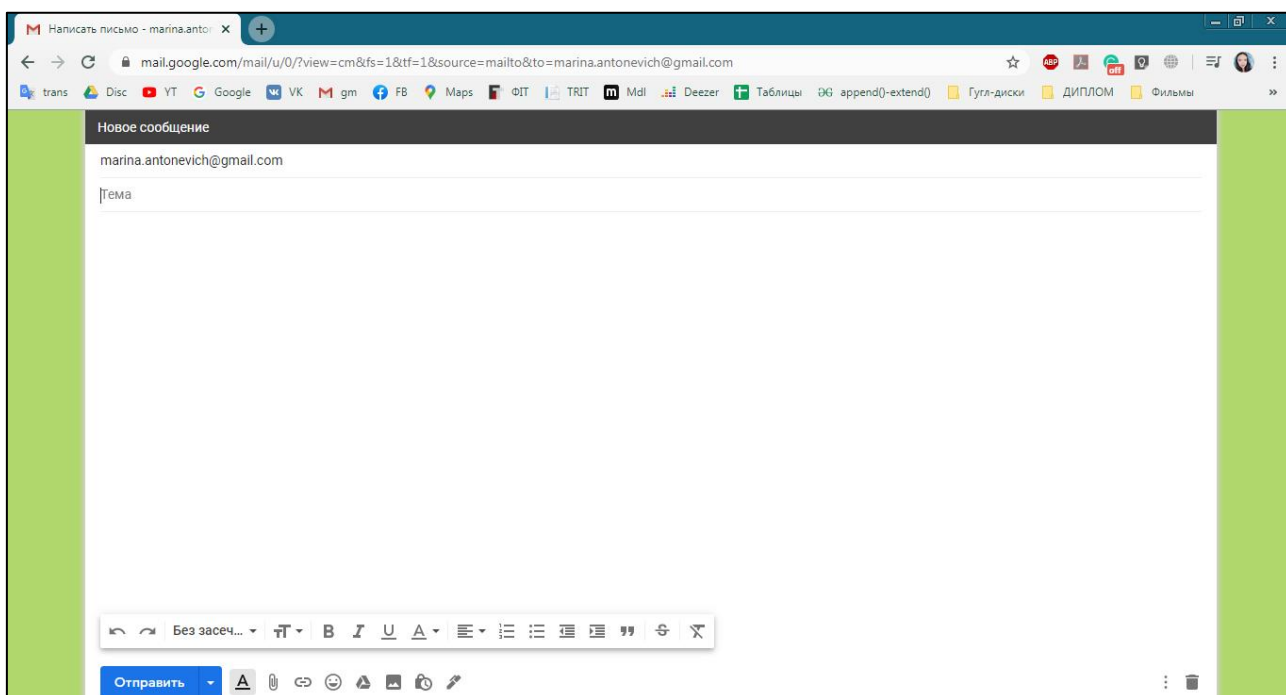


Рисунок 3.33 CONTACT VIA E-MAIL

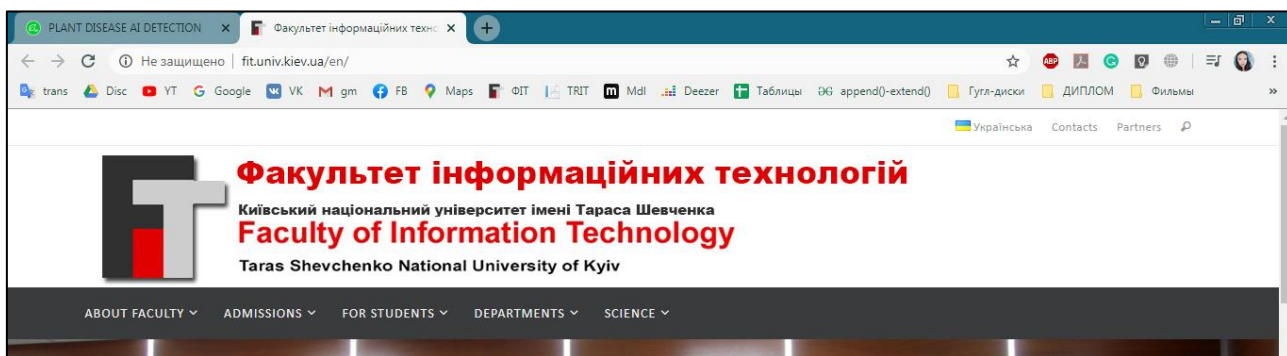


Рисунок 3.34 UNIVERSITY FACULTY

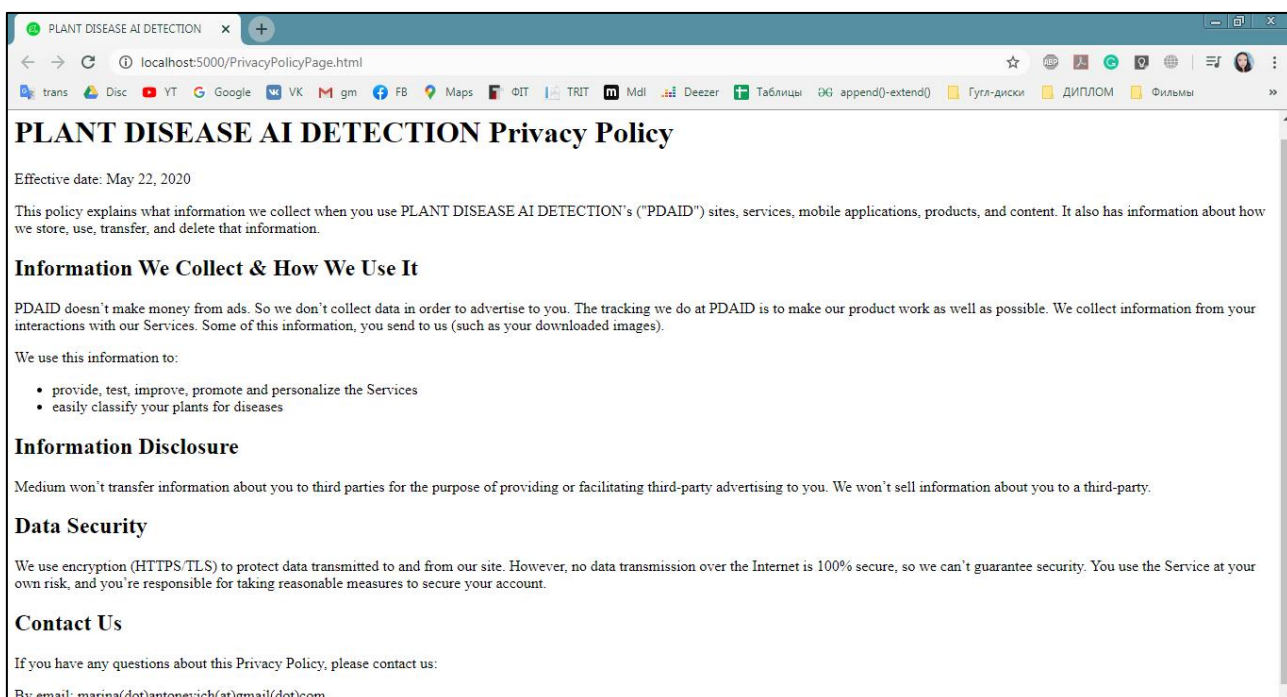


Рисунок 3.35 PRIVACY POLICY

ВИСНОВКИ

Отже, у першому розділі було розглянуто основні проблеми сільського господарства, а саме, хвороби рослин. Проаналізовано загальну статистику сільського господарства. Досліджено, які є захворювання рослин. Разом з тим, розглянуто, як машинне навчання використовується у агробізнесі. Проаналізовано методи розпізнавання образів існують. Також, зроблено огляд основних попередніх робіт, що досліджували хвороби рослин та їх класифікацію за допомогою методів штучного інтелекту. Більше того, була поставлена основна задача даного проекту, розглянуто системні, функціональні та нефункціональні вимоги. Проаналізовано, що буде на вході та виході системи.

У другому розділі було спроектовано майбутню систему інтелектуального додатку із використанням DFD діаграм – контекстну та першого рівня. Також, було побудовано структурну діаграму додатку. Більше того, було розглянуто згорткові мережі та їх основні типи, обрано, що для розробки інтелектуального додатку буде використовуватися згорткова нейронна мережа, що має архітектуру AlexNet. Для побудови БД було здійснено вибір виконувати у СУБД MySQL. Було побудовано фізичну модель БД, виділено основні зв'язки у схемі, описано атрибути таблиць БД. Потім, побудовано макети сторінок додатку та зроблений їх детальний опис. Більше того, було побудовано діаграму переходів із сторінки на сторінку, щоб користувач мав змогу краще розібратися в архітектурі інтелектуальної системи.

У третьому розділі було проаналізовано та обрано, які бібліотеки можливо буде використовувати для створення класифікатора та інтелектуального додатку. Крім того, було навчено згорткову нейронну мережу AlexNet шляхом тестування різноманітних налаштувань параметрів моделі та комп'ютерів. Побудовано функції втрат та точності для всіх моделей, як для навчальної, так і для тестової вибірок. Після цього був створений інтелектуальний додаток за допомогою веб-складових HTML/CSS/JavaScript, серверної сторони Python Flask та бази даних MySQL, із використанням MySQL Workbench та MySQL Server.

ВИКОРИСТАНІ ДЖЕРЕЛА

1. Antonevych M. M., Domanetska I. M. Intelligent application for the agricultural plants' diseases identification based on deep learning / M. M. Antonevych, I. M. Domanetska. // Інформаційні технології та взаємодії (IT&I – 2019). – 2019. – С. 183–184.
2. Сільське господарство України [Електронний ресурс] // Матеріал з Вікіпедії — вільної енциклопедії. – Режим доступу до ресурсу: https://uk.wikipedia.org/wiki/Сільське_господарство_України.
3. Pests and Pathogens Place Global Burden on Major Food Crops [Електронний ресурс]. – 2019. – Режим доступу до ресурсу: <https://californiaagtoday.com/pests-diseases-cause-worldwide-damage-crops/>.
4. Crop diseases [Електронний ресурс] – Режим доступу до ресурсу: <https://www.agric.wa.gov.au/pests-weeds-diseases/diseases/crop-diseases>.
5. «АгроГід». Рейтинг шкодочинності хвороб зернових культур. Прогноз від «АгроГід» на 2019 рік [Електронний ресурс] / «АгроГід». – 2019. – Режим доступу до ресурсу: <https://www.syngenta.ua/news/zernovi/rejting-shkodochnosti-hvorob-zernovih-kultur-prognoz-vid-agrogid-na-2019-rik>.
6. Классификация и диагностика болезней растений [Електронний ресурс] // Атлас болезней полевых культур – Режим доступу до ресурсу: https://agromage.com/stat_id.php?id=13.
7. Как машинное обучение меняет сельское хозяйство [Електронний ресурс] // AGGEEK. – 2018. – Режим доступу до ресурсу: <https://aggeek.net/ru-blog/kak-mashinnoe-obuchenie-menyaet-selskoe-hozyajstvo>.
8. BOLT-LMM v2.3.4 User Manual [Електронний ресурс] – Режим доступу до ресурсу: <https://data.broadinstitute.org/alkesgroup/BOLT-LMM/>.
9. See & Spray [Електронний ресурс] // Blue River – Режим доступу до ресурсу: <https://www.youtube.com/watch?v=1I6mSQK4aPI>.
10. BitRefine Group. Plant disease detection [Електронний ресурс] / BitRefine Group – Режим доступу до ресурсу:

<https://bitrefine.group/industries/precision-agriculture/88-industries/agriculture-food/agriculture-solutions/184-plant-disease-detection>.

11. Теорія розпізнавання образів [Електронний ресурс] – Режим доступу до ресурсу: https://uk.wikipedia.org/wiki/Теорія_розпізнавання_образів.
12. Sharada P. Mohanty. Using Deep Learning for Image-Based Plant Disease Detection [Електронний ресурс] / Sharada P. Mohanty, David P. Hughes, Marcel Salathé. – 2016. – Режим доступу до ресурсу: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5032846/>.
13. Krizhevsky A., Sutskever I., Hinton G. E. (2012). Imagenet classification with deep convolutional neural networks, in *Advances in Neural Information Processing Systems*, eds Pereira F., Burges C. J. C., Bottou L., Weinberger K. Q., editors. (Curran Associates, Inc.), 1097–1105.
14. Szegedy C., Liu W., Jia Y., Sermanet P., Reed S., Anguelov D., et al. (2015). Going deeper with convolutions, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.
15. Russakovsky O., Deng J., Su H., Krause J., Satheesh S., Ma S., et al. (2015). ImageNet large scale visual recognition challenge. *Int. J. Comput. Vis.* 115, 211–252. 10.1007/s11263-015-0816-y
16. Deng J., Dong W., Socher R., Li L.-J., Li K., Fei-Fei L. (2009). Imagenet: A large-scale hierarchical image database, in *Computer Vision and Pattern Recognition, 2009. CVPR 2009. IEEE Conference on*. (IEEE).
17. LeCun Y., Boser B., Denker J. S., Henderson D., Howard R. E., Hubbard W., et al. (1989). Backpropagation applied to handwritten zip code recognition. *Neural Comput.* 1, 541–551. 10.1162/neco.1989.1.4.541
18. Lin M., Chen Q., Yan S. (2013). Network in network. arXiv:1312.4400.
19. ARCHITECTURE AND BASIC PRINCIPLES OF THE MULTIFUNCTIONAL PLATFORM FOR PLANT DISEASE DETECTION / P.Goncharov, G. Ososkov, A. Nechaevskiy, A. Uzhinskiy. – Dubna, Moscow region, Russia,: Proceedings of the VIII International Conference "Distributed Computing and Grid-technologies in Science and Education", 2018.

20. Simonyan K., Zisserman A. Very deep convolutional networks for large-scale image recognition //arXiv preprint arXiv:1409.1556. –2014.
21. Szegedy C. et al. Rethinking the inception architecture for computer vision //Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. – 2016. – p. 2818-2826
22. He K. et al. Deep residual learning for image recognition //Proceedings of the IEEE conference on computer vision and pattern recognition. – 2016. – p. 770-778.
23. Chollet F. Xception: Deep learning with depthwise separable convolutions //arXiv preprint. – 2016.
24. Kingma D.P., Ba J. Adam: A method for stochastic optimization //arXiv preprint arXiv:1412.6980. – 2014.
25. Mendu M. LeafAI: A convolutional neural network based approach to plant disease diagnosis [Електронний ресурс] / Maanasa Mendu – Режим доступу до ресурсу:
<https://www.googlesciencefair.com/intl/ru/projects/2018/d223525f77451032c96dbb9057c7e088a4af2dc6e686eb8390fdd3eee17cc113>.
26. Convolutional Neural Networks for the Automatic Identification of Plant Diseases [Електронний ресурс] / J.Boulent, S. Foucher, J. Théau, J. St-Charles. – 2019. – Режим доступу до ресурсу:
<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6664047/>.
27. PlantVillage Dataset [Електронний ресурс] – Режим доступу до ресурсу: <https://plantvillage.psu.edu/>.
28. Hughes D. P., Salathé M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. arXiv:1511.08060
29. Діаграма потоків даних [Електронний ресурс] // Матеріал з Вікіпедії — вільної енциклопедії. – Режим доступу до ресурсу:
https://uk.wikipedia.org/wiki/Діаграма_потоків_даних.

30. Черемных С. В. Моделирование и анализ систем. IDEF-технологии: практикум / С. В. Черемных, И. О. Семенов, В. С. Ручкин. – Москва: "Финансы и статистика", 2006. – 191 с.
31. Нейрон [Электронный ресурс] // Матеріал з Вікіпедії — вільної енциклопедії. – Режим доступу до ресурсу: <https://uk.wikipedia.org/wiki/Нейрон>.
32. Згорткова нейронна мережа [Электронный ресурс] // Матеріал з Вікіпедії — вільної енциклопедії. – Режим доступу до ресурсу: https://uk.wikipedia.org/wiki/Згорткова_нейронна_мережа.
33. Костецкая Г.Ю., Федяев О.И. Распознавание изображений человеческих лиц с помощью свёрточной нейронной сети // Сб. трудов Международной студенческой научно-технической конференции «Информационные управляющие системы и компьютерный мониторинг» – Донецк: ДонНТУ, 2010. – с. 216-218.
34. Backpropagation Applied to Handwritten Zip Code Recognition, Neural Computation / [Y. LeCun, B. Boser, J. Denker та ін.], 1989.
35. Умяров Н.Х., Федяев О.И. Параметрическая модель свёрточной нейронной сети // VI международная научно-техническая конференция студентов, аспирантов и молодых научных работников «Информатика и компьютерные технологии»: Т. 2 – Донецк, ДонНТУ, 2010. – 292с.
36. Parmar R. Demystifying Convolutional Neural Networks [Электронный ресурс] / Ravindra Parmar. – 2019. – Режим доступу до ресурсу: <https://towardsdatascience.com/demystifying-convolutional-neural-networks-384785791596>.
37. Fukushima, Kunihiro (October 1979). "位置ずれに影響されないパターン認識機構の神経回路のモデル --- ネオコグニトロン ---" [Neural network model for a mechanism of pattern recognition unaffected by shift in position — Neocognitron —]. Trans. IECE (in Japanese). J62-A (10): 658–665.

38. Fukushima K. Neocognitron: A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position / Kunihiro Fukushima. // Biol. Cybernetics. – 1980. – №36. – С. 193–202.
39. Mittal U. Comparing and Implementing 3 Neural Models / Utkarsh Mittal. – 2017.
40. LeCun Y. Gradient-based learning applied to document recognition / Y. LeCun, Y. Bengio, P. Haffner. // Proceedings of the IEEE. – 1998. – №86(11). – С. 2278–2324.
41. Krizhevsky A. ImageNet Classification with Deep Convolutional Neural Networks / A. Krizhevsky, I. Sutskever, G. Hinton. // Communications of the ACM. – 2012. – №60(6). – С. 84–90.
42. ImageNet Large Scale Visual Recognition Challenge [Електронний ресурс] – Режим доступу до ресурсу: <http://www.image-net.org/challenges/LSVRC/>.
43. Szegedy C. Going deeper with convolutions / Christian Szegedy. – 2014.
44. Zeiler M. Visualizing and Understanding Convolutional Networks / M. Zeiler, R. Fergus. – 2013.
45. Simonyan K. VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION / K. Simonyan, A. Zisserman. // ICLR 2015. – 2015.
46. Deep Residual Learning for Image Recognition / K. He, X. Zhang, S. Ren, J. Sun. – 2015.
47. Python [Електронний ресурс] – Режим доступу до ресурсу: <https://www.python.org/>.
48. Python [Електронний ресурс] // Матеріал з Вікіпедії — вільної енциклопедії. – Режим доступу до ресурсу: <https://uk.wikipedia.org/wiki/Python>.
49. Преимущества Python перед другими языками программирования [Електронний ресурс] // Digest pro – Режим доступу до ресурсу: <https://www.digest.pro/news/preimushhestva-python-pered-drugimi-jazykami-programmirovaniya/>.

50. Matplotlib: Visualization with Python [Электронный ресурс] – Режим доступа до ресурсу: <https://matplotlib.org/>.

51. NumPy [Электронный ресурс] – Режим доступа до ресурсу: <https://numpy.org/>.

52. Tensorflow [Электронный ресурс] – Режим доступа до ресурсу: <https://www.tensorflow.org/>.

53. Keras: the Python deep learning API [Электронный ресурс] – Режим доступа до ресурсу: <https://keras.io/>.

54. PyTorch [Электронный ресурс] – Режим доступа до ресурсу: <https://pytorch.org/>.

55. Django [Электронный ресурс] – Режим доступа до ресурсу: <https://www.djangoproject.com/>.

56. Flask [Электронный ресурс] – Режим доступа до ресурсу: <https://flask.palletsprojects.com/en/1.1.x/>.

ДОДАТОК А Лістинг програми навчання згорткової мережі

```

def run(): #Виносимо весь код в одну єдину функцію, щоб потім її запустити

    """----- LIBRARIES ----- """
    import torch
    import torch.nn as nn
    import torch.optim as optim
    import numpy as np
    import torchvision
    from torchvision import datasets, models, transforms
    import matplotlib.pyplot as plt
    import time
    import os
    import copy
    import datetime

    print("PyTorch Version: ",torch.__version__)
    print("Torchvision Version: ",torchvision.__version__)
    print()

    """----- LOADING ALEXNET FROM PYTORCH ----- """
    #alexnet_model = models.alexnet(pretrained=True) #завантажуємо модель
    #print(alexnet_model) #виведення всіх параметрів моделі
    #torch.save(alexnet_model, './input/py_torch/alexnet_original_pytorch.pth') #зберігаємо оригінальну
    модель

    """----- PARAMETERS FOR ALEXNET ----- """
    data_dir = "../datasets/PlantVillage/plantvillage-dataset/segmented" #тут зберігаються фото для
    навчання і тестування
    model_name = "alexnet" #назва моделі, що буде використовуватися
    num_classes=38
    batch_size = 64 #розмір партії зображень batch_size*iterations --> 1 epoch
    num_epochs = 13 #кількість епох для тренування
    feature_extract = True # False->Finetuning, True->Feature extraction

    if(feature_extract==True): # це для назви файлу збереженої моделі
        type_of_training="feature_extract"
    else:
        type_of_training="finetuning"

    """----- HELPER FUNCTIONS ----- """
    #НАВЧАННЯ І ТЕСТУВАННЯ МОДЕЛІ
    #model - PyTorch модель (у нашому випадку Алекснет)
    #dataloaders - словники інструментів для завантаження даних
    #criterion - функція втрат
    #optimizer - оптимізатор
    #num_epochs - к-сть епох для навчання та тестування
    #is_inception - чи є дана модель моделлю inception, у якої трохи інші там параметри
    #після кожної епохи відбувається повний валідаційний етап
    def train_model(model, dataloaders, criterion, optimizer, num_epochs=25, is_inception=False):
        since = time.time()
        val_acc_history = []
        best_model_wts = copy.deepcopy(model.state_dict())
        best_acc = 0.0

        for epoch in range(num_epochs):
            print('Epoch {}/{}'.format(epoch, num_epochs - 1))
            print('-' * 10)

```

```

# Кожна епоха має фазу навчання і валідації
for phase in ['train', 'val']:
    if phase == 'train':
        model.train() # Відправляємо модель на тренування
    else:
        model.eval() # Відправляємо модель на її оцінювання

    running_loss = 0.0
    running_corrects = 0

    # Перебір даних
    for inputs, labels in dataloaders[phase]:
        inputs = inputs.to(device)
        labels = labels.to(device)

        # обнулення параметрів градієнта
        optimizer.zero_grad()

        # вперед
        # слідкувати за історією, якщо відбувається навчання
        with torch.set_grad_enabled(phase == 'train'):
            # Отримуємо вихідні дані моделі і обчислюємо втрати
            # Особливий випадок для inception, тому що при навчанні вона має допоміжний вихід

            # У режимі навчання ми обраховуємо втрату сумючи кінцевий та додатковий виходи
            # Але у режіі тестування ми використовуємо лише фінальний вихід
            if is_inception and phase == 'train':
                # Взято із https://discuss.pytorch.org/t/how-to-optimize-inception-model-with-auxiliary-classifiers/7958
                outputs, aux_outputs = model(inputs)
                loss1 = criterion(outputs, labels)
                loss2 = criterion(aux_outputs, labels)
                loss = loss1 + 0.4*loss2
            else:
                outputs = model(inputs)
                loss = criterion(outputs, labels)

            _, preds = torch.max(outputs, 1)

            # назад + оптимізуємо, якщо лише знаходимося на етапі тренування
            if phase == 'train':
                loss.backward()
                optimizer.step()

        # статистика
        running_loss += loss.item() * inputs.size(0)
        running_corrects += torch.sum(preds == labels.data)

    epoch_loss = running_loss / len(dataloaders[phase].dataset)
    epoch_acc = running_corrects.double() / len(dataloaders[phase].dataset)

    print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))

    # deep copy моделі
    if phase == 'val' and epoch_acc > best_acc:
        best_acc = epoch_acc
        best_model_wts = copy.deepcopy(model.state_dict())
    if phase == 'val':
        val_acc_history.append(epoch_acc)

'''----- ЗБЕРІГАСМО ОТРИМАНУ МОДЕЛЬ У КОЖНІЙ ЕПОСІ -----'''

```

```

        path_to_save_each_epoch='../input/py_torch/alexnet_' + str(type_of_training) +
        '_pytorch_new_'+str(epoch)+'.pth'
        torch.save(model, path_to_save_each_epoch)

    """----- ЗБЕРІГАЄМО ЧЕКПОІНТ МОДЕЛІ У КОЖНІЙ ЕПОСІ -----"""
    path_to_save_each_epoch_checkpoint='../input/py_torch/alexnet_' + str(type_of_training) +
    '_pytorch_new'+ '_CHECKPOINT_'+str(epoch)+'.pth'
    torch.save({
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'loss': loss,
    }, path_to_save_each_epoch_checkpoint)

    #виводимо поточний системний час
    currentDT = datetime.datetime.now()
    print (str(currentDT))
    print()

    time_elapsed = time.time() - since
    print("Training complete in {:.0f}m {:.0f}s".format(time_elapsed // 60, time_elapsed % 60))
    print('Best val Acc: {:.4f}'.format(best_acc))

    # Завантаження найкращих ваг моделі
    model.load_state_dict(best_model_wts)

    return model, val_acc_history

"""----- SET MODEL PARAMETERS' ----- """
def set_parameter_requires_grad(model, feature_extracting):
    if feature_extracting: #якщо виконується feature extracting
        for param in model.parameters():
            param.requires_grad = False #ставить ці всі параметри моделі у False

    #За замовчуванням, коли завантажується pretrained модель,
    #всі її параметри мають .requires_grad=True, що є добрим лише у випадку, коли
    #ми навчаємо модель з нуля (training from scratch) чи робимо finetuning.
    #Однак, коли ми виконуємо feature extracting і лише хочемо обчислити градієнт для
    #щойно ініціалізованого нового шару, тоді ми хочемо,
    #щоб інші параметри не вимагали градієнтів

    """----- INITIALIZE AND RESHAPE THE NETWORK ----- """
    #Необхідно перевизначити к-сть виходів моделі для нашої задачі,
    #адже кожна pretrained модель була навчена на датасеті Imagenet, тому має 1000 виходів
    #Ціль цього блоку кода - reshape останній шар ШНМ під к-сть класів для нашої задачі

    #!!! РІЗНИЦЯ МІЖ FINETUNING TA FEATURE-EXTRACTION
    #--- Коли виконується feature extracting, ми хочемо лише оновити параметри останнього шару,
    #тобто, ми хочемо лише оновити параметри тих шарів, що ми reshape.
    #Тобто, нам не треба обчислювати градієнти тих параметрів, що ми не змінюємо
    #тому, ми визначаємо .requires_grad attribute як False.
    #І тоді, коли ми ініціалізуємо новий шар, за замовчуванням, нові параметри
    #мають .requires_grad=True, тому лише параметри нового шару будуть оновлятися
    #--- Коли ж ми виконуємо finetuning, ми можемо залишити всі параметри .required_grad як True

    #AlexNet потребує на вході розмір зображення 224*224

    def initialize_model(model_name, num_classes, feature_extract, use_pretrained=True):

```

```

model_ft = None
input_size = 0

if model_name == "alexnet":
    model_ft = models.alexnet(pretrained=use_pretrained) #використовуємо pretrained модель
    set_parameter_requires_grad(model_ft, feature_extract)
    num_fts = model_ft.classifier[6].in_features
    model_ft.classifier[6] = nn.Linear(num_fts,num_classes) #задаємо к-сть класів
    input_size = 224
else:
    print("Invalid model name, exiting...")
    exit()

return model_ft, input_size

#ІНІЦІАЛІЗУЄМО МОДЕЛЬ ДЛЯ ЗАПУСКУ
model_ft, input_size = initialize_model(model_name, num_classes, feature_extract,
use_pretrained=True)
#Виводимо модель, яку ми тільки що створили на екран
print(model_ft)
print()

"""----- LOAD DATA -----"""
#Моделі, що були pretrained від PyTorch, були жорстко закодовані під певні значення
нормалізації!
#Описано тут: https://pytorch.org/docs/master/torchvision/models.html

# Аугментація даних та їх нормалізація для навчання
# Просто нормалізація для перевірки
data_transforms = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(input_size),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
    'val': transforms.Compose([
        transforms.Resize(input_size),
        transforms.CenterCrop(input_size),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
}

print("Initializing Datasets and Dataloaders...")
print()

# Створюємо датасети для навчання та валідації
image_datasets = {x: datasets.ImageFolder(os.path.join(data_dir, x), data_transforms[x]) for x in
['train', 'val']}
# Створюємо завантажувачі даних для навчання та валідації
dataloaders_dict = {x: torch.utils.data.DataLoader(image_datasets[x], batch_size=batch_size,
shuffle=True, num_workers=4) for x in ['train', 'val']}

# Визначаємо, чи є доступний GPU / CPU
#device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
device="cpu" #відправляємо на CPU
print("Device: ", device)
print()

"""----- CREATE THE OPTIMIZER -----"""
#Створення оптимізатора, що буде оновлювати бажані параметри

```

```

#Тепер, всі параметри, що мають .requires_grad=True, мають бути оптимізовані
#Тому, створюємо список параметрів та передаємо їх в алгоритм SGD

# Відправляємо модель на GPU / CPU
model_ft = model_ft.to(device)

# Збираємо параметри, які будуть оптимізовані/оновлені у цьому запуску.
# Якщо ми finetuning, то ми оновлюємо всі параметри. Однак, якщо
# ми використовуємо метод feature extract, ми лише оновлюємо параметри, що
# ми щойно ініціалізували, тобто, параметри із requires_grad, що рівні True
params_to_update = model_ft.parameters()
print("Params to learn:")
if feature_extract:
    params_to_update = []
    for name,param in model_ft.named_parameters():
        if param.requires_grad == True:
            params_to_update.append(param)
            print("\t",name)
else:
    for name,param in model_ft.named_parameters():
        if param.requires_grad == True:
            print("\t",name)
print()

# Observe that all parameters are being optimized
optimizer_ft = optim.SGD(params_to_update, lr=0.001, momentum=0.9)

"----- RUN TRAINING AND VALIDATION STEP -----"
#learning rate не є оптимальним для всіх pretrained моделей, тому його треба підбирати
# Здаємо loss fxn
criterion = nn.CrossEntropyLoss()

#Тренування та оцінювання моделі
model_ft, hist = train_model(model_ft, dataloaders_dict, criterion, optimizer_ft,
num_epochs=num_epochs, is_inception=False)

#!!! Збереження найкращої моделі із всіх, що були створені після кожної епохи
torch.save(model_ft, './input/py_torch/alexnet_'+_pytorch_new_last.pth) #зберігаємо отриману
модель

#Побудова графіку
ohist = []
ohist = [h.cpu().numpy() for h in hist]
plt.title("Validation Accuracy vs. Number of Training Epochs")
plt.xlabel("Training Epochs")
plt.ylabel("Validation Accuracy")
plt.plot(range(1,num_epochs+1),ohist,label="Pretrained")
plt.ylim((0,1.))
plt.xticks(np.arange(1, num_epochs+1, 1.0))
plt.legend()
plt.show()

"----- LAUNCHING A PROGRAM -----"
if __name__ == '__main__': #ЦЕЙ МЕЙН ТРЕБА ЛИШЕ ДЛЯ ВІНДОВС, ДЛЯ ЛІНУКС НЕ ТРЕБА,
ЦЕ ВСЕ ПОВ'ЯЗАНО ІЗ РОЗПАРАЛЕЛЕННЯМ ОБЧИСЛЕНЬ
    import datetime #дублюємо бібліотеку для виведення часу
    current_DT = datetime.datetime.now() #виводимо поточний системний час
    print (str(current_DT))
    print()

```

```
run() # запуск всього коду
```

```
"""----- RUN ALEXNET ON GPU ----- """
```

```
#alexnet_model.cuda() #відправка АлексНет на графічний процесор (GPU)
```

```
#print(torch.cuda.is_available()) #чи доступний GPU для роботи із PyTorch
```

```
#print(next(alexnet_model.parameters()).is_cuda) #перевірка, чи параметри моделі розташовані на GPU
```

ДОДАТОК Б Лістинг програми створення Flask-додатку

```

"""----- FLASK LIBRARIES ----- """
import os
import datetime
import flask
from flask import Flask, render_template, request, redirect, flash, url_for
print("• Flask Version: ", flask.__version__)

"""----- PYTORCH LIBRARIES ----- """
import torch
from torchvision import datasets, models, transforms
from PIL import Image
import os
import torch.nn.functional as nnf

"""----- MYSQL LIBRARIES ----- """
import mysql.connector
from mysql.connector import Error

"""----- OTHER LIBRARIES ----- """
from collections import Counter
import matplotlib.pyplot as plt

number_of_classes=38

"""----- MYSQL FUNCTIONS ----- """
#ПІДКЛЮЧЕННЯ ДО MYSQL
def connect_to_mysql_database():
    try:
        connection = mysql.connector.connect(host='localhost',
                                             database='mydb',
                                             user='root',
                                             password='Diploma2020',
                                             use_pure=True)

        db_Info = connection.get_server_info()

        print("• Connected to MySQL Server version ", db_Info)
        cursor = connection.cursor()
        cursor.execute("select database();")
        record = cursor.fetchone()
        print("• You're connected to database: ", record)
        print("• DB connection = ", connection)
        print()
        return connection

    except Error as e:
        print("ERROR WHILE CONNECTING TO MySQL: ", e)
        print()
        raise SystemExit()

"""----- PYTORCH FUNCTIONS ----- """
#ПЕРЕТВОРЕННЯ ЗОБРАЖЕННЯ НА НОРМАЛІЗОВАНЕ
transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],

```

```

std=[0.229, 0.224, 0.225]
))

#СПИСОК КЛАСІВ - із текстового файлу зчитування
def find_classes(directory):
    f = open(directory, "r")
    classes = list(f)
    classes.sort()
    class_to_idx = {classes[i]: i for i in range(len(classes))}
    return classes, class_to_idx

#КЛАСИФІКАЦІЯ ЗОБРАЖЕННЯ
def image_classification(pth):
    img = Image.open(pth)
    img_t = transform(img) #трансформування зображення
    img_t = img_t.unsqueeze(0) #додаємо 4 вимір - розмір батчу
    #img.show()
    model_path = "D:\\A_PIDRUCHNYKY\\4kurs\\Diploma\\input\\25-04-2020\\40e_128bs_feF\\py_torch_1\\alexnet__pytorch_new_last.pth"
    test_model = torch.load(model_path)
    test_model.eval()

    #ВАРІАНТ КЛАСИФІКАЦІЇ А
    output=test_model(img_t)
    prob = nnf.softmax(output, dim=1)
    top_p, top_class = prob.topk(1, dim = 1)
    res_class = top_class.item()
    res_probability = round(top_p.item()*100,3)

    cl, dict_of_classes = find_classes("D:\\A_PIDRUCHNYKY\\4kurs\\Diploma\\code\\Web_application\\list_of_classes\\list_of_classes.txt")
    class_name=(list(dict_of_classes.keys()))[list(dict_of_classes.values()).index(res_class)]
    print()
    print(".")
    print("RESULT OUTPUT TO PYTHON COMMAND LINE: ")

    print("So, with the probability " + str(res_probability) + "% " + "the class number is = " + str(res_class) + ".")
    print("Moreover, the class name is " + str(class_name) + ".")
    print()

    return str(res_probability), str(res_class), str(class_name)

"""----- OTHER FUNCTIONS ----- """
#сортування
def s(val):
    return val[-1]

"""----- CONFIG ----- """
app = Flask(__name__) #створюємо додаток через Flask
app.secret_key = "\xd0@\x10\x19\xd9\xae\xe9(\xac\xba\xb8:\x17\x9dho" #секретний ключ для додатку
image_path="" #глобальна змінна, що зберігатиме шлях до оброблюваного зображення
my_date_time="" #глобальна змінна для зберігання часу та дати, коли було класифіковане зображення
для запису у історію БД
my_connection = connect_to_mysql_database() #підключаємося до MYSQL

"""----- FLASK FUNCTIONS ----- """

##### СТАРТОВА СТОРІНКА #####

```

```

@app.route("/")
def Home():
    return render_template('HomePage.html')

@app.route("/HomePage.html")
def Home2():
    return render_template('HomePage.html')

##### MAIN-1 #####
@app.route("/Main1Page.html")
def Main1Page():
    return render_template('Main1Page.html')

#UPLOAD AN IMAGE
@app.route("/Main1Page.html", methods = ["POST"])
def upload_image():
    if request.method == "POST":
        file = request.files["file"]
        if (file.filename != ""):
            #куди будемо зберігати завантажене зображення

my_path='D:\\A_PIDRUCHNYKY\\4kurs\\Diploma\\code\\Web_application\\static\\uploaded_images'

        #змінюємо назву зображення на таку, яка мене влаштовує, а не така, яка вона була з початку
        old_name = file.filename
        name, ext = old_name.rsplit('.', 1) #розбиваємо назву зображення на власне назву і розширення
        зображення з кінця і тільки один раз (тобто назва.розширення)

        #назва зображення складається із дати і часу, коли воно було завантажено
        my_date=str(datetime.datetime.now().date())
        my_time=str(datetime.datetime.now().time())

        #також зберігаємо повне значення часу та дати, для подальшого запису у історію БД
        global my_date_time
        my_date_time = my_date + " " + my_time

        #робимо потрібне форматування для того, щоб із дати та часу створити нову назву
        завантаженого зображення
        my_time = my_time.replace(":", "-") #замінюємо непотрібні символи на бажані
        my_time = my_time.replace(".", "-") #замінюємо непотрібні символи на бажані
        result = my_date + "___" + my_time #створюємо бажану назву зображення

        #утворюємо нову назву + додаємо розширення зображення
        new_name = result + "." + ext

        #зберігаємо нову назву зображення
        file.filename = new_name

        #зберігаємо зображення
        global image_path
        image_path = my_path + "\\"+file.filename
        file.save(image_path)
        return redirect('/Main2Page.html') #після завантаження фото перейти до його обробки та
        класифікації
    else:
        flash('no image selected for uploading. please, choose an image!', "warning")
        return redirect(request.url)

##### MAIN-2 #####
@app.route("/Main2Page.html")
def Main2Page():
    #ймовірність, номер класу та назва класу після класифікації зображення

```

```

prob, class_num, class_name = image_classification(image_path)

#обрізка шляху до папки "static"
cropped_path=image_path[image_path.find("static"): ]
cropped_path=cropped_path.replace("\\", "/")
print(cropped_path)

'''----- QUERY -----'''
#ЗАПИТ ДО MYSQL: повертає інформацію про хворобу (якщо така є) за номером класу
if (my_connection): #якщо існує з'єднання, то робимо запит
    #print("MEOW")
    my_select = "SELECT CLASS.class_id, DISEASE.* "
    my_from = "FROM DISEASE, CLASS "
    my_where = "WHERE (DISEASE.disease_id = CLASS.DISEASE_disease_id) and"
    (CLASS.class_id="+str(class_num)+")"
    my_query = my_select + my_from + my_where
    cursor = my_connection.cursor()
    cursor.execute(my_query)
    records = cursor.fetchall()
    print("Total number of rows is: ", cursor.rowcount) #має бути один рядок максимум, бо класифікує і
    повертає лише одну хворобу

    if (cursor.rowcount>0): #якщо існує запис
        is_disease=True
        print("_____is_disease_____ = ", is_disease)
        my_class_id = records[0][0]
        my_disease_id = records[0][1]
        my_disease_name = records[0][2]
        my_pathogen = records[0][3]
        my_disease_description = records[0][4]
        print("• Class id = ", my_class_id)
        print("• Disease id = ", my_disease_id)
        print("• Disease name = ", my_disease_name)
        print("• Pathogen = ", my_pathogen)
        print("• Disease description = ", my_disease_description)
        print()

        #ЗАПИТ: також, дізнаємося інформацію про лікування: інформація про весь препарат,
        інструкція до застосування
        my_select = "SELECT DRUG.drug_name, DRUG.pharma_company_name,"
        DRUG.drug_description, DRUG_and_DISEASE.instruction_description "
        my_from = "FROM DRUG, DRUG_and_DISEASE, DISEASE "
        my_where = "WHERE DISEASE.disease_id = " + str(my_disease_id) + " and DISEASE.disease_id"
        = DRUG_and_DISEASE.DISEASE_disease_id and DRUG_and_DISEASE.DRUG_drug_id =
        DRUG.drug_id "
        my_query = my_select + my_from + my_where
        cursor = my_connection.cursor()
        cursor.execute(my_query)
        my_records = cursor.fetchall()
        print("Total number of rows is: ", cursor.rowcount)
        #for row in my_records:
            #print ("♦",row)
        print()

        #ЗАПИТ: робимо запис у історію БД
        my_insert = "INSERT INTO CASE_OF_THE_DISEASE (case_date_time, probability,"
        CLASS.class_id)"
        my_values = "VALUES (" + str(my_date_time) + "," + str(prob) + "," + str(my_class_id) + ");"
        my_query = my_insert + my_values
        cursor = my_connection.cursor()

```

```

cursor.execute(my_query)
my_connection.commit() #ОБОВ'ЯЗКОВО!! Інакше зміни не будуть виконані

return render_template('Main2Page.html',
                        prob=prob,
                        disease_name=my_disease_name,
                        pathogen=my_pathogen,
                        disease_description=my_disease_description,
                        records = my_records,
                        is_disease=is_disease,
                        src1=cropped_path)

else: #ЗАПИТ: якщо запис у бд відсутній, тобто, рослина здорова,
    is_disease=False
    print("_____is_disease_____ = ", is_disease)
    #то вибираємо назву класу просто і повертаємо результат
    my_select = "SELECT class_id, alias_class_name "
    my_from = "FROM CLASS "
    my_where = "WHERE class_id = '"+class_num+"'"
    my_query = my_select + my_from + my_where
    cursor = my_connection.cursor()
    cursor.execute(my_query)
    records = cursor.fetchall()
    print("Total number of rows is: ", cursor.rowcount) #має бути один запис
    my_class_id = records[0][0]
    my_class_alias_name = records[0][1]
    print("• Class id = ", my_class_id)
    print("• Class alias name = ", my_class_alias_name)

    #ЗАПИТ: робимо запис у історію БД
    my_insert = "INSERT INTO CASE_OF_THE_DISEASE (case_date_time, probability,
CLASS_class_id)"
    my_values = "VALUES ('"+str(my_date_time)+"', '"+str(prob)+"', '"+str(my_class_id)+"');"
    my_query = my_insert + my_values
    cursor = my_connection.cursor()
    cursor.execute(my_query)
    my_connection.commit() #ОБОВ'ЯЗКОВО!! Інакше зміни не будуть виконані

    return render_template('Main2Page.html',
                            prob=prob,
                            class_alias_name = my_class_alias_name,
                            is_disease=is_disease,
                            src1=cropped_path)

"""_____"""

##### HISTORY #####
@app.route("/HistoryPage.html")
def HistoryPage():
    #ЗАПИТ ДО MYSQL: ПОВЕРТАЄ ВСІ ЗАПИСИ ПРО ВИПАДКИ КЛАСИФІКАЦІЇ
    ЗАХВОРЮВАННЯ І ЗДОРОВИХ РОСЛИН
    if (my_connection): #якщо існує з'єднання, то робимо запит

        #виводимо всю БД із історією
        my_select = "SELECT CASE_OF_THE_DISEASE.case_id,
CASE_OF_THE_DISEASE.case_date_time, CASE_OF_THE_DISEASE.probability,
CLASS.alias_class_name "
        my_from = "FROM CASE_OF_THE_DISEASE, CLASS "
        my_where = "WHERE CASE_OF_THE_DISEASE.CLASS_class_id = CLASS.class_id "

```

```

my_query = my_select + my_from + my_where
cursor = my_connection.cursor()
cursor.execute(my_query)
my_records = cursor.fetchall()
print("Total number of rows is: ", cursor.rowcount)
#for row in my_records:
    #print (row)

return render_template('HistoryPage.html', records = my_records)

##### TREATMENT #####
@app.route("/Treatment1Page.html")
def Treatment1Page():
    #ЗАПИТ: на отримання інформації для вибору у випадковому спискові
    if (my_connection): #якщо існує з'єднання, то робимо запит

        #ЗАПИТ: ЛИШЕ ВИПАДКИ, ЩО МІСТИЛИ ЗАХВОРЮВАННЯ
        my_select = "SELECT CASE_OF_THE_DISEASE.case_id,
CASE_OF_THE_DISEASE.case_date_time,CASE_OF_THE_DISEASE.probability,
CLASS.alias_class_name, CLASS.DISEASE_disease_id "
        my_from = "FROM CASE_OF_THE_DISEASE, CLASS "
        my_where = "WHERE CASE_OF_THE_DISEASE.CLASS_class_id = CLASS.class_id and
(CLASS.DISEASE_disease_id IS NOT NULL) "
        my_order = "ORDER BY CASE_OF_THE_DISEASE.case_id ASC "
        my_query = my_select + my_from + my_where + my_order
        cursor = my_connection.cursor()
        cursor.execute(my_query)
        my_records_1 = cursor.fetchall()
        print("Total number of rows is: ", cursor.rowcount)
        #for row in my_records_1:
            #print (row)

        #ЗАПИТ: ВСІ МОЖЛИВІ ПРЕПАРАТИ
        my_select = "SELECT DRUG.drug_id, DRUG.drug_name, DRUG.pharma_company_name "
        my_from = "FROM DRUG "
        my_query = my_select + my_from
        cursor = my_connection.cursor()
        cursor.execute(my_query)
        my_records_2 = cursor.fetchall()
        print("Total number of rows is: ", cursor.rowcount)
        #for row in my_records_2:
            #print (row)

    return render_template("Treatment1Page.html", records_1 = my_records_1, records_2 = my_records_2)

#GET DATA FROM FORM IN TREATMENT-1
@app.route("/Treatment1Page.html", methods=['GET', 'POST'])
def get_form_data():
    if request.method == "POST":
        #Отримуємо результат вибору із випадкового списку (вибір випадку захворювання)
        get_selected_case = str(request.form.get("select_1"))
        #Отримуємо результат вибору із випадкового списку (Отримуємо обраний препарат)
        get_selected_drug = str(request.form.get("select_2"))
        #Отримуємо опису про treatment result
        get_selected_treatment_result = str(request.form.get("text_area"))

        #Якщо всі ячейки заповнені (непорожні)
        if(get_selected_case!=" and get_selected_drug!=" and get_selected_treatment_result!="):
            print("-----")
            get_selected_case = get_selected_case.split("/") #розбиваємо запис на окремі складові
            get_selected_drug = get_selected_drug.split("/") #розбиваємо запис на окремі складові

            my_selected_case_id = get_selected_case[0]

```

```

my_selected_disease_id_1 = get_selected_case[-1]
print(my_selected_disease_id_1)
print(type(my_selected_disease_id_1))

my_selected_drug_id = get_selected_drug[0]

#ШУКАЄМО ЗА ВІДПОВІДНИМ ПРЕПАРАТОМ СПИСОК ВСІХ ХВОРОБ, ЩО ВІН ЛІКУЄ
if (my_connection): #якщо існує з'єднання, то робимо запит
    my_select = "SELECT DRUG_and_DISEASE.DISEASE_disease_id "
    my_from = "FROM DRUG, DRUG_and_DISEASE "
    my_where = "WHERE DRUG.drug_id = '"+str(my_selected_drug_id)+"' and
DRUG.drug_id=DRUG_and_DISEASE.DRUG_drug_id "
    my_query = my_select + my_from + my_where
    cursor = my_connection.cursor()
    cursor.execute(my_query)
    my_records_3 = cursor.fetchall()
    print("Total number of rows is: ", cursor.rowcount)
    for i in range(len(my_records_3)):
        my_records_3[i] = my_records_3[i][0]
    print(my_records_3)

#Якщо ІД хвороби випадку не існує у списку всіх хвороб, що може лікувати даний препарат
if (int(my_selected_disease_id_1) not in my_records_3):
    flash("this drug is not suitable for this disease. choose another one. try again", "warning")
    return redirect(request.url)

#Якщо хвороба із випадку існує у списку хвороб, які лікує препарат, то записуємо
else:
    print(get_selected_case)
    print(get_selected_drug)
    print(get_selected_treatment_result)

    current_date=str(datetime.datetime.now().date())
    current_time=str(datetime.datetime.now().time())

    #також зберігаємо повне значення часу та дати, для подальшого запису у історію БД
    current_date_time=current_date + " "+current_time
    print("current_date_time = ", current_date_time)
    print("-----")

    #ЗАПИТ: запис даних до БД
    my_insert = "INSERT INTO TREATMENT_HISTORY (treatment_date_time,
CASE_OF_THE_DISEASE_case_id, DRUG_drug_id, treatment_result) "
    my_values = "VALUES('"+str(current_date_time)+"','"+str(my_selected_case_id)+"','"+str(my_selected_drug_id)+"','"+str
(get_selected_treatment_result)+"'); "
    my_query = my_insert + my_values
    cursor = my_connection.cursor()
    cursor.execute(my_query)
    my_connection.commit() #ОБОВ'ЯЗКОВО!! Інакше зміни не будуть виконані

    #Повідомляємо користувачу, що запис відбувся
    flash("a record was successfully added to treatment history", "success")
    return redirect("/Treatment1Page.html")

else:
    flash("not all not all items are filled / selected. try again", "warning")
    return redirect(request.url)

@app.route("/Treatment2Page.html")
def Treatment2Page():
    if (my_connection): #якщо існує з'єднання, то робимо запит
        my_select = "SELECT TREATMENT_HISTORY.treatment_id, \

```

```

        TREATMENT_HISTORY.treatment_date_time, \
        TREATMENT_HISTORY.CASE_OF_THE_DISEASE_case_id, \
        CASE_OF_THE_DISEASE.case_date_time, \
        CASE_OF_THE_DISEASE.probability, \
        CLASS.alias_class_name, \
        DRUG.drug_name, \
        DRUG.pharma_company_name, \
        TREATMENT_HISTORY.treatment_result "
my_from = "FROM TREATMENT_HISTORY, CASE_OF_THE_DISEASE, CLASS, DRUG "
my_where = "WHERE TREATMENT_HISTORY.CASE_OF_THE_DISEASE_case_id =
CASE_OF_THE_DISEASE.case_id and \
        CASE_OF_THE_DISEASE.CLASS_class_id = CLASS.class_id and \
        TREATMENT_HISTORY.DRUG_drug_id = DRUG.drug_id "
my_query = my_select + my_from + my_where
cursor = my_connection.cursor()
cursor.execute(my_query)
my_records = cursor.fetchall()
print("Total number of rows is: ", cursor.rowcount)
#for row in my_records:
    #print (row)

return render_template("Treatment2Page.html", records = my_records)

##### STATISTICS #####
@app.route("/Statistics1Page.html")
def Statistics1Page():
    if (my_connection): #якщо існує з'єднання, то робимо запит
        #ЗАПИТ: ВСІ МОЖЛИВІ ЗАПИСИ У ТАБЛИЦІ ВИПАДКІВ ЗАХВОРЮВАННЯ
        my_select = "SELECT * "
        my_from = "FROM CASE_OF_THE_DISEASE "
        my_query = my_select + my_from
        cursor = my_connection.cursor()
        cursor.execute(my_query)
        my_records_1 = cursor.fetchall()
        print("Total number of rows is: ", cursor.rowcount)
        #for row in my_records_1:
            #print (row)

        #ВИЗНАЧАЄМО ЧАСТОТУ КОЖНОГО КЛАСУ
        class_id = [class_id[-1] for class_id in my_records_1] #виписуємо всі останні значення рядочка
        (номер класу)
        c = Counter(class_id) #Отримуємо частотний словник
        #print("c = ", c)

        c = dict(c) #конвертуємо у словник

        #Додаємо ті класи, які іще не зустрічалися у ВИПАДКАХ ЗАХВОРЮВАННЯ (частота = 0)
        for i in range(number_of_classes):
            if i not in c.keys():
                c_new={i:0}
                dict.update(c,c_new)

        #print ("c_new = ", c)

        c = list(sorted(c.items(), reverse=False)) #сортуємо у порядку зростання
        #print (c)

        my_classes=[] #список класів
        my_frequencies=[] #список відповідних частот

        #Створюємо два масиви - клас/частота
        for i in range(len(c)):

```

```

        my_classes.append(c[i][0])
        my_frequencies.append(c[i][1])

#ЗАПИТ: знаходимо назви класів
my_select = "SELECT CLASS.class_id, CLASS.alias_class_name "
my_from = "FROM CLASS "
my_query = my_select + my_from
cursor = my_connection.cursor()
cursor.execute(my_query)
my_records_2 = cursor.fetchall()
#for row in my_records_2:
    #print (row)
'''

#Замінюємо номери класів на назви класів
for i in range(len(my_classes)):
    my_classes[i]=my_records_2[i][1]
'''

print(my_classes)
print(my_frequencies)

plt.figure(figsize=(16, 9))
ax = plt.gca()
ax.set_title('CLASSES FREQUENCY IN CASES', fontsize=30, fontname="Times New Roman")
ax.set_xlabel('class', fontsize=30, fontname="Times New Roman")
ax.set_ylabel('frequency', fontsize=30, fontname="Times New Roman")

x=range(len(my_frequencies))

ax.bar(x, my_frequencies, align='center', width=1, color='green', edgecolor='black')
ax.set_xticks(x)
labels = [labels[-1] for labels in my_records_2]
ax.set_xticklabels(my_classes, rotation='vertical')

#plt.grid()
path1
path2 = '/static/images/classes_histogram.png'
plt.savefig(path1, transparent=True)
#plt.show()

return render_template('Statistics1Page.html', path=path2, records=my_records_2)

#BUILD A HISTOGRAM
@app.route("/Statistics1Page.html", methods=['GET', 'POST'])
def BuildHistogram():
    #if request.method == "POST":
        return redirect("/Statistics1Page.html")

##### ABOUT #####
@app.route("/AboutPage.html")
def AboutPage():
    return render_template('AboutPage.html')

##### PRIVACY POLICY #####
@app.route("/PrivacyPolicyPage.html")
def PrivacyPolicyPage():
    return render_template('PrivacyPolicyPage.html')

'''----- MAIN -----'''
if __name__ == "__main__":
    app.run(host='0.0.0.0')

```